ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

HONORIS UNITED UNIVERSITIES

AZNet Security & Networking

Rapport de stage

5ème année

Ingénierie Informatique et Réseaux

AUTOMATISATION DES RAPPORTS JOURNALIERS DE CHANTIER PAR INTELLIGENCE ARTIFICIELLE

Transcription de notes vocales en darija et extraction structurée

Réalisé par :
Mohammed Yassine Magri

Encadré par :
Encadrante Académique: Mme Kawtar Imgharene
Encadrants Industriels: MM. Adil Chaachoui et Adil Zellou

Entreprise : AZNet
Activité : Réseaux, cybersécurité, sécurité physique, communication, câblage(depuis 2000).

Remerciements

Je tiens tout d'abord à exprimer ma gratitude à mon encadrante académique, Mme Kawtar Imgharene, pour son accompagnement, sa disponibilité et ses conseils méthodologiques tout au long de ce projet de fin d'études.

Je remercie également mes encadrants industriels, MM. Adil Chaachoui et Adil Zellou, ainsi que l'ensemble de l'équipe d'AZNET, pour leur accueil, leur confiance et le partage de leur expérience du terrain, qui ont permis d'ancrer ce projet dans les besoins réels des équipes de câblage.

Mes remerciements vont aussi au corps professoral de l'EMSI, dont les enseignements ont constitué le socle technique et scientifique de ce travail.

Enfin, je remercie ma famille et mes proches pour leur soutien constant durant l'ensemble de mon cursus.

Résumé

Dans le secteur du câblage réseau, le suivi de l'activité des techniciens repose encore largement sur des comptes rendus oraux ou manuscrits, hétérogènes et difficiles à exploiter. Ce projet de fin d'études, réalisé au sein de l'entreprise AZNET, automatise la production des rapports journaliers de chantier. Le technicien envoie, en fin de journée, une simple note vocale en arabe marocain (darija) — ou une photo de relevé — via Telegram ou WhatsApp ; le système transcrit l'audio, en extrait les données métier typées (mètres de goulotte, de câble préparé et installé, prises bureau, prises panneau, prises testées, mètres de fibre, interconnexions, brins optiques, étages et ailes concernés, stock, remarques), reformule le tout en un résumé professionnel en français, écrit la ligne dans le journal de chantier Google Sheets correspondant et notifie l'encadrement.

L'architecture retenue est hybride : une plateforme d'orchestration low-code (n8n, 64 nœuds) assure la réception multicanale et les intégrations, tandis qu'un microservice développé en Python/FastAPI (85 routes, 15 380 lignes) prend en charge la transcription, l'extraction structurée par grands modèles de langage, la reformulation, la tenue du journal, l'ordonnancement des tâches périodiques et le service du tableau de bord. Un tableau de bord React (16 139 lignes) trilingue français/anglais/arabe complète l'ensemble. La méthodologie suivie est une démarche agile de type Scrum allégé.

Un banc d'essai comparatif de **treize** moteurs de transcription du darija — trois par API et douze en local sur GPU — a été conduit et a renversé l'hypothèse initiale : la **référence d'exactitude** n'est pas le service commercial de référence, mais un modèle open-source affiné sur le darija, `BounharAbdelaziz/Morocco-Darija-STT-large`, seul à préserver simultanément les quantités numériques et les emprunts techniques français dont dépend l'intégralité du schéma métier. Son coût — 7,7 s et 4 Go de mémoire graphique — lui vaut en production la position de repli au sein d'une chaîne de trois moteurs, arbitrage explicité au §4.3.5. La chaîne complète, de la note vocale à la réponse de confirmation, est opérationnelle depuis la phase de développement du cœur IA, une phase avant la date prévue.

Mots-clés : reconnaissance vocale, darija, grands modèles de langage, extraction d'information, automatisation de workflows, langue peu dotée.

Abstract

In the network cabling industry, tracking technicians' daily activity still relies heavily on oral or handwritten reports, which are inconsistent and hard to exploit. This graduation project, carried out at AZNET, automates the production of daily site reports. At the end of the day a technician sends a plain voice note in Moroccan Arabic (Darija) — or a photo of a hand-written tally — through Telegram or WhatsApp; the system transcribes the audio, extracts typed business data (metres of cable tray, of prepared and installed cable, office sockets, patch-panel sockets, tested sockets, metres of fibre, interconnections, optical strands, floors and wings involved, stock, remarks), rewrites everything into a professional French summary, writes the row into the matching Google Sheets site journal, and notifies management.

The chosen architecture is hybrid: a low-code orchestration platform (n8n, 64 nodes) handles multi-channel reception and integrations, while a custom Python/FastAPI microservice (85 routes, 15,380 lines) performs transcription, LLM-based structured extraction, rewriting, journal bookkeeping, job scheduling and dashboard serving. A trilingual French/English/Arabic React dashboard (16,139 lines) completes the system. The project follows a lightweight Scrum methodology.

A comparative benchmark of **thirteen** Darija speech-to-text engines — three via API and twelve locally on GPU — was carried out and overturned the initial hypothesis: the **accuracy reference** is not the leading commercial service but an open-source Darija fine-tune, `BounharAbdelaziz/Morocco-Darija-STT-large`, the only one preserving both numeric quantities and the French technical loanwords on which the entire business schema depends. Its cost — 7.7 s and 4 GB of GPU memory — places it in production as the fallback within a three-engine chain, a trade-off set out in §4.3.5. The full chain, from voice note to confirmation reply, has been operational since the core-AI phase, one phase ahead of the plan.

Keywords: speech recognition, Darija, large language models, information extraction, workflow automation, low-resource language.

ملخص

في قطاع التمديدات الشبكية، لا يزال تتبع النشاط اليومي للتقنيين يعتمد إلى حد كبير على تقارير شفوية أو مكتوبة بخط اليد، غير متجانسة ويصعب استغلالها. يقوم مشروع نهاية الدراسة هذا، المنجز داخل شركة AZNET، بأتمتة إعداد التقارير اليومية للأوراش. يرسل التقني في نهاية اليوم رسالة صوتية بسيطة بالدارجة المغربية — أو صورة لكشف مكتوب بخط اليد — عبر تيليغرام أو واتساب؛ فيقوم النظام بتحويل الصوت إلى نص، واستخراج المعطيات المهنية المصنّفة (أمتار القنوات، الكابلات المحضّرة والمركّبة، مآخذ المكاتب، مآخذ اللوحات، المآخذ المختبرة، أمتار الألياف، الوصلات البينية، الشعيرات البصرية، الطوابق والأجنحة المعنية، المخزون، الملاحظات)، ثم إعادة صياغتها في ملخص مهني بالفرنسية، وتسجيل السطر في دفتر الورش المناسب على Google Sheets وإشعار المسؤولين.

تعتمد البنية المختارة على مقاربة هجينة: منصة أتمتة منخفضة البرمجة (n8n، 64 عقدة) تتكفل بالاستقبال عبر قناتين وبالتكاملات، بينما تتولى خدمة مصغرة مطوّرة بلغة Python/FastAPI (85 مسارًا، 15380 سطرًا) تحويل الصوت إلى نص، والاستخراج المهيكل بواسطة النماذج اللغوية الكبيرة، وإعادة الصياغة، ومسك الدفتر، وجدولة المهام الدورية، وخدمة لوحة القيادة. وتكتمل المنظومة بلوحة قيادة React (16139 سطرًا) بثلاث لغات: الفرنسية والإنجليزية والعربية.

وقد أُجري اختبار مقارن لـ**ثلاثة عشر** محركًا للتعرف على الكلام بالدارجة، ثلاثة عبر واجهات برمجية واثنا عشر محليًا على بطاقة رسومية، فقلب الفرضية الأولية: المرجع في الدقة ليس الخدمة التجارية المرجعية، بل نموذج مفتوح المصدر مُدرَّب على الدارجة، `BounharAbdelaziz/Morocco-Darija-STT-large`، وهو الوحيد الذي يحافظ في آن واحد على الكميات الرقمية وعلى المصطلحات التقنية الفرنسية المقترضة التي يقوم عليها المخطط المهني بأكمله. غير أن كلفته، 7,7 ثانية وقرابة أربعة غيغابايت من ذاكرة بطاقة الرسوميات، تجعله في التشغيل الفعلي محرك الاحتياط داخل سلسلة من ثلاثة محركات، وهي مفاضلة مشروحة في القسم 4.3.5. والسلسلة الكاملة، من الرسالة الصوتية إلى رد التأكيد، تشتغل منذ مرحلة تطوير نواة الذكاء الاصطناعي، أي قبل الموعد المقرر بمرحلة كاملة.

الكلمات المفاتيح: التعرف على الكلام، الدارجة، النماذج اللغوية الكبيرة، استخراج المعلومات، أتمتة سير العمل، لغة قليلة الموارد.

Table des matières

Remerciements ii
Résumé iii
Abstract iii
ملخص iv
Table des matières v
Liste des figures ix
Liste des tableaux xi
Liste des abréviations et acronymes xii
Introduction générale 1
Chapitre 1 : Contexte général du projet 2
1.1 Introduction du chapitre 2
1.2 Présentation de l'organisme d'accueil 2
1.3 Étude de l'existant 2
1.3.1 Processus actuel 2
1.3.2 Limites identifiées 2
1.3.3 Contrainte clé : l'expression orale en darija 3
1.4 Problématique 3
1.5 Solution proposée 3
1.6 Conduite du projet 4
1.6.1 Méthodologie : Scrum allégé 4
1.6.2 Planification 4
1.6.3 Gestion des risques 5
1.7 Conclusion du chapitre 6
Chapitre 2 : État de l'art et étude technique 7
2.1 Introduction du chapitre 7
2.2 Reconnaissance vocale de l'arabe dialectal 7
2.2.1 Le darija, une langue peu dotée 7
2.2.2 Les modèles de bout en bout : Whisper 7
2.2.3 L'architecture concurrente : wav2vec2 et la classification temporelle connexionniste 7
2.2.4 Modèles affinés sur le darija 8
2.2.5 Métriques d'évaluation 8

2.3 Extraction d'information structurée par grands modèles de langage 8
2.3.1 De l'extraction classique aux LLM 8
2.3.2 Sorties structurées et fiabilité 8
2.3.3 Modèles candidats 9
2.4 Plateformes d'orchestration 9
2.5 Canal de réception : étude comparative 10
2.6 Étude comparative des solutions existantes 11
2.7 Technologies et langages retenus 12
2.7.1 Vue d'ensemble 12
2.7.2 Les choix qui méritaient un débat 14
2.8 Synthèse des choix technologiques 14
2.9 Conclusion du chapitre 15
Chapitre 3 : Conception de la solution 16
3.1 Introduction du chapitre 16
3.2 Analyse des besoins 16
3.2.1 Besoins fonctionnels 16
3.2.2 Besoins non fonctionnels 16
3.2.3 Priorisation MoSCoW 16
3.3 Architecture générale 17
3.3.1 Le choix d'une architecture hybride 17
3.3.2 Frontière entre orchestration et code 18
3.3.3 Flux nominal 18
3.3.4 Diagrammes UML 19
3.4 Modèle de données 20
3.4.1 Le rapport structuré, contrat central 20
3.4.2 Politique « aucune donnée inventée » 24
3.4.3 Le modèle relationnel : une personne, plusieurs canaux 24
3.4.4 Les tables en détail 25
3.5 Conception du service applicatif 28
3.5.1 Interface unique 28
3.5.2 Patron Stratégie pour la transcription 28
3.5.3 Chaînes de repli 29

3.5.4 Contrat d'erreurs 31
3.5.5 Les instructions comme contenu versionné 31
3.5.6 Le profil du technicien : un cinquième modèle 32
3.5.7 Le journal de chantier 32
3.5.8 Les tâches planifiées 34
3.5.9 Les avertissements : deux familles, deux destins 36
3.5.10 Ignorer une alerte sans l'effacer 37
3.5.11 La matière reçue n'est pas la matière posée 38
3.5.12 La file d'attente des messages 38
3.6 Conception du workflow d'orchestration 38
3.6.1 Structure 38
3.6.2 L'ordre réponse-d'abord 40
3.7 Conception du tableau de bord 41
3.8 Sécurité 42
3.9 Protection des données personnelles 43
3.9.1 Ce qui est collecté, et où 44
3.9.2 Les principes appliqués 44
3.9.3 Ce qui reste à faire avant une mise en production 45
3.10 Ce que lit le modèle, et ce qu'il n'a pas le droit d'en faire 45
3.11 Conclusion du chapitre 47
Chapitre 4 : Réalisation, banc d'essai, tests et résultats 49
4.1 Introduction du chapitre 49
4.2 Environnement et outils de développement 49
4.2.1 Déploiement : une seule origine 49
4.2.2 La base de connaissances du projet 50
4.3 Les bancs d'essai comparatifs 51
4.3.1 Protocole 51
4.3.2 Une décision de méthode : ni WER, ni CER 51
4.3.3 Résultats — moteurs par API 52
4.3.4 Résultats — moteurs locaux sur GPU 53
4.3.5 Le moteur de référence et sa place dans la chaîne 54
4.3.6 Une hypothèse renversée, puis rétablie 55

4.3.7 Trois bogues de harnais, dont un silencieux 56
4.3.8 L'invite de vocabulaire : survivable par API, fatale en local 56
4.3.9 Second banc d'essai : du texte au tableau 57
4.3.10 Troisième banc d'essai : de l'image au tableau 62
4.4 Difficultés techniques rencontrées et résolues 64
4.4.1 La boucle de répétition de Gemini 64
4.4.2 Validation de format par Groq 64
4.4.3 Rotation des modèles chez le fournisseur 64
4.4.4 Chaînes « null » des modèles 64
4.4.5 Une identité scindée en deux 64
4.4.6 Une migration qui se planifie avant de s'exécuter 65
4.5 Stratégie de test et intégration continue 65
4.6 Résultats 65
4.6.1 Chaîne de bout en bout opérationnelle 65
4.6.2 Périmètre livré 66
4.6.3 Métriques du projet 67
4.7 Le tableau de bord en fonctionnement 68
4.8 Analyse des coûts et des quotas 68
4.8.1 Du tableau estimé à la mesure : la page Consommation 69
4.9 Statistiques et qualité du projet 69
4.9.1 Répartition du code 69
4.9.2 Indicateurs de qualité mesurés 70
4.9.3 La chaîne d'intégration continue 71
4.9.4 Analyse statique externe 73
4.9.5 Activité du dépôt : rythme et volume 76
4.9.6 Chronologie réelle du développement 78
4.10 Conclusion du chapitre 79
Bilan et compétences acquises 79
Conclusion générale et perspectives 80
Références bibliographiques 82
Annexes 83
Annexe A : Documentation d'installation 83

A.1 Prérequis 83
A.2 Configuration 83
A.3 Démarrage de la pile 83
A.4 Développement du service 83
A.5 Deux pièges de déploiement 83
A.6 Étape manuelle après l'import du workflow 83
Annexe B : Protocole et données du banc d'essai 84
B.1 Protocole 84
B.2 Catalogue des moteurs évalués 84
B.3 Reproductibilité 85
B.4 Limite assumée 85
Annexe C : Le tableau de bord, écran par écran 86
C.1 Écrans principaux 86
C.2 Usage mobile 89

Liste des figures

Figure 3.1 : Architecture générale : les cinq responsabilités et leurs échanges. 18

Figure 3.2 : Diagramme de cas d'utilisation. 19

Figure 3.3 : Cycle de vie complet d'une note vocale, de l'émission à la ligne du journal, y compris la déduplication en amont et la mise en file de reprise en aval. 20

Figure 3.4 : Cycle de vie d'un rapport, vu comme une machine à états. 23

Figure 3.5 : Modèle relationnel. persons est au centre et se référence elle-même par manager_id : c'est l'organigramme. 25

Figure 3.6 : Les tables d'audit, seules tables d'exploitation à porter une clé étrangère : une session, une action et une visite appartiennent toutes à une personne. 25

Figure 3.7 : Chaîne de transcription. L'ordre est celui de l'exactitude mesurée, non de la latence : groq occupe la dernière place en tant que socle rapide et toujours disponible. 29

Figure 3.8 : Les chaînes de repli d'extraction. 30

Figure 3.9 : Tenue du journal : une ligne par message, réconciliation de stock et création de colonne à la demande. 34

Figure 3.10 : Ordonnanceur : neuf tâches, un fil unique, et une réclamation atomique du droit d'enregistrer le résultat — sans laquelle une tâche terminée juste au seuil du délai inscrivait deux issues contradictoires pour une même exécution. 36

Figure 3.11 : Workflow d'orchestration : les 64 nœuds regroupés par fonction. 39

Figure 3.12 : Workflow, seconde moitié : aiguillage, traitement, réponse au technicien puis journal. 40

Figure 3.13 : Contrôle d'accès. Les deux voies — cookie humain et clé machine — convergent vers trois niveaux d'exigence. 43

Figure 3.14 : Ce que lit le modèle. En bleu, les contextes injectés qui ne viennent pas du message : l'âme, l'instruction, le profil du technicien et le vocabulaire de transcription — ce dernier étant le seul à ne pas recevoir l'âme, parce que Whisper le lirait comme un début de transcription. 47

Figure 4.1 : Topologie de déploiement. Un seul service publie un port ; tout le reste n'existe que sur le réseau interne de la pile. 50

Figure 4.2 : Démarche du banc d'essai, y compris ses détours. 52

Figure 4.3 : La chaîne d'intégration continue : les deux sources de changement, les dix-huit contrôles et l'analyse statique externe avant fusion. 73

Figure 4.4 : Le tableau de bord SonarCloud du projet, capturé le 21 août 2026 sur la branche principale. 76

Figure C.1 : Vue d'ensemble. Les repères du jour, le rythme horaire des messages, l'état des sept services et l'activité de l'année. 86

Figure C.2 : Réception. Le flux des messages entrants, canal par canal, avec le média d'origine et le texte transcrit — l'écran qui permet de remonter d'une ligne douteuse à l'audio qui l'a produite. .. 86

Figure C.3 : Rapports. Le miroir plat du journal, filtrable et corrigeable ; une correction faite ici est écrite dans le classeur, pas seulement affichée. 87

Figure C.4 : Automatisation. Les neuf tâches planifiées, leur dernier résultat et leur horaire. 87

Figure C.5 : Journaux. La chronologie du pipeline reconstituée par identifiant de corrélation : les étapes publiées par n8n et les erreurs journalisées par le service apparaissent dans une seule suite. 87

Figure C.6 : Consommation des API. Groq publie son quota dans les en-têtes de chaque réponse et la carte est marquée measured ; Gemini ne publie rien, sa consommation est mesurée mais son plafond est une constante documented ; WhatsApp expose son palier d'envoi et sa qualité de numéro, le décompte des messages restant le nôtre. 88

Figure C.7 : Chantiers, la page entière. 88

Figure C.8 : Organigramme. Trois bandes fixes — directeur, chefs, techniciens — et non une récursion, ce qui est la raison pour laquelle aucun garde-fou anti-cycle n'est nécessaire. 88

Figure C.9 : Vue d'ensemble sur téléphone : la barre latérale se replie et les indicateurs passent sur une colonne. 89

Liste des tableaux

Tableau 1.1 : Planning prévisionnel du projet. 4
Tableau 1.2 : Registre des risques principaux. 5
Tableau 2.1 : Comparaison des solutions d'orchestration. 9
Tableau 2.2 : Comparaison des canaux de réception. 10
Tableau 2.3 : Positionnement face aux solutions existantes. 11
Tableau 2.4 : Technologies retenues et alternatives écartées. 12
Tableau 2.5 : Récapitulatif des choix technologiques. 14
Tableau 3.1 : Priorisation MoSCoW du périmètre fonctionnel. 16
Tableau 3.2 : Schéma du rapport extrait (`ExtractedReport`). 20
Tableau 3.3 : Contrat d'erreurs du service. 31
Tableau 3.4 : Les neuf tâches planifiées, leur cadence et leur budget de temps. 34
Tableau 3.5 : Données personnelles traitées par le système. 44
Tableau 4.1 : Environnement de développement. 49
Tableau 4.2 : Comparaison des moteurs accessibles par API 52
Tableau 4.3 : Campagne locale complète, 10 moteurs, 11 clips, RTX 3070 .Trié par confiance décroissante. 53
Tableau 4.4 : Comportement comparé sur les champs métier d'un clip de référence. 54
Tableau 4.5 : Effet de l'invite de vocabulaire en exécution locale. 56
Tableau 4.6 : Les dix cas de test de l'extraction, et le défaut que chacun cherche à provoquer. 57
Tableau 4.7 : Les quinze cas ajoutés avec les champs d'article et le chantier énoncé. 58
Tableau 4.8 : Extraction texte → rapport structuré, 8 modèles, 10 cas (27 juillet 2026). 59
Tableau 4.9 : Extraction texte → rapport structuré, 8 modèles, 20 cas (15 août 2026). 61
Tableau 4.10 : Extraction image → rapport structuré, 4 modèles, 10 cas (27 juillet 2026). 62
Tableau 4.11 : Extraction depuis des plans annotés, 6 modèles, 3 photos réelles (15 août 2026). .. 63
Tableau 4.12 : État du périmètre livré. 66
Tableau 4.13 : Métriques du projet à la date de rédaction. 67
Tableau 4.14 : Consommation des offres gratuites. 68
Tableau 4.15 : Répartition du code par langage. 70
Tableau 4.16 : Indicateurs de qualité mesurés. 70
Tableau 4.17 : Les dix-huit contrôles d'intégration continue, par famille. 71
Tableau 4.18 : Indicateurs SonarCloud du projet. 73
Tableau 4.19 : Répartition des 376 commits par type (convention *Conventional Commits*). 77
Tableau 4.20 : Déroulement constaté, comparé au planning prévisionnel. 78

Liste des abréviations et acronymes

Acronyme	Signification
API	Application Programming Interface
ASR	Automatic Speech Recognition (reconnaissance automatique de la parole)
CER	Character Error Rate (taux d'erreur caractère)
CSV	Comma-Separated Values
CTC	Connectionist Temporal Classification
E2E	End-to-End (de bout en bout)
FK	Foreign Key (clé étrangère)
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model (grand modèle de langage)
LoRA	Low-Rank Adaptation (affinage à faible rang)
MSA	Modern Standard Arabic (arabe standard moderne)
MVP	Minimum Viable Product (produit minimum viable)
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
REST	Representational State Transfer
RTL	Right-To-Left (écriture de droite à gauche)
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
STT	Speech-To-Text (transcription de la parole en texte)
TTL	Time To Live (durée de validité)
UML	Unified Modeling Language
VRAM	Video Random Access Memory (mémoire de la carte graphique)
WABA	WhatsApp Business Account
WER	Word Error Rate (taux d'erreur mot)

Introduction générale

Contexte. La transformation numérique des métiers de terrain constitue aujourd'hui un enjeu majeur pour les entreprises de services techniques. Dans le domaine du câblage réseau et des infrastructures informatiques, les équipes passent leurs journées sur les chantiers, loin des outils bureautiques, et la remontée d'information vers l'encadrement reste un maillon faible : les comptes rendus d'activité sont transmis oralement, par messages dispersés ou sur papier, quand ils ne sont pas simplement omis. L'entreprise AZNET, spécialisée dans les infrastructures réseau, souhaite fiabiliser ce processus sans alourdir la charge administrative de ses techniciens, dont la langue de travail naturelle est l'arabe marocain (darija), souvent mêlé de vocabulaire technique français.

Problématique. Comment permettre à un technicien de chantier de produire, en moins de deux minutes et sans aucune saisie complexe, un rapport journalier structuré, exploitable par l'encadrement et archivé de manière centralisée ? La difficulté est triple : (1) le canal d'entrée doit être un outil que le technicien utilise déjà — la messagerie instantanée et la note vocale ; (2) la parole spontanée en darija, dialecte peu doté en ressources de reconnaissance vocale, doit être transcrite avec une fiabilité suffisante ; (3) le texte libre obtenu doit être transformé en données structurées normalisées sans invention d'informations.

Objectifs. Ce projet vise à concevoir et réaliser une chaîne de traitement complète : réception d'une note vocale ou d'une photo via une messagerie instantanée, transcription automatique du darija, extraction des données métier par grands modèles de langage, reformulation en français professionnel, tenue automatique du journal de chantier dans Google Sheets et notification de l'encadrement. Les choix de moteurs d'intelligence artificielle doivent être justifiés par un banc d'essai comparatif, et l'ensemble doit être réalisé sous une contrainte forte : un budget nul, en s'appuyant exclusivement sur les offres gratuites des fournisseurs.

Méthodologie. Le projet est conduit selon une démarche agile de type Scrum allégé : des sprints de deux semaines, précédés d'une phase de cadrage et de neutralisation des risques d'intégration, avec démonstration à l'encadrant à chaque itération. L'architecture retenue est hybride : une plateforme d'orchestration low-code (n8n) pour les intégrations, et un microservice développé sur mesure (Python/FastAPI) pour le cœur algorithmique.

Organisation du rapport. Ce rapport s'articule en quatre chapitres. Le chapitre 1 présente l'organisme d'accueil, l'étude de l'existant, la problématique, la solution proposée et la démarche de conduite du projet. Le chapitre 2 dresse un état de l'art de la reconnaissance vocale de l'arabe dialectal et de l'extraction structurée par grands modèles de langage, puis justifie les choix technologiques. Le chapitre 3 détaille la conception de la solution : analyse des besoins, architecture générale, modèle de données, conception du service d'intelligence artificielle, du workflow d'orchestration et du tableau de bord. Le chapitre 4 décrit la réalisation, le banc d'essai, les tests et les résultats obtenus. Une conclusion générale synthétise les contributions et ouvre les perspectives d'évolution.

Chapitre 1 : Contexte général du projet

1.1 Introduction du chapitre

Ce premier chapitre situe le projet dans son environnement. Il présente l'entreprise d'accueil, analyse le processus existant de remontée d'information et ses limites, formule la problématique, esquisse la solution proposée, puis décrit la méthodologie de conduite du projet et la gestion des risques.

1.2 Présentation de l'organisme d'accueil

AZNET est une entreprise marocaine spécialisée dans les infrastructures réseau et le câblage informatique : pose de goulottes, tirage et raccordement de câbles cuivre et fibre optique, équipement de bureaux, de panneaux de brassage et de locaux techniques pour des clients professionnels. Fondée en 2000, AZ Net Security & Networking compte parmi ses références des clients tels que le Cabinet Royal, l'Université Al Akhawayn, la Fondation Mohammed VI, plusieurs ministères (Intérieur, Éducation nationale, Équipement et Eau, Habous, Économie et Finances), ainsi que des établissements comme l'ENSIAS, l'EMI, l'ENIM, l'INPT, l'ENCG, les universités Mohammed V et Hassan II, le CHIS, le CNRST et Maroc Telecom. Les équipes de techniciens interviennent quotidiennement sur plusieurs chantiers simultanés. Le suivi de l'avancement — quantités posées, zones terminées, incidents, besoins en matériel — conditionne directement la facturation, la planification et l'approvisionnement.

1.3 Étude de l'existant

1.3.1 Processus actuel

En fin de journée, chaque technicien rend compte de son activité de manière informelle : appel téléphonique au chef de chantier, message vocal ou textuel sur une messagerie personnelle, ou note manuscrite. Ces informations sont ensuite ressaisies — partiellement — par l'encadrement dans un fichier de suivi Excel, le *Tableau de Suivi projet*, organisé en un onglet par chantier.

1.3.2 Limites identifiées

Ce processus présente plusieurs faiblesses structurelles :

- **Hétérogénéité** : chaque technicien rapporte à sa manière ; les quantités sont parfois absentes ou approximatives.
- **Perte d'information** : les messages vocaux dispersés ne sont ni archivés ni centralisés ; l'historique d'un chantier est difficile à reconstituer.
- **Double saisie** : l'encadrement retranscrit manuellement les informations, tâche chronophage et source d'erreurs.
- **Latence** : les responsables ne disposent pas d'une vue consolidée le jour même.
- **Barrière linguistique de l'écrit** : la rédaction d'un rapport écrit en français représente un effort important pour des techniciens dont la langue de travail orale est la darija ; l'oral est le canal naturel.

1.3.3 Contrainte clé : l'expression orale en darija

Le point central de l'étude de l'existant est que la note vocale en arabe marocain — mêlée de termes techniques français (« goulotte », « panneau de brassage », « câble », « fibre ») — est déjà le mode de communication spontané des équipes. Toute solution imposant une saisie structurée à la source serait rejetée par l'usage. La solution doit donc s'adapter au technicien, et non l'inverse.

1.4 Problématique

La problématique se formule ainsi : comment transformer automatiquement une note vocale spontanée en darija en un rapport journalier structuré, fiable et exploitable, en moins de deux minutes d'effort pour le technicien et sans budget logiciel ?

Elle se décompose en trois verrous techniques :

1. **Transcription** : les moteurs de reconnaissance vocale sont majoritairement entraînés sur l'arabe standard moderne ; le darija est une langue peu dotée, au code-switching arabe/français fréquent.
2. **Structuration** : le texte transcrit est du langage libre ; il faut en extraire des champs normalisés — quantités numériques, localisations, observations — sans inventer de données absentes.
3. **Intégration** : la chaîne complète (messagerie, téléchargement du média, traitement IA, journal de chantier, notifications) doit fonctionner de bout en bout de manière fiable et gratuite.

1.5 Solution proposée

La solution retenue renverse la charge : plutôt que de demander au technicien de saisir un rapport, le système part de ce qu'il fait déjà spontanément — une note vocale en darija envoyée depuis sa messagerie — et se charge seul du reste. Aucune application à installer, aucun formulaire à remplir, aucune langue d'écriture imposée.

La chaîne de traitement envisagée enchaîne cinq maillons :

- **Réception** : le message vocal, la photo ou le texte arrive par une messagerie instantanée grand public, déjà installée sur le téléphone du technicien.
- **Transcription** : un moteur de reconnaissance vocale adapté au darija et au code-switching arabe/français convertit l'audio en texte.
- **Extraction structurée** : un grand modèle de langage transforme ce texte libre en champs normalisés — chantier, travaux réalisés, quantités, incidents, besoins en matériel — sans jamais inventer une valeur absente.
- **Reformulation** : le même modèle produit un résumé en français professionnel, directement lisible par l'encadrement.
- **Diffusion** : le rapport est consigné dans le tableau de suivi, le chef de chantier et le responsable technique sont notifiés, et le technicien reçoit une confirmation.

Trois partis pris encadrent cette solution : un coût nul, obtenu en s'appuyant exclusivement sur les offres gratuites des fournisseurs ; une fiabilité stricte, où tout champ absent de l'audio reste vide plutôt que d'être deviné ; et une évolutivité assumée, les moteurs d'intelligence artificielle demeurant interchangeables.

Le périmètre fonctionnel qui traduit cette solution en exigences — besoins fonctionnels, besoins non fonctionnels et priorisation MoSCoW — est présenté au chapitre 3 (§3.2), au plus près de la conception qu'il gouverne ; l'architecture retenue y est détaillée à partir du §3.3.

1.6 Conduite du projet

1.6.1 Méthodologie : Scrum allégé

Le projet étant mené par un développeur unique avec un encadrant industriel jouant le rôle de *product owner*, une méthode Scrum complète serait artificielle. Une déclinaison allégée est adoptée : backlog priorisé, sprints de deux semaines, démonstration à l'encadrant en fin de sprint, rétrospective individuelle. Cette approche conserve les bénéfices essentiels — itérations courtes, feedback régulier, priorisation explicite — sans le cérémonial d'équipe.

Le travail s'étend sur **six mois, du 2 mars au 31 août 2026**, soit vingt-six semaines. Chaque phase se clôt par une démonstration à l'encadrant industriel, et le versionnage Git assure une trace continue de l'avancement.

1.6.2 Planification

Le **Tableau 1.1** présente le découpage en phases retenu au démarrage.

Tableau 1.1 : Planning prévisionnel du projet.

Phase	Période	Objectifs principaux
Phase 0 - Cadrage	2 – 27 mars	Immersion métier, étude de l'existant, entretiens avec le directeur et les chefs de chantier, cahier des charges, création des comptes et squelette du dépôt
Phase 1 - État de l'art et étude technique	30 mars – 30 avril	Revue de la transcription de l'arabe dialectal et de l'extraction structurée par LLM, étude comparative des canaux et des plateformes d'orchestration, constitution du jeu de test
Phase 2 - Conception	4 mai – 5 juin	Architecture hybride, modèle de données, contrat `ExtractedReport`, frontière

Phase	Période	Objectifs principaux
		orchestration/code, conception du tableau de bord
Phase 3 - Cœur IA	8 juin – 17 juillet	Service FastAPI, interface d'adaptateurs STT, chaînes de repli, extraction structurée, suite de tests hors ligne, premier bout-en-bout réel
Phase 4 - Chaîne complète et application Web	20 juillet – 1er août	Workflow n8n complet, réception bicanale, observabilité PostgreSQL, tableau de bord, gestion des cas d'échec
Phase 5 - Banc d'essai et durcissement	3 – 14 août	Banc d'essai comparatif des moteurs de transcription et des modèles d'extraction, journal par chantier, organigramme, sécurité et intégration continue
Phase 6 - Pilote, documentation, rapport	17 – 28 août	Déploiement pilote, livrables documentaires, rédaction finale, présentation
Tampon	28 – 31 août	Finitions, répétition de la soutenance

1.6.3 Gestion des risques

Le **Tableau 1.2** recense les risques identifiés au lancement, leur criticité et la parade retenue pour chacun.

Tableau 1.2 : Registre des risques principaux.

Risque	Impact	Mitigation
Taux d'erreur de transcription du darija trop élevé sur tous les moteurs	Fort	Banc d'essai sur 13 moteurs ; le modèle de langage corrige le bruit grâce au vocabulaire métier ; confirmation du résumé au technicien
Friction d'accès au canal de réception (politique de la plateforme)	Fort	Conception indépendante du canal ; Telegram d'abord,

Risque	Impact	Mitigation
		WhatsApp ensuite une fois les contraintes Meta levées
Quotas des offres gratuites	Moyen	Volumétrie très faible ; chaîne de repli alternant les fournisseurs, qui mesurent leurs quotas séparément ; moteur local sur GPU en dernier recours
Panne d'un service externe (Google Sheets, fournisseur LLM)	Moyen	File de reprise persistante en base ; journal d'événements en PostgreSQL, délibérément hors du Sheet
Glissement de planning (développeur seul)	Moyen	Ordre de coupe explicite ; le cœur du périmètre *Must* est protégé

1.7 Conclusion du chapitre

Ce chapitre a établi le cadre du projet : un besoin métier réel — fiabiliser la remontée d'information des chantiers — contraint par l'usage (l'oral en darija), par le budget (offres gratuites uniquement) et par le calendrier. Le chapitre suivant examine l'état de l'art des deux verrous scientifiques du projet, la transcription du darija et l'extraction structurée par grands modèles de langage, et en déduit les choix technologiques.

Chapitre 2 : État de l'art et étude technique

2.1 Introduction du chapitre

Ce chapitre passe en revue les fondements techniques du projet. Il examine d'abord la reconnaissance vocale de l'arabe dialectal marocain, puis l'extraction d'information structurée par grands modèles de langage. Il compare ensuite les plateformes d'orchestration et les canaux de messagerie envisageables, avant de synthétiser et de justifier les choix technologiques retenus.

2.2 Reconnaissance vocale de l'arabe dialectal

2.2.1 Le darija, une langue peu dotée

L'arabe marocain (darija) est la langue vernaculaire du Maroc. Il diffère sensiblement de l'arabe standard moderne (MSA) sur les plans phonologique, lexical et syntaxique, et se caractérise sur les chantiers par un code-switching permanent avec le français : les termes techniques — « goulotte », « câble », « panneau de brassage », « fibre », « vérifier » — sont empruntés tels quels. Or les corpus d'entraînement des systèmes de reconnaissance vocale arabes sont dominés par le MSA ; le darija appartient à la catégorie des langues dites *low-resource*, pour lesquelles les données annotées sont rares [1].

Une conséquence pratique, centrale pour ce projet, mérite d'être posée dès maintenant : l'écriture du darija n'est pas normalisée. Un même énoncé peut être transcrit en caractères arabes ou en caractères latins (*arabizi*), et les chiffres peuvent apparaître en chiffres arabes, en chiffres indo-arabes ou en toutes lettres (ربعين pour 40). Cette variabilité rend le WER, qui compare des chaînes de mots, particulièrement peu représentatif de la valeur métier réelle — point sur lequel le chapitre 4 revient longuement.

2.2.2 Les modèles de bout en bout : Whisper

L'approche dominante actuelle est le modèle de bout en bout entraîné par supervision massive et faiblement supervisée. Whisper, publié par OpenAI, est entraîné sur 680 000 heures d'audio multilingue et constitue la référence ouverte en robustesse multilingue [1]. Sa déclinaison `large-v3` prend en charge l'arabe ; ses performances sur les dialectes restent toutefois inférieures à celles obtenues sur le MSA, avec une tendance documentée à « normaliser » le dialecte vers l'arabe standard. Point favorable au cas d'usage : les emprunts français du vocabulaire du câblage sont souvent bien reconnus, précisément parce qu'ils appartiennent à une langue richement dotée.

Whisper est accessible gratuitement à travers plusieurs fournisseurs d'inférence ; Groq propose une API à très faible latence exposant `whisper-large-v3` avec une offre gratuite adaptée à une volumétrie de prototype [2].

2.2.3 L'architecture concurrente : wav2vec2 et la classification temporelle connexionniste

Une seconde famille d'architectures a été retenue pour le banc d'essai : wav2vec2, modèle auto-supervisé suivi d'une tête CTC (*Connectionist Temporal Classification*). Contrairement à Whisper, qui est un modèle génératif encodeur-décodeur, wav2vec2-CTC émet une distribution par trame acoustique sans modèle de langage interne. Deux conséquences pour ce projet :

- **Il ne peut pas boucler.** Les répétitions dégénérées observées sur les modèles génératifs (§4) sont structurellement impossibles en CTC.
- **Sa mesure de confiance n'est pas comparable à celle de Whisper.** La confiance CTC est une moyenne de *softmax* sur les trames émises ; celle de Whisper est `exp(mean token logprob)`. Les deux vivent dans l'intervalle [0, 1] et mesurent des quantités différentes. Ce piège s'est matérialisé lors du benchmark et est analysé au chapitre 4.

2.2.4 Modèles affinés sur le darija

La communauté Hugging Face publie des affinages (*fine-tunes*) de Whisper et de wav2vec2 sur des corpus marocains. Leur qualité est variable et leur documentation souvent minimale. Le projet en a recensé et téléchargé onze, auxquels s'ajoutent deux contrôles non affinés — les modèles OpenAI d'origine —, sans lesquels aucune conclusion sur l'apport de l'affinage ne serait possible. Ce point de méthode s'est révélé décisif : sans les contrôles, un affinage de qualité moyenne aurait été couronné vainqueur.

2.2.5 Métriques d'évaluation

La qualité de transcription se mesure classiquement par le **WER** (*Word Error Rate*), proportion de mots insérés, supprimés ou substitués par rapport à une transcription de référence, et par le **CER** (*Character Error Rate*), plus tolérant aux variations orthographiques.

Ce projet a délibérément retenu une **métrique applicative** à la place : la préservation des champs métier — quantités numériques et emprunts techniques français. La justification est développée en §4.3.2, mais son principe tient en une phrase : le tableau de suivi ne consomme ni des mots ni des caractères, il consomme des nombres et des noms de lieux ; un moteur qui écrit ربعين là où la référence dit « 40 » est pénalisé par le WER alors qu'il fournit exactement l'information dont le système a besoin, et un moteur qui écrit une phrase fluide en perdant la quantité est récompensé par le WER alors qu'il produit une ligne vide dans le journal.

2.3 Extraction d'information structurée par grands modèles de langage

2.3.1 De l'extraction classique aux LLM

L'extraction d'information — entités, quantités, relations — reposait classiquement sur des règles ou des modèles supervisés spécifiques. Les grands modèles de langage ont déplacé ce paradigme : un modèle génératif correctement instruit extrait des champs arbitraires d'un texte libre sans entraînement dédié, y compris depuis un texte bruité — capacité déterminante ici, puisque la transcription du darija est imparfaite par nature. Le LLM joue alors un double rôle : extracteur de données et correcteur implicite du bruit de transcription, guidé par le vocabulaire métier fourni dans l'instruction.

2.3.2 Sorties structurées et fiabilité

Le risque principal des LLM est l'hallucination : produire une donnée plausible mais absente de la source. Deux mécanismes le contiennent :

- **La sortie structurée contrainte** : les API récentes permettent d'imposer un schéma JSON au modèle (*structured output* de Gemini [3], mode JSON de Groq [2]), garantissant la forme de la réponse.
- **La validation applicative** : côté service, chaque réponse est validée par un schéma Pydantic strict (types, bornes, énumérations) ; l'instruction impose la valeur nulle pour tout champ non mentionné, et la politique du projet interdit toute valeur inventée par défaut.

Ce projet ajoute un troisième mécanisme, propre à son domaine et décrit au chapitre 3 : une valeur de localisation non reconnue **lève une erreur** au lieu d'être ignorée, afin que la chaîne de repli réessaie plutôt que d'écrire un étage inventé.

2.3.3 Modèles candidats

Sous contrainte d'offre gratuite, deux fournisseurs se distinguent : **Gemini** (Google AI Studio), doté d'une sortie structurée native et d'une forte qualité rédactionnelle en français pour la reformulation, et **Groq**, qui sert des modèles ouverts en mode JSON avec une latence très faible. Plutôt que d'en choisir un seul, le projet les organise en **chaîne de repli** alternant délibérément les fournisseurs — Gemini et Groq mesurant leurs quotas gratuits séparément, l'alternance double la capacité effective — la défaillance de tous étant remontée comme une erreur explicite plutôt que masquée.

2.4 Plateformes d'orchestration

Le cahier des charges recommande n8n comme outil d'automatisation. L'étude comparative, résumée dans le **Tableau 2.1**, confirme ce choix :

Tableau 2.1 : Comparaison des solutions d'orchestration.

Critère	n8n	Zapier / Make	Code sur mesure
Coût	Gratuit (auto-hébergé, licence fair-code)	Quotas gratuits restrictifs	Gratuit
Hébergement	Docker local ou serveur	SaaS uniquement	Au choix
Intégrations (messageries, Sheets, e-mail)	Natives	Natives	À développer
Logique personnalisée	Nœuds de code JavaScript	Limitée	Totale
Valeur pédagogique / maintenance	Workflows visuels documentables	Boîte noire	Coût de maintenance élevé

n8n auto-hébergé en Docker combine gratuité, intégrations prêtes à l'emploi et possibilité d'insérer du code JavaScript. Le choix architectural du projet — détaillé au chapitre 3 — est toutefois **hybride** : n8n orchestre les intégrations, mais le cœur métier est un microservice codé sur mesure, testable et versionné.

Le chapitre 4 documente la limite pratique qui a fixé la frontière entre les deux : une logique qui exige un *read-modify-write* — lire l'onglet pour reconstituer le stock et reconnaître une journée déjà ouverte — n'est pas exprimable dans un nœud conditionnel. C'est cette contrainte, et non une préférence esthétique, qui a fait migrer l'écriture du journal de n8n vers le service Python.

2.5 Canal de réception : étude comparative

Le cahier des charges cite WhatsApp Business API « ou passerelle équivalente ». L'étude pratique des deux principaux candidats a structuré le calendrier du projet, comme le récapitule le **Tableau 2.2** :

Tableau 2.2 : Comparaison des canaux de réception.

Critère	WhatsApp Cloud API (Meta)	Telegram Bot API
Processus d'accès	Compte développeur Meta, application publiée, validation d'entreprise	Création d'un bot en une minute (BotFather)
Jeton d'accès	Expire toutes les 24 h en mode test ; permanent via un *System User*	Permanent
Destinataires (mode test)	Liste blanche limitée à 5 numéros	Illimité
Réception des messages	Webhook obligatoire (URL publique HTTPS + poignée de main de vérification)	Webhook **ou** interrogation périodique (long polling)
Coût	Gratuit en test, conversations facturées en production	Gratuit
Adoption par le public cible	Très forte au Maroc	Moyenne

Décision et évolution. Telegram a été retenu en premier pour le développement, précisément parce qu'il n'impose aucune approbation et autorise le *polling* — donc aucune exposition publique du poste de développement. WhatsApp, indispensable à terme du fait de son adoption au Maroc, a été mis en service dans un second temps une fois les obstacles Meta levés (jeton *System User* sans expiration, application publiée, souscription `subscribed_apps` du WABA). **Les deux canaux sont aujourd'hui en production simultanément**, servis par un seul workflow et un seul service.

Cette asymétrie a une conséquence de diagnostic qu'il faut garder à l'esprit : Telegram *interroge* (connexions sortantes, aucun tunnel nécessaire) tandis que WhatsApp est *poussé* par webhook et exige une URL publique HTTPS. Par conséquent, « Telegram répond une erreur mais WhatsApp reste muet » ne décrit pas une panne mais deux : une réponse d'erreur prouve que le message a atteint la chaîne ; le silence prouve qu'il n'y est jamais arrivé.

2.6 Étude comparative des solutions existantes

Avant de concevoir un système, il faut justifier de ne pas en acheter un. Trois familles de produits couvrent partiellement ce besoin.

Les applications de suivi de chantier — Procore, Fieldwire, Kizeo Forms — proposent des formulaires mobiles, des photos géolocalisées et des tableaux de bord. Elles échouent sur deux points décisifs ici. D'une part, le prix : toutes sont facturées par utilisateur et par mois, ce qui est incompatible avec la contrainte de budget nul. D'autre part, et c'est plus fondamental, **elles imposent une saisie structurée à la source** : le technicien doit ouvrir une application, choisir un chantier dans une liste et remplir des champs. L'étude de l'existant (§1.3.3) a montré que c'est précisément ce qui serait rejeté par l'usage. Aucune n'accepte une note vocale en darija comme entrée principale.

Les plateformes d'automatisation avec IA managée — Microsoft Power Automate couplé à Azure Speech, ou Zapier couplé à OpenAI. Techniquement viables et rapides à assembler, elles sont écartées pour trois raisons : le coût à l'usage dès la sortie des quotas d'essai ; l'absence de prise en charge sérieuse du darija dans les services de transcription généralistes, qui est le cœur du problème ; et une logique métier enfermée dans une interface propriétaire, non versionnée et non testable — inacceptable pour un travail d'ingénierie qui doit être défendu et repris.

Les assistants de transcription généralistes — Otter, Notta, ou l'envoi manuel à un modèle de langage. Ils transcrivent, mais ne structurent pas, n'écrivent pas dans le tableau de suivi, ne notifient personne et ne connaissent pas le vocabulaire du câblage. Ils résolvent un cinquième du problème.

Le **Tableau 2.3** situe le besoin d'AZNET face aux catégories d'outils existantes.

Tableau 2.3 : Positionnement face aux solutions existantes.

Critère	Applications de chantier	Automatisation + IA managée	Assistants de transcription	**Ce projet**
Coût	Par utilisateur / mois	À l'usage	Abonnement	**Nul**
Entrée en darija	Non	Faible	Partielle	**Oui, moteur dédié**
Effort du technicien	Formulaire à remplir	Formulaire à remplir	Envoi manuel	**Une note vocale**
Extraction structurée métier	Oui, par formulaire	À construire	Non	**Oui, par LLM**
Écriture dans le fichier du client	Export	Oui	Non	**Oui, fusion par chantier**
Logique versionnée et testée	Non	Non	—	**Oui, 860 tests**

La conclusion n'est pas que ces produits sont mauvais, mais que le besoin d'AZNET a une contrainte que le marché ne traite pas : **l'entrée doit être de la parole spontanée en langue peu dotée**, et tout le reste en découle.

2.7 Technologies et langages retenus

2.7.1 Vue d'ensemble

Le **Tableau 2.4** reprend les technologies retenues et, pour chacune, l'alternative écartée.

Tableau 2.4 : Technologies retenues et alternatives écartées.

Technologie	Rôle dans le projet	Principales alternatives	Raison du choix
Python 3.11	Service applicatif, banc d'essai	Node.js, Go, Java	Écosystème IA/ML sans équivalent ; toutes les bibliothèques de modèles sont d'abord publiées en Python
FastAPI	Framework HTTP	Django, Flask	Validation et documentation dérivées des types ; asynchrone natif
Pydantic	Contrat de données	dataclasses, marshmallow	La validation *est* le garde-fou anti-hallucination du projet
TypeScript	Tableau de bord	JavaScript	Le contrat d'API change souvent ; le typage transforme une rupture en erreur de compilation
React 19	Interface	Vue, Svelte, Angular	Standard industriel, écosystème et employabilité
Vite	Outil de construction	webpack, Next.js	Démarrage quasi instantané ; aucun besoin de rendu serveur

Technologie	Rôle dans le projet	Principales alternatives	Raison du choix
Tailwind CSS v4	Style	CSS modules, styled-components, shadcn/ui	Couche de jetons entièrement en CSS, sans fichier de configuration JavaScript
PostgreSQL 16	Stockage technique	SQLite, MongoDB	Accès concurrents réels (API, ordonnanceur, n8n) ; JSON natif
Google Sheets	Registre métier	Base de données seule	**Imposé par l'usage** : c'est le fichier que le directeur ouvre déjà
n8n	Orchestration	Zapier, Make, code	Auto-hébergeable et gratuit ; intégrations natives (voir §2.4)
Docker Compose	Déploiement	Installation manuelle, Kubernetes	Cinq services reproductibles ; Kubernetes serait disproportionné pour une machine
PyTorch + Transformers	Banc d'essai local	ONNX Runtime, llama.cpp	Seul écosystème où les treize points de contrôle évalués sont publiés
Playwright	Contrôle visuel	Selenium, Cypress	Capture multi-navigateur scriptable ; sert aussi à produire les captures de ce rapport
pytest	Tests	unittest	Fixtures et paramétrage ; 860 tests, dont 810 sans aucune clé d'API ni base

Technologie	Rôle dans le projet	Principales alternatives	Raison du choix
GitHub Actions	Intégration continue	GitLab CI, Jenkins	Gratuit sur dépôt privé ; au plus près du dépôt
Mermaid	Diagrammes	draw.io, Visio	Diagrammes en texte, donc versionnés et révisables comme du code

2.7.2 Les choix qui méritaient un débat

Quatre décisions n'allaient pas de soi et ont été tranchées sur des critères vérifiables.

FastAPI plutôt que Django ou Flask. Django apporte un ORM, une administration et une authentification — l'essentiel de ce qu'il offre est inutile ici, puisque le registre métier est un classeur Google et non une base relationnelle. Flask est minimal mais n'a ni validation ni documentation automatique. FastAPI dérive la validation, la documentation OpenAPI et le contrat d'erreurs **des annotations de type déjà écrites**, ce qui compte doublement dans un système dont la règle principale est « aucune donnée inventée » : le schéma qui documente est le schéma qui valide.

PostgreSQL plutôt que SQLite. SQLite aurait suffi au volume — quelques dizaines de lignes par jour. Il a été écarté sur la concurrence, non sur la taille : trois processus écrivent simultanément — l'API sur le chemin de réponse, l'ordonnanceur en arrière-plan, n8n à chaque étape — et le verrouillage de fichier de SQLite transforme cela en attentes sur le chemin où un technicien attend une réponse.

Tailwind v4 sans bibliothèque de composants. shadcn/ui aurait fait gagner des semaines. Il a été écarté parce que ce tableau de bord a une identité visuelle propre, décrite dans un document de conception, et qu'adopter une bibliothèque revient à hériter de ses décisions esthétiques puis à lutter contre elles. Les graphiques sont en SVG et CSS écrits à la main, pour la même raison — et cela évite une dépendance de plus de 400 Ko pour quatre visualisations.

Le darija en local plutôt qu'une API. C'est le choix le plus coûteux du projet : 7,7 s de latence contre 1,53 s, environ 4 Go de mémoire graphique, et une machine à maintenir. Il est justifié au chapitre 4 par la seule chose qui compte ici — la préservation des quantités et des emprunts techniques français — et il a été confirmé à l'oreille par un locuteur natif avant d'être adopté.

2.8 Synthèse des choix technologiques

Le **Tableau 2.5** rassemble ces décisions en une vue unique.

Tableau 2.5 : Récapitulatif des choix technologiques.

Besoin	Choix	Justification résumée
Canal de réception	Telegram Bot API **et** WhatsApp Cloud API	Gratuit et sans approbation pour le premier ; adoption réelle du public cible pour le second
Orchestration	n8n (Docker, auto-hébergé)	Recommandé par le cahier des charges, gratuit, intégrations natives
Transcription	Chaîne `gemini → local → groq`	Ordre fixé par accuracy mesurée (chapitre 4), non par latence
Extraction / reformulation	Chaîne de repli alternant Gemini et Groq	Sortie structurée native ; quotas gratuits mesurés séparément par fournisseur
Service métier	Python / FastAPI	Écosystème IA, validation Pydantic, tests, documentation OpenAPI automatique
Interface Web	React 19 + TypeScript + Vite + Tailwind v4	Standard industriel ; rendu statique servi par la même origine que l'API
Stockage métier	Google Sheets	Imposé par le cahier des charges ; c'est le fichier que le directeur lit déjà
Stockage technique	PostgreSQL	Journal d'événements, idempotence, sessions, file de reprise — délibérément hors du Sheet

2.9 Conclusion du chapitre

L'état de l'art confirme la faisabilité du projet sous ses contraintes : Whisper fournit une base de transcription multilingue robuste dont les limites sur le darija sont compensées en aval par l'extraction LLM ; les sorties structurées contraintes et la validation stricte contiennent le risque d'hallucination ; et l'écosystème gratuit couvre l'intégralité de la chaîne. Il a également posé deux points de méthode que le chapitre 4 exploitera : la nécessité de contrôles non affinés dans tout banc d'essai de *fine-tunes*, et l'inadéquation du WER à un système dont la valeur tient aux nombres. Le chapitre suivant traduit ces choix en une conception détaillée.

Chapitre 3 : Conception de la solution

3.1 Introduction du chapitre

Ce chapitre présente la conception de la solution : l'analyse des besoins issue du cahier des charges, l'architecture générale hybride, le modèle de données qui constitue le contrat entre les composants, la conception détaillée du service applicatif (patrons de conception, chaînes de repli, contrat d'erreurs, tenue du journal), celle du workflow d'orchestration, du tableau de bord, et les dispositions de sécurité.

3.2 Analyse des besoins

3.2.1 Besoins fonctionnels

Le cahier des charges fourni par l'entreprise définit les exigences suivantes :

- **BF1 — Saisie vocale (prioritaire)** : le technicien envoie une note vocale en darija via une messagerie instantanée ; aucune autre action ne lui est demandée.
- **BF2 — Saisie alternative** : un second canal alimente le même traitement.
- **BF3 — Transcription automatique** de l'audio en texte, avec prise en charge du darija et du mélange arabe/français.
- **BF4 — Extraction automatique** des travaux réalisés et des quantités.
- **BF5 — Reformulation** du rapport en français professionnel.
- **BF6 — Enregistrement** de chaque rapport dans un tableau de suivi Google Sheets structuré.
- **BF7 — Notification automatique** du chef de chantier et du responsable technique.
- **BF8 — Confirmation** au technicien que son rapport a bien été traité.

3.2.2 Besoins non fonctionnels

- **BNF1 — Simplicité d'usage** : temps de soumission inférieur à deux minutes.
- **BNF2 — Coût nul** : uniquement des offres gratuites ; volumétrie faible (5 à 10 rapports/jour) compatible avec les quotas gratuits.
- **BNF3 — Fiabilité** : aucune donnée inventée — tout champ absent de l'audio doit rester vide ; les transcriptions douteuses doivent être signalées plutôt que silencieusement acceptées.
- **BNF4 — Latence** : traitement de bout en bout visé sous 60 secondes.
- **BNF5 — Évolutivité** : moteurs d'IA interchangeables ; migration possible vers une base de données.
- **BNF6 — Sécurité** : secrets hors du code source ; validation stricte des entrées.

3.2.3 Priorisation MoSCoW

Le **Tableau 3.1** répartit ces exigences selon la méthode MoSCoW.

Tableau 3.1 : Priorisation MoSCoW du périmètre fonctionnel.

Priorité	Périmètre
Must	Chaîne vocale complète (réception → transcription → extraction → résumé français → Sheets → notification) ; documentation d'installation ; guide utilisateur
Should	Tableau de bord de consultation ; chapitre de benchmark STT ; réponse de confirmation au technicien ; signalement des transcriptions douteuses
Could	Second canal de réception ; deuxième vague de benchmark sur enregistrements réels ; document de migration base de données
Won't (ce PFE)	Application de bureau ; multi-entreprises

Le périmètre effectivement livré dépasse cette priorisation sur trois points, documentés au chapitre 4 : le second canal de réception (WhatsApp Cloud API) est en service et non resté en « Could » ; la base de données PostgreSQL a été introduite en complément de Google Sheets et non reportée ; et le tableau de bord a dépassé la consultation pour devenir l'outil d'administration du système (organigramme, comptes, ordonnanceur, journaux).

3.3 Architecture générale

3.3.1 Le choix d'une architecture hybride

Deux approches extrêmes étaient possibles : tout construire dans n8n — fidèle à la lettre du cahier des charges, mais logique métier enfouie dans des nœuds difficiles à tester — ou tout coder sur mesure, ce qui imposait de réécrire des intégrations que n8n fournit nativement. L'architecture retenue est **hybride** et répartit les responsabilités selon la valeur ajoutée de chaque outil :

- **n8n (orchestration)** : réception des messages Telegram et WhatsApp, vérification du webhook Meta, déduplication, téléchargement des médias, appel du service, réponses au technicien, journalisation d'étape.
- **Service applicatif (FastAPI, sur mesure)** : transcription, extraction structurée, reformulation, tenue du journal de chantier, ordonnancement des tâches périodiques, authentification, service du tableau de bord — le cœur, versionné, testé et documenté.
- **Tableau de bord (React)** : consultation, correction, administration de l'organigramme et des comptes, supervision de l'ordonnanceur et des journaux.
- **PostgreSQL** : journal d'événements, idempotence, sessions, file de reprise, organigramme, chantiers.
- **Google Sheets** : le registre métier permanent — le fichier que le directeur lit déjà.

La **Figure 3.1** met ces cinq responsabilités en relation et fait apparaître les échanges qui les lient.

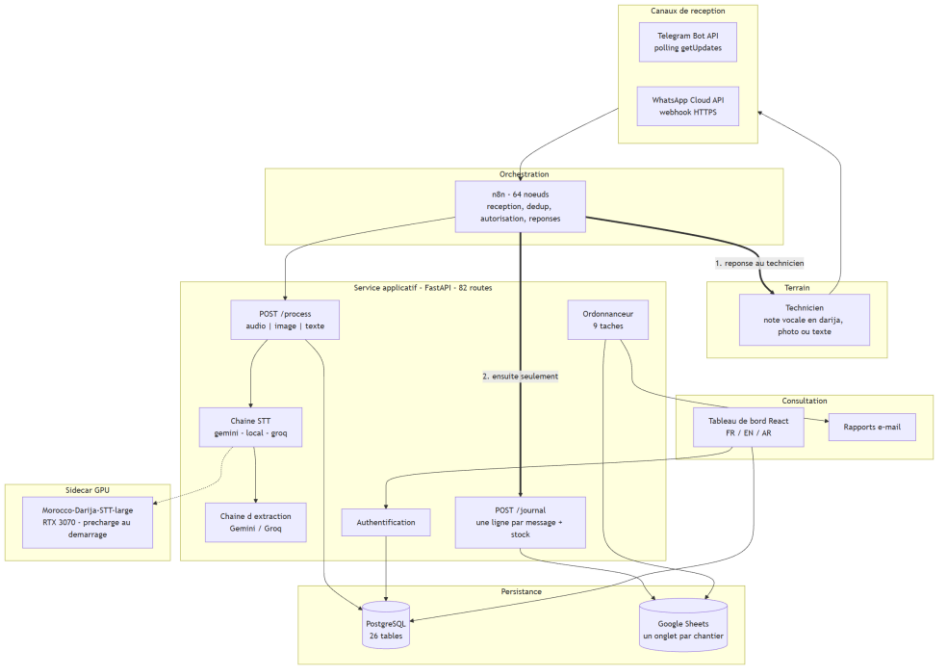


Figure 3.1 : *Architecture générale : les cinq responsabilités et leurs échanges.*

3.3.2 Frontière entre orchestration et code

La frontière n'est pas esthétique, elle est fonctionnelle, et l'expérience du projet l'a déplacée une fois. À l'origine, l'écriture dans Google Sheets était un nœud n8n. Elle a migré dans le service Python le 5 août 2026 pour une raison précise : tenir le journal exige un *read-modify-write* — lire l'onglet pour réconcilier le stock et reconnaître une journée déjà ouverte — qu'un nœud conditionnel ne sait pas exprimer. La correspondance entre les champs extraits et les colonnes du Sheet, jusque-là écrite en expressions n8n non testables, est devenue du code Python couvert par des tests.

Règle retenue : ce qui est *intégration* reste dans n8n ; ce qui est *décision* passe dans le service.

3.3.3 Flux nominal

1. Le technicien envoie une note vocale, une photo ou un texte au bot, sur Telegram ou WhatsApp.
2. Le workflow n8n détecte le message, **réclame** son identifiant auprès du service (déduplication : WhatsApp re-livre un webhook non acquitté), identifie l'expéditeur dans l'organigramme et vérifie son autorisation.
3. Le média est téléchargé et transmis au service (`POST /process`, multipart).
4. Le service transcrit, extrait les champs, produit le résumé français et renvoie un JSON structuré assorti d'avertissements de qualité.
5. **n8n répond au technicien** avec son résumé.

6. **Ensuite seulement**, n8n appelle `POST /journal`, qui ajoute la ligne dans l'onglet du chantier et met à jour le miroir plat.
7. Les responsables sont notifiés.

L'ordre des étapes 5 et 6 est un invariant de conception, développé en §3.6.2.

3.3.4 Diagrammes UML

La **Figure 3.2** recense les cas d'utilisation du système et leurs acteurs.

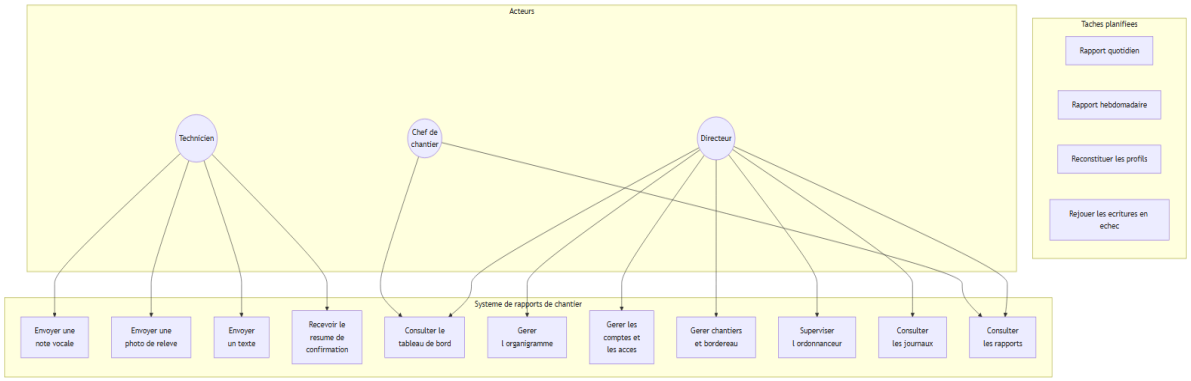


Figure 3.2 : *Diagramme de cas d'utilisation.*

La **Figure 3.3** suit une note vocale sur l'ensemble de son parcours, de l'émission à la ligne du journal.

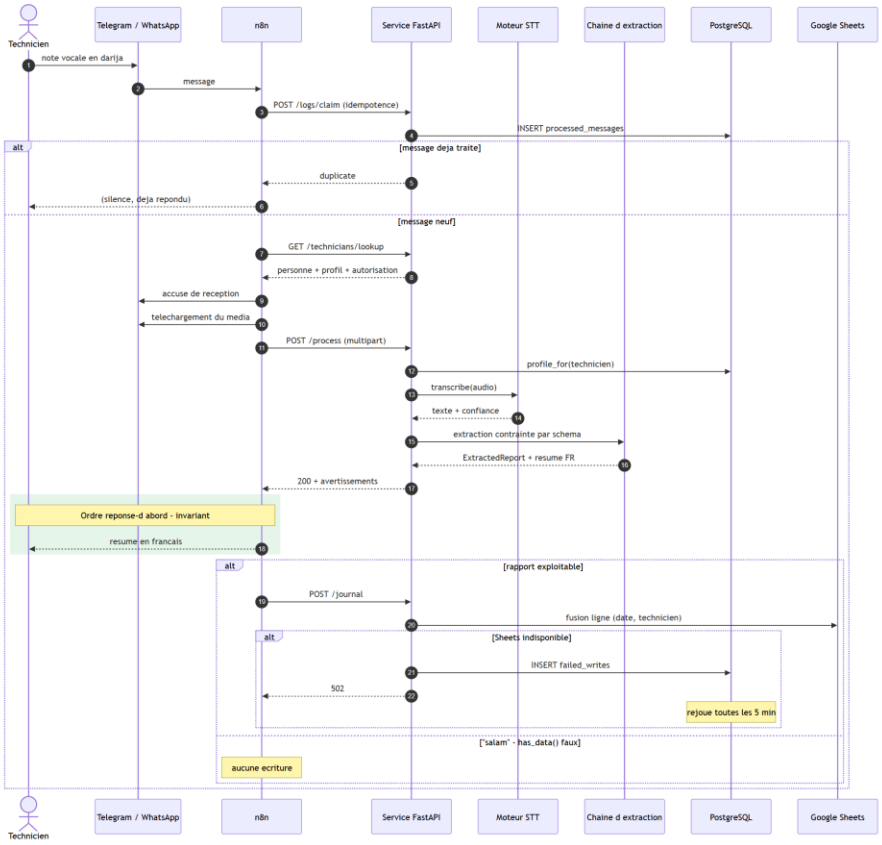


Figure 3.3 : *Cycle de vie complet d'une note vocale, de l'émission à la ligne du journal, y compris la déduplication en amont et la mise en file de reprise en aval.*

3.4 Modèle de données

3.4.1 Le rapport structuré, contrat central

Le schéma `ExtractedReport` est le contrat entre le service, n8n et le journal. Il a été **reconstruit le 5 août 2026 à partir du fichier de suivi réel du directeur**, en remplacement d'un schéma générique initial. Le **Tableau 3.2** en donne les champs :

Tableau 3.2 : Schéma du rapport extrait (`ExtractedReport`).

Champ	Type	Description
`cables_prepares_m`	décimal ≥ 0, optionnel	Mètres de câble cuivre préparés (tirés et mesurés)
`cables_installes_m`	décimal ≥ 0, optionnel	Mètres de câble cuivre installés et raccordés
`goulotte_m`	décimal ≥ 0, optionnel	Mètres de goulotte posés
`prises_connectees`	entier ≥ 0, optionnel	Prises RJ45 raccordées côté bureau

Champ	Type	Description
`panneaux_prises_brassees`	entier ≥ 0, optionnel	Prises raccordées côté panneau de brassage
`prises_testees`	entier ≥ 0, optionnel	Prises testées et étiquetées
`fibre_m`	décimal ≥ 0, optionnel	Mètres de câble fibre optique posés
`interconnexions`	entier ≥ 0, optionnel	Nombre d'interconnexions installées entre locaux
`connexions_optiques_brins`	entier ≥ 0, optionnel	Brins de fibre optique fusionnés/connectés

Vingt-cinq champs d'article supplémentaires (`cordons_brassage_1m_u`, `tiroir_optique_24_u`, `switch_acces_u`, `module_sfp_u`, `cable_elec_3x2_5_m`, `disjoncteur_2x16a_u`…) ont été ajoutés le 15 août 2026 : ils sont l'**union des trois bordereaux réels** — DGH, Extralait, ACAPS — repliée de sorte qu'un même objet physique soit un seul champ. La liste complète figure dans le dépôt du projet.

Champ	Type	Description
`etages`	liste, optionnel	Étages travaillés (RDC, E1..E4)
`ailes`	liste, optionnel	Ailes (Droite, Gauche)
`locaux_techniques`	liste, optionnel	Locaux techniques (RG, SR1..SR10)
`stock_cable_m`	décimal ≥ 0, optionnel	Mètres de câble restants en stock sur site
`stock_goulotte_m`	décimal ≥ 0, optionnel	Mètres de goulotte restants en stock
`stock_fibre_m`	décimal ≥ 0, optionnel	Mètres de fibre optique restants en stock
`remarques`	texte, optionnel	Problèmes, blocages, matériel manquant, remarques
`heure_entree`	texte, optionnel	Heure d'arrivée sur le chantier, format HH:MM
`heure_sortie`	texte, optionnel	Heure de départ du chantier, format HH:MM
`equipe`	texte, optionnel	Les personnes présentes, telles que nommées

Champ	Type	Description
`temps_arret_min`	décimal ≥ 0, optionnel	Minutes perdues à l'arrêt (attente, bureaux fermés, instruction)
`motif_arret`	texte, optionnel	Raison de l'arrêt, une ligne
`besoins`	texte, optionnel	Matériel manquant ou à commander
`prochaines_etapes`	texte, optionnel	Ce que le technicien prévoit de faire ensuite
`chantier_mentionne`	texte, optionnel	Nom du chantier tel qu'énoncé par le technicien, sinon null
`date_declaree`	texte, optionnel	Date énoncée dans le message (« Salam 17-08-2026 »), telle quelle, sinon null
`nature`	énuméré (pose / livraison), optionnel	'livraison' uniquement pour un bon de livraison / une réception de matériel ; sinon null
`livraisons`	objet {texte: décimal}, optionnel	Quantités reçues par champ (cables_installes_m, pigtail_sc_u, …), pour un bon de livraison
`articles_libres`	objet {texte: décimal}, optionnel	Articles posés qui ne correspondent à AUCUN champ existant, {désignation courte au singulier: quantité}
`resume`	texte, **obligatoire**	Résumé professionnel en français (2-4 phrases)

Trois décisions de modélisation méritent d'être justifiées.

Trois compteurs de prises, pas un. La même prise physique est comptée trois fois dans le fichier du directeur, à trois stades différents ; une seule journée peut légitimement rapporter « 14 côté bureaux, 17 côté panneau, 15 testées ». Fusionner ces compteurs perdrait l'information d'avancement qui les rend utiles.

Le chantier vient de la voix, et de nulle part ailleurs. `chantier_mentionne` est un champ du modèle : le nom du site tel que le technicien le prononce, extrait mot pour mot et jamais deviné. Le routage n'a que **deux sources**, dans cet ordre : le site que le technicien nomme, puis, à défaut, celui de sa propre dernière ligne — un rattachement « collant », sans expiration, et le résumé

précise « (repris du dernier rapport) » pour qu'un site hérité soit visible dès le premier message. Rien d'autre. Sans correspondance et sans historique, la ligne va dans l'onglet `Non affecté`, qui n'est pas un échec de routage mais l'énoncé visible que personne n'a dit où le travail avait eu lieu — et qu'un seul message nommant le site suffit à corriger. Le chantier est estampillé sur la ligne, puis figé, de sorte qu'une réaffectation ultérieure ne réécrit jamais l'histoire.

Une troisième source serait possible et est **délibérément absente** : le chantier porté par le chef et hérité par ses techniciens. Elle échoue exactement là où on l'attend — elle répond avec assurance sur le seul cas où l'on ne sait rien, le premier message d'un technicien, et cette supposition devient aussitôt son site collant, qui se réapplique ensuite indéfiniment. Sur données réelles, cela suffit à rattacher un technicien plusieurs jours durant à un chantier que personne n'a jamais nommé. Une valeur par défaut plausible coûte plus cher qu'une absence visible, et c'est la raison pour laquelle l'organigramme ne route rien.

Deux colonnes seulement échappent à la parole. L'hypothèse de départ était plus large : équipe, heures d'entrée et de sortie, jours-homme, ordre du jour et taux de réalisation étaient réputés « saisis par le chef ». Elle était fausse pour la moitié d'entre elles. Un technicien dit « entré à 7 h, sorti à 8 h, on était trois » dans sa note ; le schéma n'avait pas de champ pour l'accueillir, la colonne restait vide et l'information dormait dans la transcription, hors d'atteinte. `heure_entree`, `heure_sortie` et `equipe` sont donc des champs du modèle, et **JH** est calculé de ceux-ci plutôt que saisi.

Restent l'**ordre du jour** et le **taux de réalisation**, qui sont d'une autre nature : le premier est planifié *avant* la journée, le second est un jugement porté *après* elle contre ce plan. Aucun rapport ne peut les énoncer, et le tableau de bord est le seul endroit où ils s'écrivent.

La **Figure 3.4** représente ce même parcours comme une machine à états, ce qui fait ressortir la variété des issues possibles.

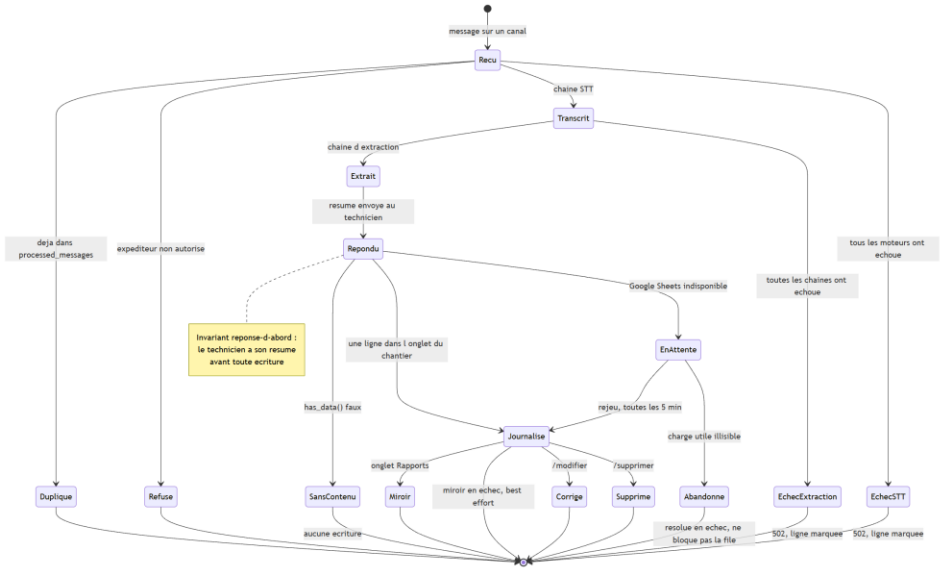


Figure 3.4 : *Cycle de vie d'un rapport, vu comme une machine à états.*

3.4.2 Politique « aucune donnée inventée »

Tous les champs métier sont optionnels et valent nul par défaut. L'instruction impose explicitement la valeur nulle pour toute information absente ; la validation Pydantic rejette les valeurs hors bornes. Trois mécanismes complètent cette politique :

- **Normalisation avant typage** : les modèles en mode JSON émettent parfois la *chaîne* `"null"` ; un validateur la ramène à la valeur nulle avant le typage strict.
- **Normalisation des formes, pas des faits** : `rdc`, `1er étage`, `SR 4` sont des erreurs de forme, pas de contenu ; une table d'alias les normalise. Mais une localisation **non reconnue lève une erreur** au lieu d'être ignorée : la chaîne de repli réessaie, plutôt que d'écrire un étage inventé.
- **`has_data()`** : « salam » s'extrait parfaitement — tous les champs nuls et un `resume` poli — et un code 200 devient alors indiscernable d'un vrai rapport. `has_data()` ignore délibérément `resume`, le seul champ que le modèle remplit toujours et qui ne peut donc pas distinguer les deux cas. Il compte en revanche les remarques, la localisation et le stock : une journée sans mètres mais avec un blocage (« arrêtés 1h30 sur instruction ») **est** le rapport dont le directeur a le plus besoin.

3.4.3 Le modèle relationnel : une personne, plusieurs canaux

Le stockage technique repose sur PostgreSQL. Sa décision structurante date du 1er août 2026 : la table `users` a été **fusionnée dans `persons` puis supprimée**.

L'argument initial du découpage était le cycle de vie (« un technicien ne se connecte jamais »), qui est un argument sur les *identifiants de connexion*, non sur l'*identité*. Deux tables pour un même humain faisaient d'un chef qui envoie aussi des notes vocales deux lignes sans relation — donc deux cases sur l'organigramme. Le modèle retenu est :

- **`persons`** : l'humain (nom, rôle, `manager_id`, profil **et identifiants de connexion**).
- **`technicians`** : ses identités de canal (`channel`, `external_id`, `active`), plusieurs par personne.

Conséquence directe : **`password_hash IS NOT NULL` constitue à lui seul la définition d'« un compte »**. La suppression d'un compte efface les identifiants sans supprimer la ligne — un `DELETE` emporterait en cascade la personne, ses canaux et son profil.

Les vingt-quatre autres tables portent le pipeline, l'exploitation et la mesure : `pipeline_logs` (journal d'étapes), `processed_messages` (idempotence), `failed_writes` et `message_queue` (les deux files de reprise), `media_files` et `chat_messages` (l'archive des médias et des messages reçus), `exchanges` (les échanges qui ne sont jamais devenus une ligne), `sessions`, `user_actions`, `page_visits`, `person_settings`, `notifications`, `tasks`, `manager_reports`, `report_versions`, `service_samples`, `app_settings`, `chantiers`, `articles`, `article_registry`, `report_rows`, `receptions`, `alert_mutes` et `api_usage`. Elles sont décrites au §3.4.4.

La **Figure 3.5** donne le modèle relationnel de la partie identité.

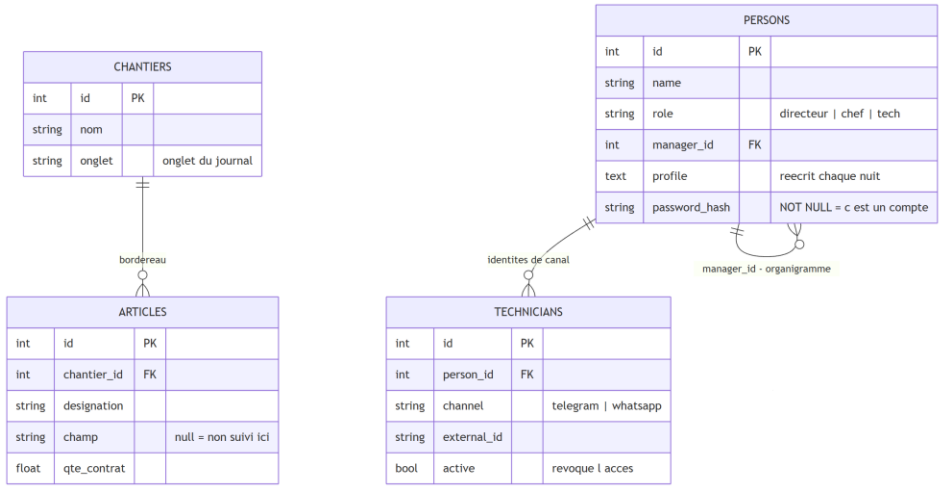


Figure 3.5 : *Modèle relationnel. persons est au centre et se référence elle-même par manager_id : c'est l'organigramme.*

La **Figure 3.6** complète le modèle avec les tables d'exploitation et d'audit.

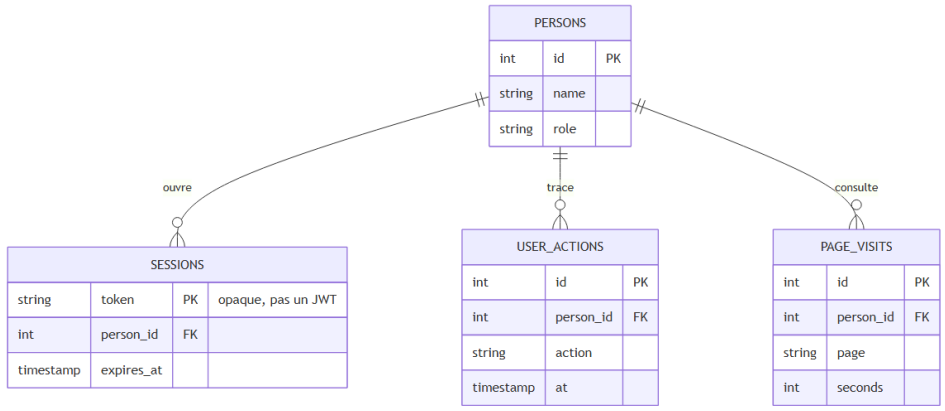


Figure 3.6 : *Les tables d'audit, seules tables d'exploitation à porter une clé étrangère : une session, une action et une visite appartiennent toutes à une personne.*

3.4.4 Les tables en détail

Le schéma compte **vingt-six tables**. Les seize décrites en premier portent l'essentiel : quatre le métier et l'organigramme ; trois l'usage humain ; neuf la mécanique du pipeline et la mesure, et ces dernières n'ont **délibérément aucune clé étrangère** — ce sont des journaux, des files et des registres, pas des entités liées. Les dix restantes, regroupées en fin de section, sont les tables de service apparues au fil des écrans du tableau de bord.

Table	**Rôle**
persons	l'humain. Une ligne par personne, quel que soit son rôle. role vaut directeur, chef ou tech

Table	**Rôle**
	et pilote l'affichage de l'organigramme ; manager_id pointe un technicien vers son chef.
technicians	les identités de canal. Une ligne par couple (personne, canal).
chantiers	les sites. nom et onglet, ce dernier désignant l'onglet du classeur où le journal de ce site est tenu.
articles	le bordereau. Les quantités contractuelles d'un chantier.
sessions	les connexions.
user_actions et page_visits	la piste d'audit. Qui a fait quoi, et quel temps a été passé sur quelle page.
pipeline_logs	le journal d'étapes.
processed_messages	l'idempotence. WhatsApp relivre un webhook non acquitté ; sans cette table, un message relivré produirait une seconde réponse et une seconde ligne.
failed_writes	la file de reprise. Charge utile complète et statut.
article_registry	les articles hors bordereau. Une ligne par article déclaré qu'aucun champ du schéma ne couvre : slug normalisé comme identité, designation comme libellé affiché, confirmed et un compteur seen.
report_rows	ce qu'une machine additionne.
receptions	le registre du matériel reçu.
api_usage	la consommation des offres gratuites. Une ligne par appel de modèle : fournisseur, modèle, jetons consommés et, lorsque le fournisseur les publie, les compteurs de quota lus dans ses en-têtes.
service_samples	l'historique de disponibilité. Un échantillon par service et par intervalle, alimenté par l'échantillonneur du tableau de bord.

Table	**Rôle**
app_settings	les réglages vivants. Horaires des tâches, ordre des chaînes, tâches mises en pause, seuils d'alerte.
message_queue	la seconde file de reprise.
media_files	l'archive des médias.
chat_messages	l'archive des messages écrits. Le classeur ne contient que des rapports extraits ; cette table garde le texte reçu tel quel.
exchanges	la conversation qui n'a rien filé. Un « c'est qui ? » ou un « ok » n'a aucune ligne où apparaître.
notifications	la cloche du tableau de bord. Une ligne par compte : un rapport arrivé pour son chef, une ligne à vérifier, une écriture Sheets en échec, un rappel de tâche.
tasks	la liste partagée. Rappels récurrents, avec répétition personnalisée et rappel anticipé.
person_settings	les réglages par compte. Un document JSON par personne.
manager_reports	le dernier rapport produit pour chaque chef. Une ligne par (chef, période), écrite par l'ordonnanceur seul.
report_versions	l'historique d'édition humaine. manager_reports garde ce que le modèle a produit ; celle-ci garde ce qu'un humain en a fait, une ligne par enregistrement, jamais écrasée.
alert_mutes	la sourdine d'une alerte. Clé (chantier, article), et elle photographie le reste au moment du clic : toute aggravation fait ressurgir l'alerte d'elle-même.

Toutes les migrations sont idempotentes et rejouées à chaque démarrage. Celle qui a fusionné les comptes est encapsulée dans un bloc conditionnel dynamique : elle lit une colonne qu'elle supprime elle-même, et PostgreSQL planifie une instruction entière avant de l'exécuter.

3.5 Conception du service applicatif

3.5.1 Interface unique

Le service expose `POST /process`, acceptant **exactement une** de trois entrées : `audio`, `image` ou `text` — toute autre combinaison est un 422. L'aiguillage par type d'entrée est fait par n8n, jamais à l'intérieur du service. Les trois modes convergent vers le même pipeline d'extraction, ce qui rend la solution indépendante du canal.

Le service compte **85 routes** et **15 380 lignes** de Python, organisées en modules : `main.py` (routes), `extraction.py`, `journal.py`, `sheets.py`, `logs.py`, `auth.py`, `jobs.py`, `memory.py`, `mailer.py`, `reply.py`, `daily_report.py`, `docx_report.py`, `drive.py`, `stock.py`, `usage.py`, `prompt_loader.py`, `schemas.py`, `config.py`.

3.5.2 Patron Stratégie pour la transcription

Le choix du moteur étant arbitré par un banc d'essai, la conception rend les moteurs interchangeables. Le patron **Stratégie** est appliqué : une classe abstraite `TranscriptionProvider` définit le contrat, et chaque moteur est un adaptateur concret enregistré dans un registre par un décorateur `@register("nom")`. Les adaptateurs sont importés en bas de `providers/__init__.py` pour déclencher leur enregistrement.

Trois moteurs sont implémentés : `groq` (Whisper large-v3 par API), `gemini` (modèle multimodal natif) et `local` (client HTTP vers un service GPU dédié).

La **Figure 3.7** montre l'ordre de la chaîne de transcription et la condition de passage d'un moteur au suivant.

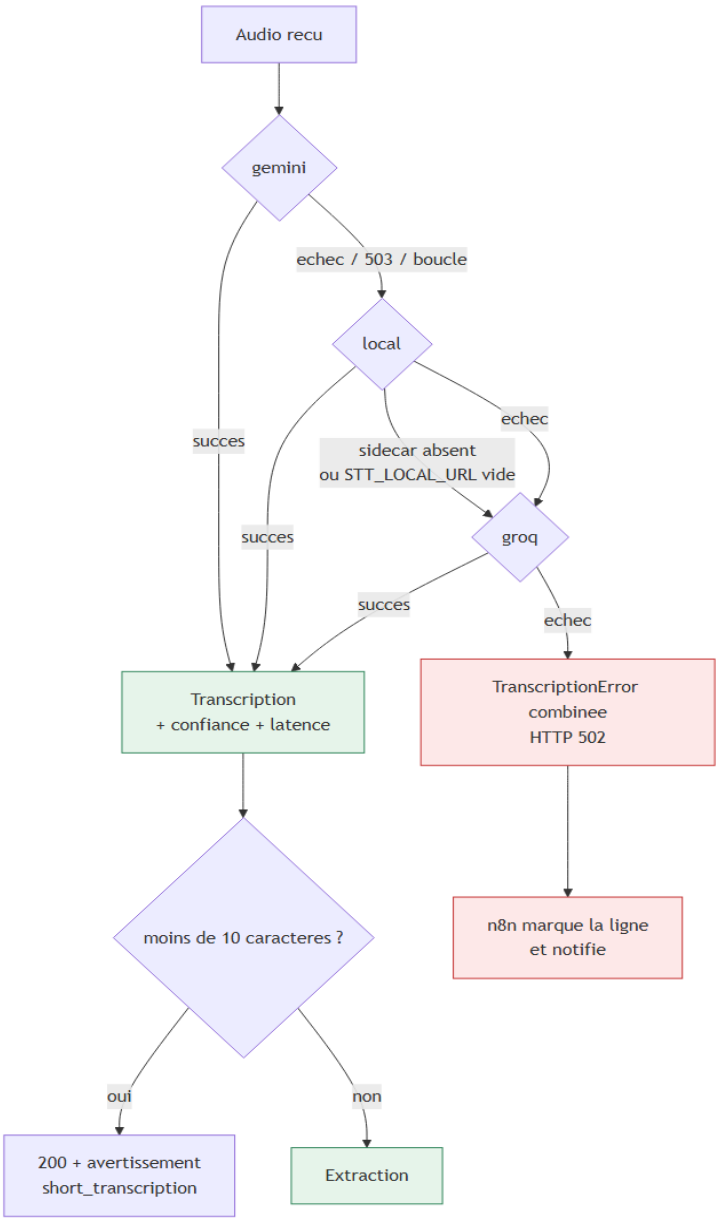


Figure 3.7 : *Chaîne de transcription. L'ordre est celui de l'exactitude mesurée, non de la latence : groq occupe la dernière place en tant que socle rapide et toujours disponible.*

3.5.3 Chaînes de repli

Le service exploite **trois** chaînes ordonnées `fournisseur:modèle`, une par usage : `extraction_text_chain`, `extraction_image_chain` et `profile_chain`, auxquelles s'ajoute la chaîne STT décrite ci-dessus. Chaque chaîne est essayée de gauche à droite, retourne quel modèle a répondu, et l'échec de toutes lève une `ExtractionError` combinée détaillant chaque cause — **jamais de repli silencieux**.

La chaîne texte **alterne délibérément les fournisseurs**, Gemini et Groq mesurant leurs quotas gratuits séparément ; la chaîne image est exclusivement Google, le catalogue gratuit de Groq ne comportant aucun modèle de vision — et c'est une fragilité réelle, non une préférence : ces trois noms partagent un seul compteur, si bien que les replis protègent contre la surcharge d'un modèle, jamais contre l'indisponibilité de Google.

Une chaîne de repli protège contre un moteur qui **échoue** ; elle est sans effet contre un moteur qui **répond avec succès et sans rien**. Ce cas n'est pas théorique : il est décrit au §4.3.9, et il est la raison de l'ordre actuel de la chaîne texte.

La **Figure 3.8** réunit les chaînes de repli et le traitement réservé à leurs erreurs.

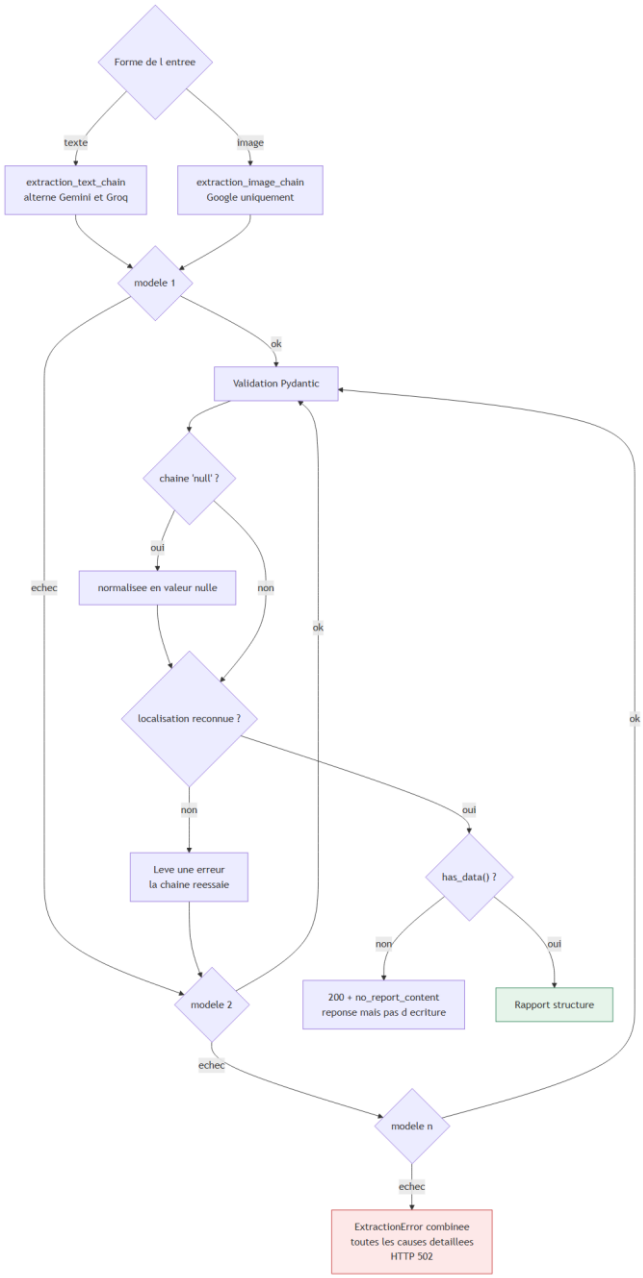


Figure 3.8 : *Les chaînes de repli d'extraction.*

3.5.4 Contrat d'erreurs

Le **Tableau 3.3** fixe le contrat d'erreurs du service, code par code.

Tableau 3.3 : Contrat d'erreurs du service.

Situation	Code HTTP	Comportement attendu de n8n
Entrée invalide (ni audio, ni image, ni texte ; fournisseur inconnu)	422	Erreur de configuration à corriger
Défaillance d'un moteur (`TranscriptionError`, `ExtractionError`)	502	Marquage de la ligne en anomalie, notification
Échec de transport de la réponse au technicien	502	Distinguer une panne externe d'un défaut du service
Transcription anormalement courte (moins de 10 caractères)	200 + `short_transcription`	Ligne enregistrée mais signalée
Message sans contenu exploitable	200 + `no_report_content`	Réponse au technicien, **pas** d'écriture dans le journal
Moteur indisponible ou quota épuisé, message mis en file	**202** + `queued` ou `quota`	Chemin de succès : le technicien est prévenu que son rapport est reçu et sera enregistré

La distinction 422/502 sépare les erreurs de l'appelant des défaillances des moteurs externes ; l'avertissement plutôt que l'erreur illustre le principe : un signal de qualité douteuse est un avertissement structuré, ni une erreur bloquante ni un silence.

3.5.5 Les instructions comme contenu versionné

Les instructions destinées aux modèles vivent dans `api/app/prompts/` en **Markdown versionné**, non en littéraux Python : un changement d'instruction est un changement de contenu et doit être visible en revue. Cinq fichiers : `00-soul.md`, `extraction-text.md`, `extraction-image.md`, `profile.md`, `stt-vocabulary.md`.

`00-soul.md` est préfixé à toute instruction adressée à un modèle : identité, interdiction d'inventer, « absence n'est pas zéro ». Les fichiers sont lus une fois et mis en cache, ce qui impose de redémarrer le service après une modification — propriété voulue : une instruction qui pourrait changer entre deux requêtes rendrait une mauvaise extraction impossible à reproduire.

`stt-vocabulary.md` est la seule exception et ne reçoit **pas** l'âme : Whisper interprète son paramètre `prompt` comme la transcription *précédant* le clip, et non comme des consignes ; l'âme serait recopiée dans la transcription. Le chapitre 4 montre que cette subtilité a des conséquences bien plus graves encore en local.

3.5.6 Le profil du technicien : un cinquième modèle

Un paragraphe en français par technicien est rédigé chaque nuit à 03:00 à partir de ses propres rapports passés, stocké dans `persons.profile` et injecté comme contexte étiqueté dans l'instruction d'extraction.

Trois décisions le rendent sûr. Il n'existe **aucune table de faits mémorisés** : le Sheet est déjà le registre permanent, et un second magasin qui en dériverait serait un cache sans chemin critique pour le justifier. Il ne s'exécute **jamais sur le chemin de réponse** : un technicien qui attend son résumé ne doit pas en plus attendre qu'on range son profil. Enfin, l'instruction **interdit les nombres** et le bloc d'injection présente le texte comme informatif et prohibé comme source de valeurs — sans quoi « intervient surtout sur Sidi Maarouf » est à un pas de remplir `chantier` sur un message qui n'a jamais nommé de site.

3.5.7 Le journal de chantier

`journal.py` tient **une ligne par message** dans l'onglet du chantier concerné, à l'image du fichier du directeur. Deux règles en assurent la justesse.

Ajout, jamais réécriture. Chaque message occupe sa propre ligne, y compris le second message d'une même journée. L'autre conception possible — fusionner le second dans le premier, additionner les nombres et concaténer la prose — détruit de l'information : un chiffre erroné dicté l'après-midi écrase sur place un chiffre juste du matin, sans qu'il reste rien à quoi le comparer. Avec une ligne par message, une correction fausse se supprime sans emporter la valeur correcte qu'elle prétendait corriger.

Une colonne qui se crée toute seule. Le schéma d'extraction connaît trente-cinq champs de quantité ; un chantier réel en dicte davantage. Sans un champ pour les accueillir, un rapport nommant quarante pièces réparties sur huit types d'article dépose une ligne **vide des huit** : la quantité est correctement extraite, puis abandonnée faute d'un endroit où la ranger, et aucune erreur n'est levée. C'est une perte parfaitement silencieuse — exactement le défaut que le reste du système s'interdit.

Le champ `articles_libres` recueille ces articles, `{désignation : quantité}`. Il est **volontairement en texte libre**, à la différence de `livraisons`, qui est validé contre la liste des champs connus : le sujet même de ce champ est l'article *pas encore connu*, et le valider contre la liste reviendrait à jeter précisément l'information qu'il existe pour porter. Au dépôt de la ligne, `sheets.ensure_columns` crée la colonne manquante dans l'onglet du chantier et la valeur y est écrite.

Trois décisions rendent le procédé sûr plutôt que chaotique :

- **Une identité normalisée, et non le libellé dicté.** « prises doubles étanches » et « prise double étanche encastrée » se réduisent au même *slug* — accents, casse, pluriels et mots vides retirés — de sorte que l'onglet ne gagne pas une seconde colonne parce qu'un technicien a mis un pluriel. Le premier libellé enregistré l'emporte jusqu'à ce qu'un humain en confirme un meilleur.

- **Le registre est global, la colonne est locale.** `article_registry` est commun à tous les chantiers alors que la colonne appartient à l'onglet qui l'a déclarée : le *nom* est partagé, si bien qu'un second site déclarant la même chose réutilise l'orthographe existante au lieu d'en inventer une parallèle.
- **La validation vient après, jamais avant.** La quantité est comptée immédiatement ; la page Chantiers propose ensuite de confirmer le libellé, de le corriger — un libellé corrigé vers un libellé existant **fusionne les deux colonnes** — ou de le supprimer. **Un rapport n'attend jamais une validation** : faire dépendre l'écriture d'une approbation humaine, c'est réintroduire au centre de la chaîne le délai que ce projet supprime.

L'échec de la création de colonne est traité comme tel : il émet un avertissement `article_column_failed` et la ligne est écrite quand même. Une colonne ne vaut pas un rapport.

Réconciliation de stock. La valeur de référence est *dernier stock connu moins quantité installée* ; un chiffre rapporté par le technicien l'emporte toujours et devient la nouvelle référence ; un écart émet un avertissement `stock_ecart` chiffré **au lieu de choisir silencieusement** — car une livraison et une perte sont arithmétiquement identiques, et seul un humain peut les distinguer.

La **Figure 3.9** décrit la tenue du journal, de la normalisation de la date à la réconciliation du stock.

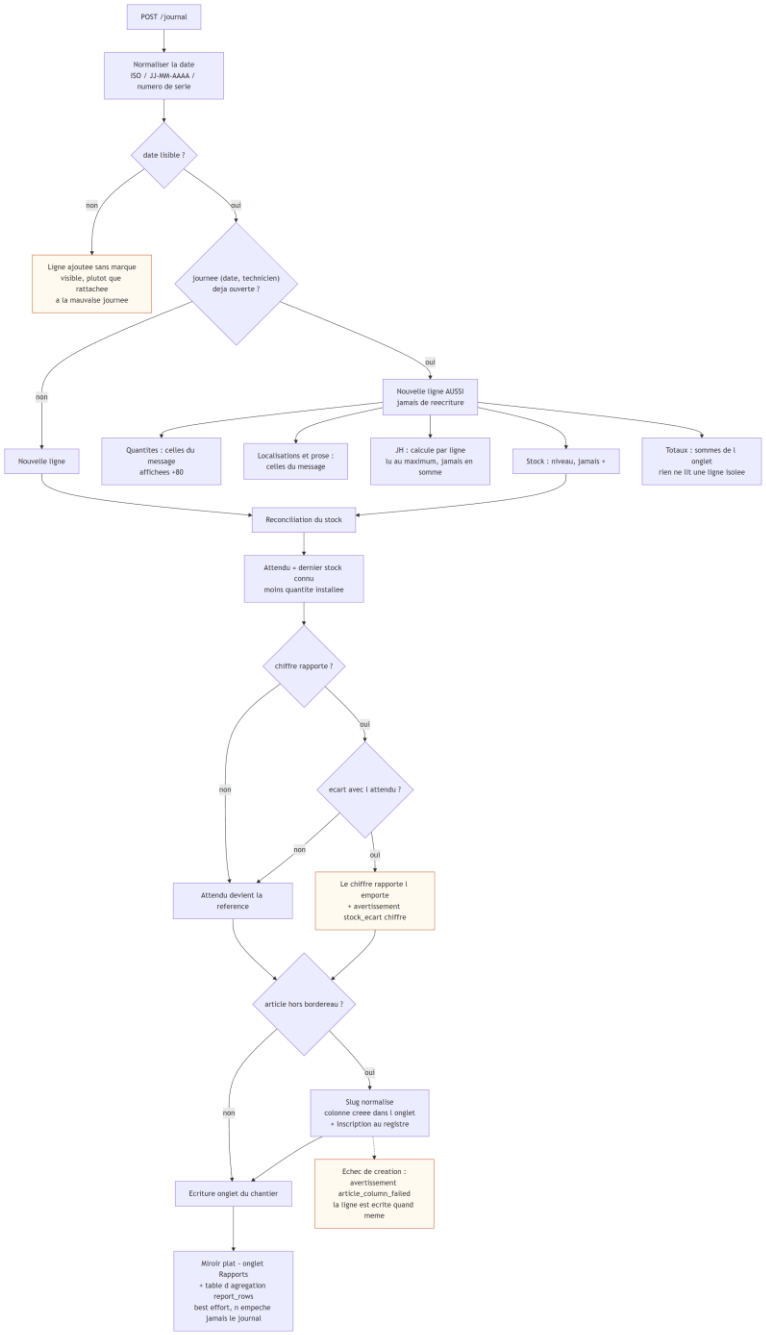


Figure 3.9 : *Tenue du journal : une ligne par message, réconciliation de stock et création de colonne à la demande.*

3.5.8 Les tâches planifiées

`jobs.py` implémente un ordonnanceur : un fil d'exécution, un battement de 30 s, **neuf tâches**. Le **Tableau 3.4** en donne la cadence et le budget.

Tableau 3.4 : Les neuf tâches planifiées, leur cadence et leur budget de temps.

Tâche	Cadence	Budget	Ce qu'elle fait
`message_queue`	30 s	480 s	Rejoue les messages garés faute de moteur disponible (§3.5.12)
`sheet_queue`	5 min	120 s	Vide la file des écritures Sheets refusées
`watch_alerts`	1 min	300 s	Veille matière : fait sonner la cloche une fois par état d'alerte
`refill_articles`	1 h	600 s	Rattrape les articles libres qu'une extraction n'avait pas rangés
`reminders`	1 min	60 s	Rappels de la liste de tâches partagée
`prune_logs`	03:00	120 s	Purge des journaux, échantillons, médias et notifications
`profiles`	03:00	300 s	Réécriture des profils techniciens
`daily_report`	`daily_time`, lundi-samedi	600 s	Rapport quotidien par courriel
`weekly_report`	`weekly_day` / `weekly_time`	600 s	Rapport hebdomadaire par courriel

Chaque budget est dimensionné sur le coût réel de la tâche : 600 s pour les rapports, qui enchaînent une lecture du Sheet, un envoi SMTP et un appel de modèle **par chef de chantier** ; 480 s pour la file des messages, parce qu'un rejeu peut payer le chargement à froid du moteur local.

Les horaires sont **modifiables depuis le tableau de bord** sans redémarrage, la ligne de réglages étant relue à chaque battement. `watch_alerts` est la seule dont la cadence est figée, et pour une raison : c'est elle qui remarque une pénurie, et une veille qu'on peut espacer est une veille qu'on finit par ne plus avoir. **Chaque tâche peut par ailleurs être mise en pause individuellement** — une pause arrête l'*ordonnanceur*, jamais l'opérateur : `POST /jobs/{nom}/run` continue de fonctionner, parce que « ne la lance pas toute seule » et « ne la lance plus jamais » sont deux instructions différentes et que seule la première est donnée.

La **Figure 3.10** détaille l'ordonnanceur et la réclamation atomique qui garantit un résultat unique par exécution.

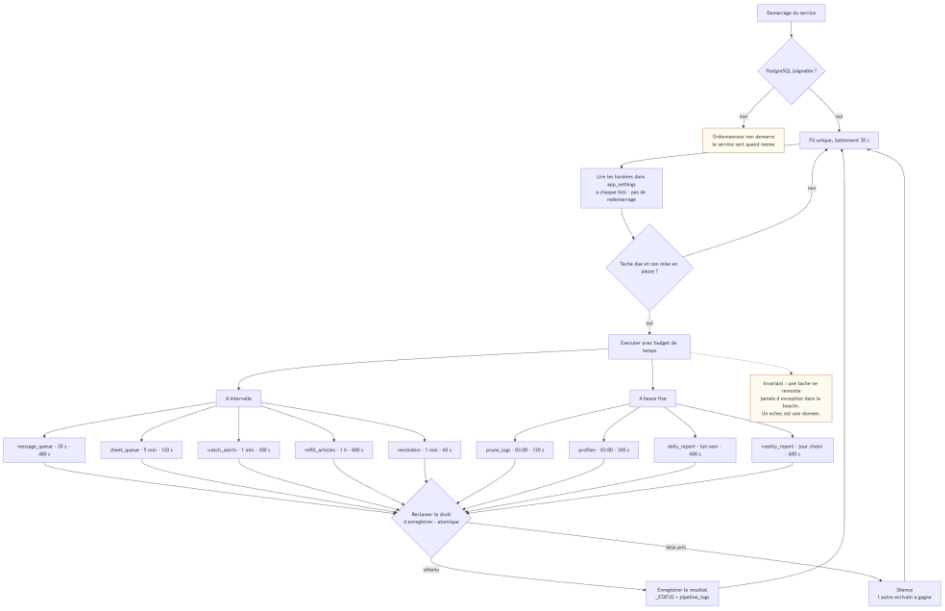


Figure 3.10 : *Ordonnanceur : neuf tâches, un fil unique, et une réclamation atomique du droit d'enregistrer le résultat — sans laquelle une tâche terminée juste au seuil du délai inscrivait deux issues contradictoires pour une même exécution.*

3.5.9 Les avertissements : deux familles, deux destins

Le système produit deux sortes d'avertissements. Elles ne vivent pas au même endroit, elles ne s'écrivent pas au même moment, et les confondre conduirait à chercher une colonne pour un chiffre qui n'en veut pas.

Famille A — les alertes matière (`stock.py`, clé `niveau`). Elles répondent à « qu'est-ce qui va manquer ». Elles sont **recalculées à la demande** depuis le journal, le registre `receptions` et le bordereau, et **stockées nulle part**. Elles alimentent le corps du courriel quotidien et hebdomadaire, le tableau du document Word et la route `GET /alerts` que lit la page Chantiers. Quatre niveaux, classés dans cet ordre :

- `depassement` — la quantité posée dépasse la quantité au marché. Ce niveau ne dépend pas du stock connu : sur-consommation ou erreur de saisie, le directeur doit l'entendre dans les deux cas.
- `critique` — au plus deux jours de couverture au rythme actuel, ou au plus 5 % du marché lorsqu'aucun rythme n'est exploitable.
- `bas` — au plus cinq jours, ou au plus 10 %.
- `surplus` — plus de deux fois ce qu'il reste à poser, **et** un excédent représentant plus de 10 % du marché.

L'ordre de ces tests porte une décision. `surplus` est évalué **avant** les niveaux de pénurie, parce qu'en fin de chantier un article peut se trouver simultanément sous deux jours de couverture *et* en excès de plusieurs fois le reste à poser : « commandez-en » serait alors exactement la phrase inverse

de celle qu'il faut dire. Sous deux jours actifs il n'existe pas de rythme fiable — un chantier dont toute la livraison est arrivée le premier jour est approvisionné, pas suralimenté — et le calcul se replie sur le pourcentage ; sans quantité au marché il n'y a pas de dénominateur, et le système se tait plutôt que d'inventer un repère. Les seuils sont surchargeables dans `app_settings` sans redémarrage.

Famille B — les avertissements de ligne (`journal.WARNING_LABELS`). Elles répondent à « que faut-il vérifier sur cette ligne-ci », et celles-là **ont** une colonne : `Motif du flag` dans l'onglet du chantier, `motif_flag` dans l'onglet Rapports, plusieurs motifs étant joints par un point médian. Elles proviennent de deux processus qui convergent sur la même ligne : `/process` transmet les siennes à travers n8n, `file_report` y ajoute les siennes au moment de l'écriture.

Dix clés existent, et **cinq seulement lèvent le drapeau de vérification** :

Avertissement	Lève `Flag review`	Ce qu'il dit
`image_input`	oui	La ligne a été lue sur une photo
`photo_zones_partielles`	oui	Zones lues sur un plan : la liste peut être incomplète
`low_confidence_transcription`	oui	La transcription est peu sûre
`short_transcription`	oui	Transcription trop courte pour être un rapport
`chantier_cree_non_verifie`	oui	Le chantier a été créé depuis un nom entendu
`stock_ecart`	non	Le stock déclaré contredit le stock calculé
`no_report_content`	non	Aucune donnée exploitable dans le message
`heure_non_dite`	non	Une heure inventée par le modèle a été retirée
`date_declaree_illisible`	non	La date énoncée n'a pas pu être lue
`date_declaree_hors_plage`	non	La date énoncée sort de la plage acceptable

3.5.10 Ignorer une alerte sans l'effacer

Une alerte matière permanente à l'écran pose une question à laquelle le système devait répondre : que fait un directeur qui a **déjà commandé** le câble qui manque ? Sans réponse, il apprend à ne plus lire le bandeau, et le jour où une alerte nouvelle s'y ajoute, elle ne se distingue plus des trois qu'il ignore déjà.

3.5.11 La matière reçue n'est pas la matière posée

L'instruction de lecture d'image demandait au modèle de reporter un **bon de livraison** dans les colonnes de quantités *installées*. Cinq cents mètres de câble livrés étaient donc comptabilisés comme cinq cents mètres posés — gonflant précisément le nombre dont `GET /chantiers/{id}/bordereau` déduit l'avancement. Le défaut est resté latent tant qu'aucune colonne d'article ne portait de valeur.

3.5.12 La file d'attente des messages

Le système comporte **deux** files de reprise, et les confondre serait une erreur de lecture : `failed_writes` rattrape une écriture Google refusée, une fois le rapport produit ; `message_queue` rattrape un message dont le traitement lui-même n'a pas abouti — aucun moteur de transcription disponible, quota d'extraction épuisé chez tous les fournisseurs. La première protège le classeur, la seconde protège le message.

Le mécanisme tient en deux décisions principales.

3.6 Conception du workflow d'orchestration

3.6.1 Structure

Le workflow unique compte **64 nœuds**, dont **54 portent de la logique** et dix sont des notes adhésives qui documentent le graphe à l'écran. Ils couvrent : le déclencheur périodique Telegram et son *offset* persistant ; le webhook WhatsApp et sa poignée de main de vérification ; la corrélation et la déduplication ; l'identification de l'expéditeur et le contrôle d'autorisation ; l'accusé de réception ; l'aiguillage média/texte puis audio/image ; les appels `/process` ; les réponses de résumé, d'erreur et de refus ; l'appel `/journal` et l'archivage des échanges sans contenu exploitable ; la journalisation de chaque étape ; et un déclencheur d'erreur.

Le workflow ne comporte **aucune commande textuelle**, et Telegram ignore tout message commençant par une barre oblique. Des commandes `/rapport`, `/modifier` et `/supprimer` constitueraient une seconde voie, analysée à la main, vers des données que le tableau de bord édite déjà : deux écrivains pour une même ligne sont un coût permanent que rien ne justifie.

La **Figure 3.11** présente la première moitié du workflow, consacrée à la réception et à l'identification de l'expéditeur.

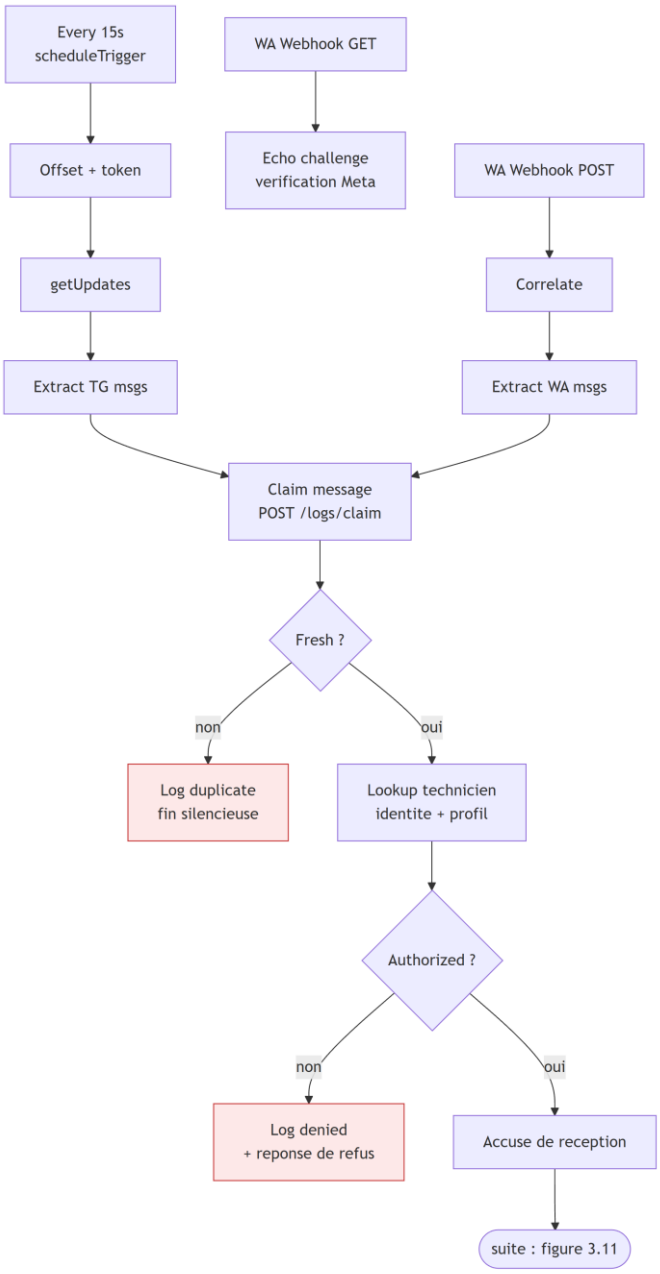


Figure 3.11 : *Workflow d'orchestration : les 64 nœuds regroupés par fonction.*

La **Figure 3.12** en donne la seconde moitié, du traitement jusqu'à l'écriture du journal.

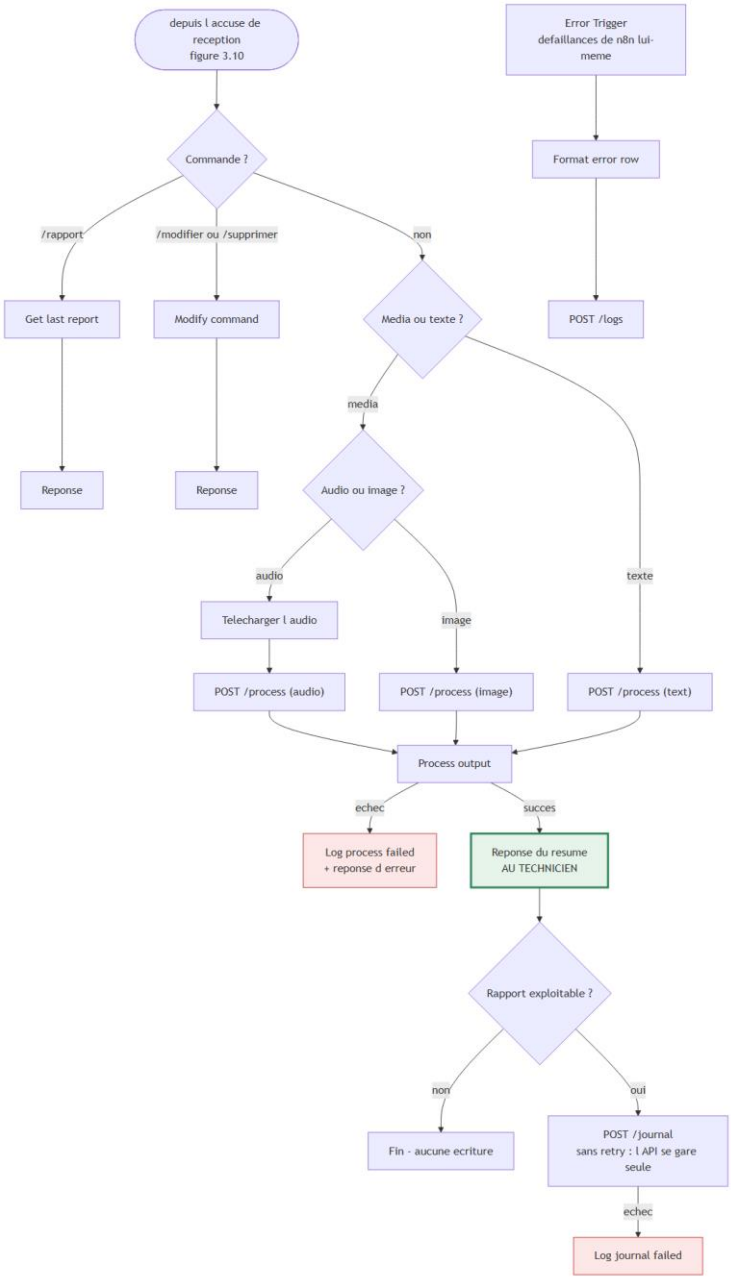


Figure 3.12 : *Workflow, seconde moitié : aiguillage, traitement, réponse au technicien puis journal.*

Le gestionnaire d'erreurs est resté dans n8n alors que les tâches planifiées en sont sorties, pour une raison précise : il intercepte les défaillances **du workflow lui-même**, que le service ne peut pas observer.

3.6.2 L'ordre réponse-d'abord

Dans le workflow, la réponse au technicien précède l'appel `/journal`. Ce n'est pas un détail d'agencement mais un invariant : une écriture Sheets en échec se gare dans la file de reprise, est

rejouée toutes les 5 minutes par le même chemin que le flux vivant, et n'est résolue que lorsque Google a effectivement accepté la ligne. **Elle ne coûte jamais au technicien son résumé.**

Le nœud qui vérifie qu'un message est exploitable est placé **après** le nœud de réponse, pour la même raison : un « salam » mérite une réponse, mais pas une ligne de blancs dans le journal.

3.7 Conception du tableau de bord

Le tableau de bord (16 139 lignes de TypeScript et de CSS) est une application React 19 servie en statique.

Toutes les chaînes d'interface vivent dans `locales/{fr,en,ar}.json`, le français faisant référence. L'arabe impose le sens droite-à-gauche, ce qui interdit les marges directionnelles au profit des propriétés logiques.

L'organigramme dessine trois bandes fixes indexées sur le rôle — directeur, chefs, techniciens — et non une récursion : c'est pourquoi aucun garde-fou anti-cycle n'est nécessaire. La réaffectation utilise le glisser-déposer natif HTML5 ; comme celui-ci ne fonctionne ni au toucher ni au clavier, la liste déroulante du panneau de détail est le chemin **réel** pour ces deux modes, non un pis-aller.

La lecture des données passe par un cache au niveau du module, afin qu'une navigation aller-retour se repeigne instantanément puis se revalide en arrière-plan. Le sondage d'état et l'échantillonneur de disponibilité vivent **au-dessus** du routeur : les placer dans une page les arrêterait à la navigation et creuserait de faux trous dans l'historique de disponibilité.

Douze pages composent l'application : vue d'ensemble, réception, rapports, rapport quotidien, rapport hebdomadaire, automatisation, tâches, journaux, file de reprise, **consommation**, équipe et **chantiers**. Les deux dernières sont aussi les plus récentes et répondent à des questions que le reste de l'interface ne posait pas. **Chantiers** rassemble ce qui relève de l'administration d'un site — alertes matériel, sites et articles à confirmer, bordereau et son avancement — là où la vue d'ensemble répond à « que se passe-t-il aujourd'hui ». **Consommation** montre ce qu'il reste de chaque offre gratuite. Aucune des deux ne recalcule dans le navigateur un chiffre que le service a déjà arrêté : elles l'affichent, avec sa provenance.

Deux mécanismes traversent toutes les pages. La **palette de commandes** ouvre la navigation, la recherche d'un rapport et les actions au clavier. La **cloche** porte les notifications du compte connecté : un rapport arrivé, une ligne à vérifier, une pénurie remarquée par la veille matière, un rappel de tâche — chaque compte ne voyant que les siennes.

- **Le bandeau d'alertes matière est permanent**, en haut, limité à trois lignes. Une pénurie est la seule information de cette page qui porte une échéance ; derrière un compteur à ouvrir, c'est une information qu'on apprend à contourner.
- **La liste de gauche est ordonnée par ce qui réclame une action** — sites non confirmés d'abord, puis nombre d'alertes — et chaque ligne porte ses compteurs. Une liste triée seulement par ordre alphabétique oblige à parcourir chaque ligne pour trouver celle qui attend quelque chose.

- **Le matériel déclaré appartient au site qui l'a déclaré.** La file de validation était globale : elle présentait des libellés sans indiquer d'où ils venaient, et ne montrait que les libellés *non confirmés*, si bien qu'un article disparaissait de l'écran au moment précis où quelqu'un l'acceptait — la seule action qui prouve qu'il est réel. Le volet d'un site liste désormais tous ses articles libres, confirmés ou non, avec la quantité cumulée. L'action, elle, reste globale, puisque le registre est indexé sur un *slug* : une note sous le titre le dit, car un bouton dont la portée dépasse le panneau qui l'accueille doit l'avouer.

Les captures du tableau de bord en fonctionnement sont rassemblées au §4.7, afin d'être lues à côté des résultats plutôt qu'au milieu de la conception.

3.8 Sécurité

Tout est protégé. Deux voies d'accès coexistent : un cookie de session pour les humains, un en-tête `X-API-Key` pour n8n, qui appelle le service à chaque message et n'a pas de navigateur. Trois niveaux de dépendance les distinguent : `require_user` accepte les deux, `require_human` rejette la machine (réglages, corrections, comptes), `require_director` restreint davantage.

Le contrôle est placé sur le service et non sur le formulaire de connexion : le tableau de bord étant un paquet statique, un écran de connexion seul serait contournable depuis les outils de développement.

Décisions complémentaires :

- **Sessions opaques en base**, non des jetons auto-portants : supprimer la ligne déconnecte ce navigateur dès sa requête suivante.
- **Clé machine non définie signifie accès machine *désactivé***, jamais « tout le monde est une machine ».
- **Origines croisées restreintes**, jamais un joker — un joker et l'envoi de cookies d'authentification sont de toute façon incompatibles.
- **Limitation de débit sur la connexion** : 8 échecs par 5 minutes et par adresse cliente, l'adresse étant résolue à travers un nombre configuré de relais de confiance. Une valeur erronée laisse soit un script faire tourner l'en-tête, soit enferme tous les visiteurs derrière une même clé.
- **Secrets hors du dépôt** ; le mot de passe PostgreSQL n'a délibérément **aucune valeur par défaut**, afin qu'un clone sans configuration échoue bruyamment plutôt que de démarrer avec un mot de passe connu.
- **Moindre privilège** : le compte de service Google n'accède qu'au classeur concerné ; les empreintes de mots de passe ne quittent jamais la base, les requêtes de lecture nommant leurs colonnes plutôt que de tout sélectionner.
- **Signature des webhooks WhatsApp vérifiée.** Meta signe chaque livraison (`x-hub-signature-256`) ; le workflow recalcule l'empreinte HMAC-SHA256 du corps reçu et compare les deux en temps constant. Sans ce contrôle, quiconque apprend le nom d'hôte public peut publier une charge utile se réclamant du numéro d'un technicien, et elle atterrit

dans le journal sous son nom : le jeton de vérification n'authentifie que la souscription initiale, pas les messages, et l'autorisation en aval ne repose que sur le numéro contenu dans le corps. Le contrôle **échoue ouvert et bruyamment** si le secret n'est pas configuré : échouer fermé couperait la réception au premier import du workflow sans la variable, et « WhatsApp devient muet » est la panne la plus difficile à diagnostiquer de ce projet — le journal dit alors explicitement que la porte est ouverte.

La **Figure 3.13** résume le contrôle d'accès et les trois niveaux d'exigence qu'il applique.

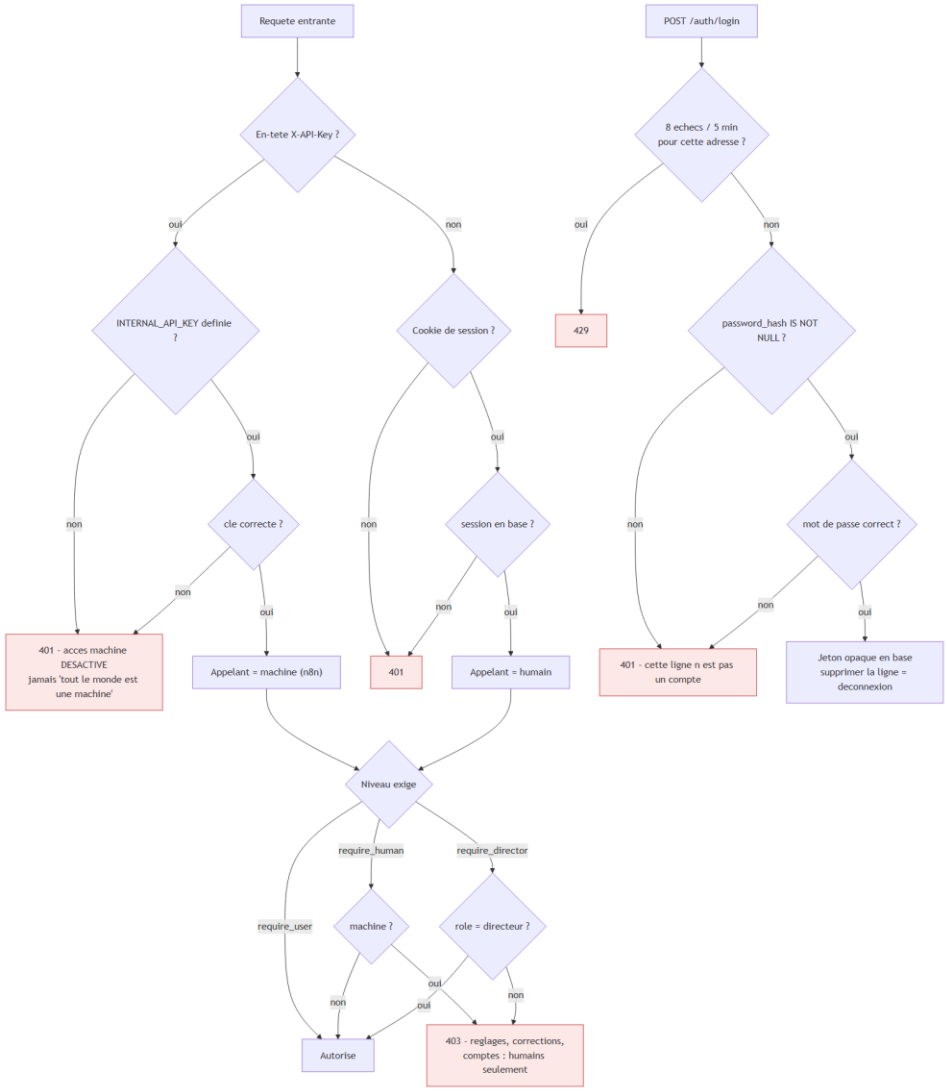


Figure 3.13 : *Contrôle d'accès. Les deux voies — cookie humain et clé machine — convergent vers trois niveaux d'exigence.*

3.9 Protection des données personnelles

Le système enregistre la voix de salariés identifiés, la rattache à leur nom et la conserve. Il s'agit de **données à caractère personnel** au sens de la loi marocaine **09-08** relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel, dont l'autorité de contrôle est la **CNDP**.

Un enregistrement vocal est en outre une donnée biométrique par nature, ce qui appelle une prudence particulière.

3.9.1 Ce qui est collecté, et où

Le **Tableau 3.5** recense les données personnelles traitées, leur finalité et leur lieu de conservation.

Tableau 3.5 : Données personnelles traitées par le système.

Donnée	Emplacement	Durée de conservation
Enregistrement audio	PostgreSQL, table des médias	Purgée par la tâche de rétention
Transcription et rapport structuré	Google Sheets, journal du chantier	Permanente — c'est le registre métier
Nom, rôle, rattachement hiérarchique	PostgreSQL, `persons`	Durée de la relation de travail
Identifiants de messagerie	PostgreSQL, `technicians`	Désactivables sans suppression
Profil rédigé automatiquement	PostgreSQL, `persons.profile`	Réécrit chaque nuit
Journal d'étapes, sessions, visites	PostgreSQL	Purgés à 03:00 par la tâche de rétention

3.9.2 Les principes appliqués

Finalité déterminée. Le traitement a un objet unique et explicite : produire le rapport journalier de chantier. Aucune donnée n'est exploitée à une autre fin, et notamment pas pour évaluer individuellement un salarié.

Minimisation. Le schéma d'extraction ne comporte que des quantités, des localisations et des remarques de chantier. Le profil technicien, qui est la donnée dérivée la plus sensible, **interdit explicitement les nombres** dans son instruction de rédaction et est cantonné à un rôle de contexte, sans jamais pouvoir alimenter une valeur du rapport.

Limitation de la conservation. Une tâche de rétention supprime chaque nuit les journaux et les échantillons au-delà de leur fenêtre. Le registre métier, lui, est conservé : c'est un document d'entreprise, au même titre que le fichier de suivi qu'il remplace.

Accès restreint. Les trois niveaux d'autorisation décrits au §3.8 s'appliquent : un technicien n'a aucun accès à l'interface, un chef accède à son périmètre, seul le directeur administre. Les sessions sont révocables individuellement.

Sécurité. Empreintes de mots de passe jamais exposées, secrets hors du dépôt, une seule origine publiée, limitation de débit sur la connexion.

3.9.3 Ce qui reste à faire avant une mise en production

Ces points relèvent de l'entreprise et non de la technique, et sont énoncés ici parce qu'ils conditionnent un déploiement réel :

1. **Déclaration du traitement auprès de la CNDP** avant toute exploitation en production.
2. **Information des personnes concernées** : les techniciens doivent être informés de la finalité, des destinataires, de la durée de conservation et de leurs droits d'accès, de rectification et d'opposition.
3. **Formalisation d'une durée de conservation** des enregistrements audio, actuellement fixée par un paramètre technique et non par une politique écrite.
4. **Encadrement du transfert hors du Maroc** : la transcription et l'extraction utilisent des services hébergés à l'étranger. C'est le point le plus sensible du dossier — et l'argument le plus fort en faveur du moteur local, qui traite l'audio **sans qu'il quitte la machine de l'entreprise**.

Ce dernier point mérite d'être souligné : le moteur de transcription local, retenu au chapitre 4 pour son exactitude, se trouve également être le seul qui n'expose aucune donnée à un tiers. Un choix technique fait pour la qualité s'avère aussi le meilleur choix pour la conformité.

3.10 Ce que lit le modèle, et ce qu'il n'a pas le droit d'en faire

Les sections précédentes ont décrit les composants un par un. Cette section les regarde ensemble, du point de vue du modèle : que reçoit-il exactement, dans quel ordre, et qu'est-ce qui lui est explicitement interdit.

Quatre sources entrent dans une extraction, et une seule est le message. Le message lui-même — transcription, texte ou photo — arrive accompagné de trois contextes qui ne viennent pas de lui :

- **`00-soul.md`**, préfixé à toute instruction adressée à un modèle : l'identité du système, l'interdiction d'inventer, et la règle qu'une absence n'est pas un zéro. Il est versionné comme du contenu et non écrit dans le code, parce qu'un changement d'instruction est un changement de comportement et doit être visible en revue.
- **L'instruction d'extraction** propre à la forme de l'entrée, `extraction-texte.md` ou `extraction-image.md`.
- **Le profil du technicien**, un paragraphe écrit chaque nuit à partir de ses rapports passés.

Une quatrième source ne suit pas ce chemin et c'est délibéré : `stt-vocabulary.md`, la liste des mots métier, **ne reçoit pas l'âme**. Whisper lit son champ `prompt` comme le texte qui *précède* le clip, et non comme des instructions ; l'âme s'y retrouverait recopiée dans la transcription.

Le profil est le seul contexte contre lequel le système se protège de lui-même. Il est injecté encadré d'une consigne qui le déclare informatif et interdit de s'en servir comme source de valeurs. Sans cette précaution, « intervient surtout sur Sidi Maarouf » est à un pas de remplir le champ `chantier` sur un message qui n'a jamais nommé de site — c'est-à-dire de produire une donnée

plausible, fausse, et indiscernable d'une donnée dictée. C'est aussi pourquoi l'instruction qui rédige les profils **interdit les nombres** : un profil n'a pas à savoir compter.

La sortie se sépare en deux immédiatement. Le champ `resume` part vers la réponse au technicien, *avant* toute écriture ; tout le reste passe par deux aiguillages successifs. `nature` décide si le message décrit une pose ou une réception — une réception n'écrit aucune ligne de journal mais alimente le registre. `has_data()` décide ensuite si ce qui reste mérite une ligne : « salam » s'extrait parfaitement, tous les champs à `null` et un résumé poli, et sans ce garde-fou la chaîne répondait « rapport enregistré » puis déposait une ligne de blancs.

La **Figure 3.14** rassemble ce parcours : les trois formes d'entrée, les contextes injectés, les deux chaînes de modèles, et les trois issues possibles.

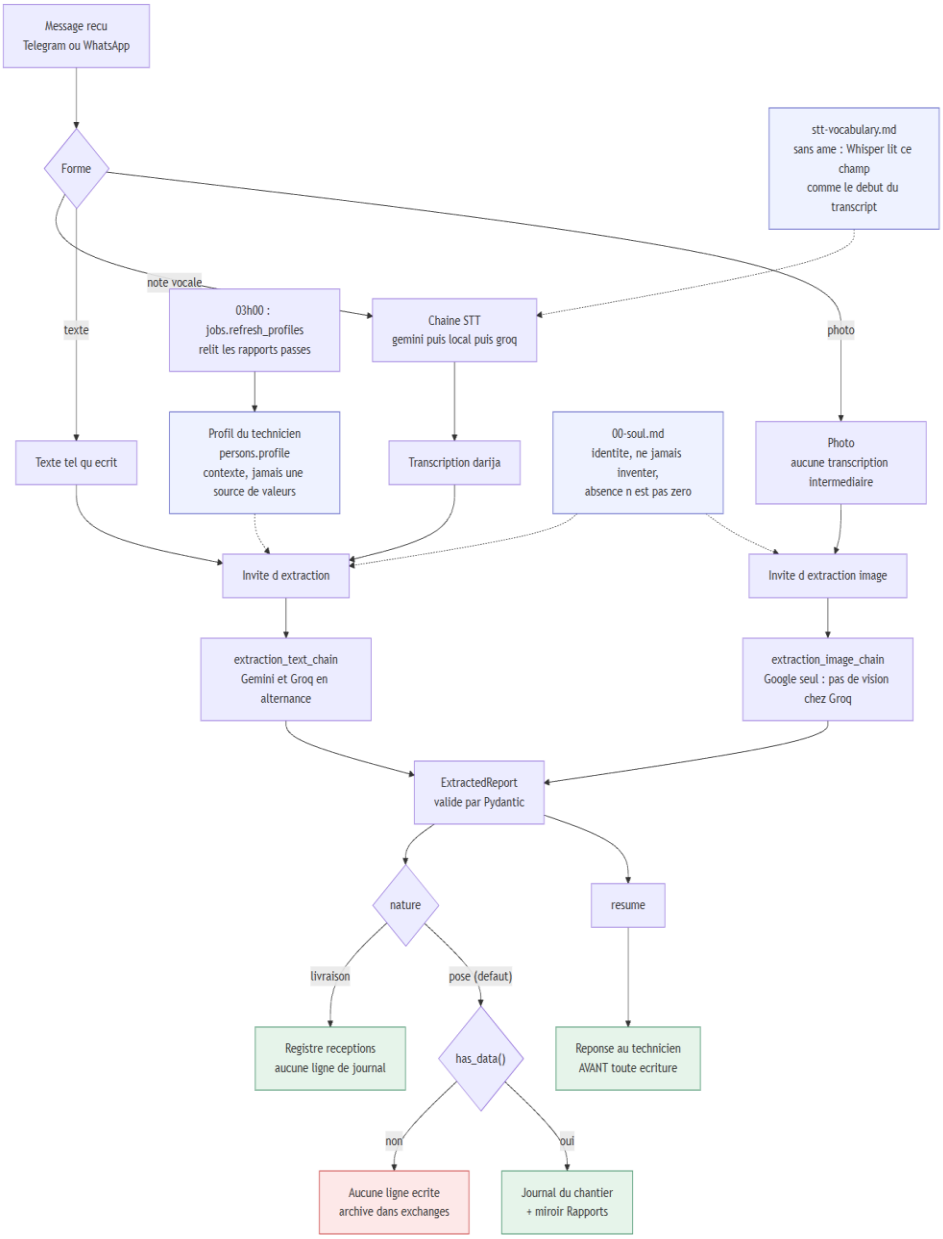


Figure 3.14 : *Ce que lit le modèle. En bleu, les contextes injectés qui ne viennent pas du message : l'âme, l'instruction, le profil du technicien et le vocabulaire de transcription — ce dernier étant le seul à ne pas recevoir l'âme, parce que Whisper le lirait comme un début de transcription.*

3.11 Conclusion du chapitre

La conception traduit les contraintes du projet en structures : une architecture hybride dont la frontière est fonctionnelle et a été déplacée par l'expérience, un contrat de données reconstruit sur le fichier réel du client, un patron Stratégie qui a permis au banc d'essai de décider, des chaînes de repli explicites, un contrat d'erreurs qui refuse les échecs silencieux, un journal dont les règles

d'ajout et de réconciliation protègent le seul chiffre qui compte, et un modèle relationnel qui fait d'un humain une seule ligne. Le chapitre suivant décrit la mise en œuvre, le banc d'essai et les résultats.

Chapitre 4 : Réalisation, banc d'essai, tests et résultats

4.1 Introduction du chapitre

Ce chapitre décrit la mise en œuvre de la conception : l'environnement de développement, la réalisation du service et du workflow, le banc d'essai comparatif des moteurs de transcription du darija — cœur scientifique du projet — les difficultés techniques rencontrées et leur résolution, la stratégie de test, puis les résultats obtenus.

4.2 Environnement et outils de développement

Le **Tableau 4.1** récapitule l'environnement de développement.

Tableau 4.1 : Environnement de développement.

Outil	Rôle
Python 3.11 / FastAPI / Pydantic	Service applicatif
pytest	Tests automatisés (860 tests)
PostgreSQL 16	Journal d'événements, idempotence, sessions, organigramme
React 19 / TypeScript / Vite / Tailwind v4	Tableau de bord
Docker / Docker Compose	Exécution de la pile complète (5 services)
n8n	Orchestration des workflows
Git / GitHub Actions	Versionnage, intégration continue (18 contrôles), Dependabot, analyse SonarCloud
PyTorch 2.5 / CUDA / transformers	Banc d'essai local sur GPU
Groq API, Google AI Studio, Hugging Face	Inférence (offres gratuites)
Obsidian	Base de connaissances du projet (§4.2.2), en Markdown dans le dépôt

Le dépôt est organisé en modules : `api/` (service), `Dashboard/` (tableau de bord), `n8n/` (workflow exporté et documentation), `benchmark/` (banc d'essai), `docs/` (cahier des charges, guides), `vault/` (base de connaissances, §4.2.2) et `Pres+report/` (source de ce document et du diaporama).

4.2.1 Déploiement : une seule origine

La pile complète est décrite par un fichier Docker Compose de cinq services : `postgres`, `n8n`, `api`, `stt-local` et `web`. **Un seul service publie un port** : le proxy `web`, qui sert le tableau de bord, l'API sous `/api/` et les webhooks sous `/webhook/`. C'est cette origine unique qui permet au cookie de session de rester en `SameSite=Lax` et qui rend les problématiques d'origines croisées sans objet — et la forme est identique en local, derrière le tunnel et sur le serveur de l'entreprise ; seul le nom d'hôte change.

Deux pièges de déploiement, rencontrés le 30 juillet 2026, méritent d'être documentés car ils font qu'**un changement peut sembler appliqué sans être exécuté** :

1. Le service est une **image construite**, code source inclus. Un simple redémarrage réexécute l'*ancien* code ; il faut reconstruire.
2. n8n conserve **sa propre copie** du workflow dans son volume. Modifier le fichier JSON versionné ne change rien tant qu'il n'est pas réimporté ou poussé par l'API de n8n.

La **Figure 4.1** donne la topologie de déploiement retenue.

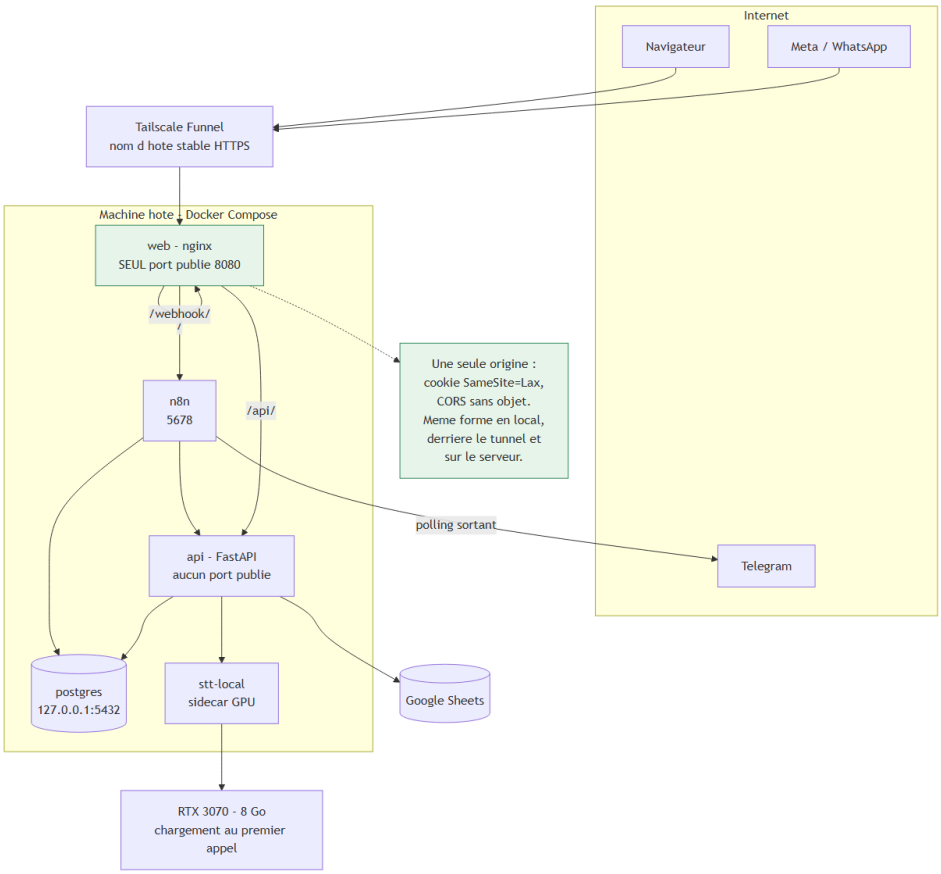


Figure 4.1 : *Topologie de déploiement. Un seul service publie un port ; tout le reste n'existe que sur le réseau interne de la pile.*

4.2.2 La base de connaissances du projet

Un projet de six mois produit deux sortes de savoir. Le premier tient dans le code et se relit : une fonction dit ce qu'elle fait. Le second ne tient nulle part — pourquoi cette chaîne de moteurs et pas une autre, ce qui a été essayé puis abandonné, ce qui casse si l'on touche à tel réglage — et c'est celui qui disparaît en premier. Le rapport en garde la partie défendable ; le reste vivait dans la mémoire du développeur, ce qui n'est pas un endroit.

Le dépôt porte donc un **coffre Obsidian** (`vault/`) : **dix-huit notes**, environ **10 200 mots**, reliées entre elles par **79 liens internes**. Ce ne sont pas des chapitres en attente. Chaque note répond à une question opérationnelle — `architecture`, `extraction-pipeline`, `stt-benchmark`, `n8n-`

`workflows`, `monitoring-observability`, `troubleshooting-runbook`, `security-secrets`, `testing-strategy` — et porte un en-tête structuré (`tags`, `type`, `priority`, `status`) qui permet de retrouver, par exemple, tout ce qui est marqué `status: current` et `priority: high`.

Document	Public	Question à laquelle il répond
`CLAUDE.md`	l'assistant de développement	Quelles décisions ne doivent **pas** être re-prises ?
`vault/`	le développeur suivant	Comment cette partie fonctionne-t-elle, et pourquoi ainsi ?
Ce rapport	le jury	Qu'a-t-on construit, et qu'est-ce qui le justifie ?

4.3 Les bancs d'essai comparatifs

C'est le volet scientifique du projet, et il porte sur **trois maillons distincts**, mesurés séparément parce qu'ils échouent différemment :

1. **La transcription** (§4.3.1 à §4.3.8) — **treize moteurs**, trois par API et douze exécutés localement sur une carte NVIDIA RTX 3070 (8 Go). Ce banc d'essai a **renversé l'hypothèse initiale** du cahier des charges.
2. **L'extraction texte → tableau** (§4.3.9) — huit modèles de langage, deux campagnes, sur un jeu de cas à vérité terrain connue passé de dix à vingt-cinq cas à mesure que le schéma s'est enrichi.
3. **L'extraction image → tableau** (§4.3.10) — quatre modèles de vision sur les dix premiers cas rendus en images, puis six modèles sur trois photos de plans réellement annotés.

Les trois protocoles diffèrent sur un point essentiel : pour la transcription, la vérité terrain a été **délibérément écartée** (§4.3.2) ; pour les deux autres, elle est écrite champ par champ dans le jeu de test. Cette asymétrie n'est pas une inconstance de méthode, c'est la conséquence directe de ce que chaque maillon produit — du darija non normalisé d'un côté, un objet typé de l'autre.

4.3.1 Protocole

Onze notes vocales réelles, issues des échanges des techniciens, ont servi de corpus. Trois campagnes ont été menées :

1. **Comparaison par API** (3 moteurs), le 7 août 2026.
2. **Première campagne locale** (9 moteurs), le même jour.
3. **Campagne locale complète** (10 moteurs), après déblocage de deux dépôts sous licence.

4.3.2 Une décision de méthode : ni WER, ni CER

La vérité terrain a été délibérément écartée, et il faut le dire explicitement plutôt que de le dissimuler : le fichier de références est resté vide, et le calcul du WER et du CER n'a donc **pas** été effectué. Ce n'est pas un oubli mais un arbitrage, fondé sur trois constats :

- **L'écriture du darija n'est pas normalisée.** Le WER compare des chaînes de mots ; sur une langue sans orthographe fixée, il mesure autant la convention d'écriture du transcripteur de référence que la qualité du moteur.
- **Le système ne consomme pas des mots, il consomme des nombres et des lieux.** Un moteur qui écrit ربعين là où la référence dit « 40 » est lourdement pénalisé par le WER, alors qu'il fournit exactement l'information dont la chaîne d'extraction a besoin — celle-ci lit le darija.
- **Le critère retenu est donc applicatif** : *quel moteur préserve les chiffres et les emprunts techniques français* — « 6 mètres », « 3ème étage », « panneau de brassage », « la fibre », « vérifier ». C'est tout ce que le journal consomme.

Ce critère a été appliqué par lecture des transcriptions clip par clip, puis **confirmé à l'oreille par un locuteur natif du darija le 8 août 2026**. Cette confirmation humaine est ce qui a fixé l'ordre définitif de la chaîne.

La **Figure 4.2** retrace la démarche du banc d'essai, détours compris.

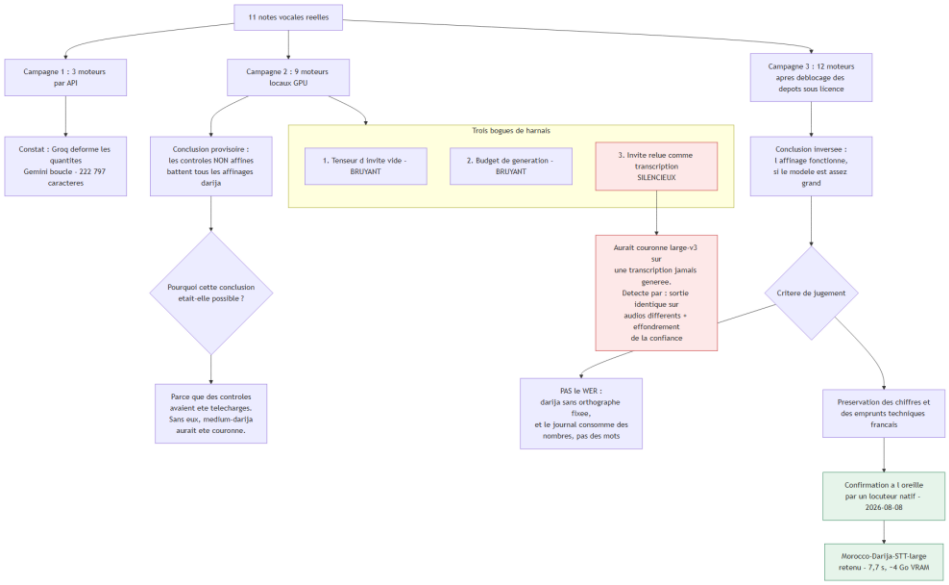


Figure 4.2 : *Démarche du banc d'essai, y compris ses détours.*

4.3.3 Résultats — moteurs par API

Le **Tableau 4.2** rassemble les résultats des trois moteurs accessibles par API.

Tableau 4.2 : Comparaison des moteurs accessibles par API .

Moteur	Clips	Erreurs	Latence moyenne	Confiance moyenne	Caractères moyens
Groq (Whisper large-v3)	11	0	**1,53 s**	0,483	613

Moteur	Clips	Erreurs	Latence moyenne	Confiance moyenne	Caractères moyens
Gemini	11	0	16,94 s	non exposée	20 651 (faussé, voir §4.4.1)

Constat majeur : Groq déforme les quantités. Sur un même clip, Gemini produit `40` كابل, `8` ديال `les panneaux`, `12` پانُو, `3ème étage` ; Groq produit `mniya dyal panno`, `Pouzina ssimetro`, `30 mietage`, `ondélique`. Le schéma métier étant intégralement composé de quantités et de localisations, un moteur qui fusionne un chiffre dans le mot suivant ne donne rien à extraire — et `has_data()` renvoie alors « faux » sur un rapport qui *contenait* pourtant des données.

4.3.4 Résultats — moteurs locaux sur GPU

Dix moteurs ont été exécutés localement en demi-précision sur la RTX 3070.

Le **Tableau 4.3** en donne le relevé complet.

Tableau 4.3 : Campagne locale complète, 10 moteurs, 11 clips, RTX 3070 .Trié par confiance décroissante.

Modèle	Latence	Confiance	Caractères	Chiffres
`boumehdi/wav2vec2-moroccan-darija`	0,7 s	0,916	501	0
`amineouaki/xlsr-53-darija-combined`	6,3 s	0,884	493	7
`amineouaki/wav2vec2-darija-combined`	4,0 s	0,853	439	22
`ychafiqui/whisper-medium-darija`	3,7 s	0,843	184	0
`BounharAbdelaziz/Morocco-Darija-STT-large`	**7,7 s**	**0,791**	**249**	**45**
`TaloCreations/whisper-darija-finetuned`	2,3 s	0,735	198	0

Modèle	Latence	Confiance	Caractères	Chiffres
`ychafiqui/whisper-small-darija`	2,3 s	0,716	199	0
`openai/whisper-large-v3-turbo` (contrôle)	1,0 s	0,597	174	30
`openai/whisper-large-v3` (contrôle)	5,7 s	0,582	214	55
`C-Ilyas/whisper-base-darija`	5,0 s	0,548	**0**	0

Ce tableau est trié par confiance précisément pour démontrer qu'il ne faut pas s'y fier. `boumehdi` domine la colonne à 0,916 et perd purement et simplement le « 40 » ; le meilleur moteur n'arrive qu'en cinquième position. C'est la troisième confirmation indépendante que `exp(mean logprob)` mesure la certitude du décodeur, non son exactitude.

Deux précautions de lecture s'imposent :

- **Les confiances ne sont pas comparables d'une famille à l'autre.** Le 0,85–0,92 de wav2vec2 est une moyenne de *softmax* CTC ; celle de Whisper est `exp(mean token logprob)`. Même échelle, grandeurs différentes.
- **La colonne « chiffres » est un leurre.** Les affinages capturent bien les nombres, mais sous forme de **mots arabes** : تمنية (8), طناشر (12), ربعين (40). La chaîne d'extraction lit le darija ; ces formes sont donc parfaitement récupérables. La colonne mesure une commodité de format, non une perte d'information.

4.3.5 Le moteur de référence et sa place dans la chaîne

`BounharAbdelaziz/Morocco-Darija-STT-large` (environ 2 milliards de paramètres, licence Apache 2.0) est le **vainqueur du banc d'essai** : c'est lui qui fixe la référence d'exactitude à laquelle les autres moteurs sont comparés. Sur un clip de référence dont le contenu est *« hier 20/07/2026, arrivés à Essaouira à 1 h, vérifié le chemin de 27 câbles, pris le métrage, coupés, vérifié le chemin de la fibre, sortis à 7 h »*, le **Tableau 4.4** compare les transcriptions obtenues :

Tableau 4.4 : Comportement comparé sur les champs métier d'un clip de référence.

Modèle	Effet sur les champs de données
Morocco-Darija-STT-large	`20` سبعة `2026, 27` كابل, فيريفينا (vérifier), لافيبر (la fibre), السبع (7 h) — **tous les champs extractibles survivent**
`whisper-large-v3`	Date et `27` كابل corrects, mais *vérifier* devient **فرحنا** (« nous nous sommes réjouis ») — un

Modèle	Effet sur les champs de données
	verbe technique français transformé en mot arabe sans rapport
`ychafiqui/medium-darija`	**`27 câbles devient كابل الين وعشر وعشر السبعة`** — la quantité est détruite ; la ligne atteint le journal sans compte de câbles

Le moteur retenu conserve également les noms propres du domaine — Novec, Essaouira, Kenitra — à travers l'ensemble des clips.

Le coût de cette exactitude est mesuré : 7,7 s de latence moyenne, le plus lent des douze, et environ 4 Go de mémoire graphique, contre 1,53 s pour l'API de Groq. C'est précisément ce coût qui sépare le **vainqueur du banc d'essai** du **moteur appelé en premier en production**, et il faut le dire sans détour, car la configuration livrée est publique et un lecteur la comparera à ce chapitre.

La chaîne effectivement déployée est `gemini, local, groq` (`api/app/config.py`). Gemini répond en premier ; `Morocco-Darija-STT-large` n'est sollicité que si Gemini échoue ; Groq ferme la chaîne comme plancher toujours disponible. Trois raisons à cet ordre, aucune ne remettant en cause le résultat du banc d'essai :

- **La latence est payée par le technicien, pas par le banc d'essai.** 7,7 s de transcription s'ajoutent à l'extraction et à l'écriture dans le journal ; sur chaque note vocale de chaque technicien, l'écart avec un appel d'API se voit.
- **Les 4 Go de mémoire graphique sont immobilisés en permanence.** Le chargement des poids coûte de 280 à 394 s, payés une fois par conteneur ; garder le modèle résident pour un moteur appelé en secours est un compromis assumé (`STT_LOCAL_PRELOAD=1`), mais en faire le moteur principal reviendrait à faire dépendre l'ensemble du service d'une seule carte graphique.
- **Le repli local est ce qui rend la chaîne robuste.** Le quota gratuit de Gemini s'épuise ; c'est un événement attendu, non un incident. Le jour où il s'épuise, le moteur le plus exact du banc d'essai prend le relais sans intervention et sans dépendance à un fournisseur.

Autrement dit, le banc d'essai n'a pas désigné un moteur unique : il a désigné une **référence d'exactitude**, et c'est cette référence qui occupe la position de secours — la seule position où sa lenteur ne se paie qu'en cas de défaillance, et où sa qualité est justement la plus nécessaire. La validation par un locuteur natif tranche l'exactitude ; l'ordre d'appel, lui, est une décision d'exploitation.

4.3.6 Une hypothèse renversée, puis rétablie

L'histoire de cette conclusion mérite d'être racontée, car elle illustre la valeur des contrôles.

À l'issue de la première campagne locale, la conclusion était **négative** : les deux contrôles OpenAI non affinés égalaient ou surpassaient *tous* les affinages darija disponibles. Ce résultat contre-intuitif — l'affinage sur la langue cible ne servirait à rien — n'a pu être formulé que parce que les

contrôles avaient été téléchargés ; sans eux, `ychafiqui/medium-darija` aurait été couronné vainqueur.

Or les deux modèles les plus prometteurs étaient **sous licence restreinte** et n'avaient pas pu être téléchargés lors de cette première campagne. Une fois l'accès obtenu et la campagne rejouée, la conclusion s'est inversée : **l'affinage sur le darija fonctionne bel et bien, à condition que le modèle affiné soit assez grand.** L'énoncé négatif reste vrai des cinq *petits* affinages ; la généralisation qu'on en avait tirée était fausse.

Une ambiguïté a été close au passage : l'adaptateur historique pointait vers un identifiant de dépôt inexistant. Il retournait une erreur 401 tant que le jeton d'authentification manquait — donc « non authentifié » — et une erreur 404 une fois le jeton fourni — donc « n'existe pas ». **Deux défauts empilés, le moins grave masquant le fatal** : cet adaptateur n'avait jamais pu fonctionner une seule fois depuis sa création.

4.3.7 Trois bogues de harnais, dont un silencieux

La mise au point du banc d'essai local a révélé trois défauts, dont l'ordre de gravité est instructif :

#	Défaut	Mode de défaillance
1	Tenseur d'invite vide, indexé sans garde	Exception à chaque clip — **bruyant**
2	Budget de génération incompatible avec la fenêtre du décodeur	Exception à chaque clip — **bruyant**
3	Les jetons d'invite relus **comme s'ils étaient la transcription**	**Silencieux**

Le troisième est le seul qui compte vraiment. Il produisait `Kmmalna khamsa dyal bureaux` — exactement ce qu'un technicien dirait — et aurait couronné `whisper-large-v3` vainqueur sur une transcription **qu'il n'avait jamais générée**. Deux signaux l'ont trahi : une sortie *identique* sur des audios différents, et un effondrement de la confiance à 0,085–0,34 contre 0,48 pour Groq. La confiance ne mesure pas l'exactitude, mais son effondrement soudain signale que le harnais ment.

4.3.8 L'invite de vocabulaire : survivable par API, fatale en local

Une même instruction produit deux comportements opposés selon la voie d'exécution.

Le **Tableau 4.5** chiffre cet écart.

Tableau 4.5 : Effet de l'invite de vocabulaire en exécution locale.

Condition	Sortie observée (`whisper-large-v3` local)
Invite activée	`The work, the work, the work…` — boucle dégénérée, sur **tous** les clips

Condition	Sortie observée (`whisper-large-v3` local)
Invite désactivée	نوفيك في اليوم لخدمة بالنسبة عليكم السلام… — darija propre

Les deux explications concurrentes ont été **éliminées avant** de conclure, et non écartées par hypothèse : le décodage audio a été validé par corrélation de 1,0000 entre deux décodeurs indépendants, et la précision de calcul a produit un texte identique octet pour octet.

Cause réelle. L'API de Groq et la bibliothèque `transformers` ne traitent pas l'invite de la même manière : la première conditionne segment par segment avec sa propre troncature ; la seconde préfixe les jetons bruts au décodeur et le laisse les *continuer*. Recevant 180 jetons de darija en caractères latins présentés comme « transcription précédente », le modèle poursuit en caractères latins et s'effondre.

Par contraste, une comparaison A/B menée sur l'API a montré que la même invite y est non seulement inoffensive mais **bénéfique** : elle contrôle décisivement le système d'écriture (6 caractères arabes avec l'invite, 3 702 sans) sans coûter de chiffres (93 contre 89), et sa suppression fait perdre un tiers du contenu transcrit. **L'invite est donc conservée pour la voie API et désactivée pour la voie locale** — deux réglages opposés pour un même fichier, parce que deux implémentations en font deux choses différentes.

4.3.9 Second banc d'essai : du texte au tableau

La transcription n'est qu'un maillon. Le maillon suivant — **transformer un texte libre en une ligne de journal typée** — a fait l'objet d'un banc d'essai distinct, mené le 27 juillet 2026 et rejoué le 2 août, avec un protocole différent de celui du §4.3.1 parce que la question posée est différente : ici, une vérité terrain **existe** et est parfaitement définie, puisque chaque cas de test est écrit avec les valeurs attendues champ par champ.

Protocole. Le jeu de cas est rédigé à la main (`benchmark/dataset/text_cases.json`) : chaque cas porte un texte et les valeurs attendues **champ par champ**, y compris les champs qui doivent rester nuls. Il a suivi le schéma : dix cas au départ, vingt lorsque les vingt-cinq champs d'article ont été ajoutés, **vingt-cinq aujourd'hui** depuis que le chantier énoncé est devenu un champ du modèle. Chaque cas cible un piège précis, et non une performance moyenne. Le **Tableau 4.6** récapitule les dix premiers, qui portent les modes de défaillance de la transcription elle-même.

Tableau 4.6 : Les dix cas de test de l'extraction, et le défaut que chacun cherche à provoquer.

Cas	Ce qu'il éprouve
`t01_plain_french`	Français propre, toutes les quantités présentes. Cas plancher : un modèle qui échoue ici est inutilisable
`t02_darija_numbers_words`	Quantités énoncées en **mots arabes** et non en chiffres — la normalisation, pas la simple lecture de chiffres

Cas	Ce qu'il éprouve
`t03_self_correction`	Le locuteur se corrige en cours de phrase ; c'est la **dernière** valeur qui compte
`t04_hallucination_trap`	Une seule quantité énoncée ; tous les autres champs doivent revenir `null`
`t05_observations`	Un blocage est signalé : `remarques` doit être rempli, les quantités doivent rester nulles
`t06_mixed_code_switch`	Alternance darija/français à l'intérieur d'une même phrase, plus un décimal
`t07_arabic_script`	Écrit en caractères arabes plutôt qu'en translittération latine — les deux arrivent réellement
`t08_no_quantities_at_all`	Message d'état sans aucun nombre : tout le numérique doit être nul et `resume` rester utile
`t09_units_confusion`	Des nombres qui ne sont **pas** des quantités (étage, numéro de local, taille d'équipe)
`t10_noisy_transcription`	Simule une mauvaise hypothèse STT : mots métier déformés, à réparer par le contexte

Les quinze cas ajoutés depuis portent sur le vocabulaire du bordereau et sur le chantier. Le **Tableau 4.7** les récapitule.

Tableau 4.7 : Les quinze cas ajoutés avec les champs d'article et le chantier énoncé.

Cas	Ce qu'il éprouve
`t11_articles_cuivre_clean`	Deux variantes de cordon nommées avec leur longueur, plus un guide-câble : trois champs distincts, pas un
`t12_delivered_not_posed`	Le piège reçu / posé. Une réception ne remplit **aucun** champ d'article — ni la quantité, ni zéro
`t13_variant_confusion`	« des jarretières » sans type : les trois champs de jarretière doivent rester nuls
`t14_cross_vocabulary_cable`	Les trois chantiers appellent la même chose « câble UTP », « câble CAT.6A » et « câble informatique »
`t15_electricite`	Articles électriques, mètres et unités dans la même phrase
`t16_fusion_vs_pigtail`	24 fusions sont des brins optiques, **pas** des pigtails

Cas	Ce qu'il éprouve
`t17_article_explicit_zero`	Négation explicite sur un champ d'article : « aucun point d'accès posé » est une valeur, pas une absence
`t18_actif_modules`	Trois variantes de module optique qu'un modèle faible réduit à un seul champ
`t19_articles_hallucination_trap`	Une journée de cuivre pur, sans aucun article de bordereau nommé : tous doivent rester nuls
`t20_arret_et_stock`	Une journée arrêtée, avec durée énoncée et stock restant déclaré
`t21_chantier_annonce`	Le technicien nomme le site franchement : extraction mot pour mot, sans rapprochement
`t22_chantier_jamais_nomme`	Aucun site énoncé — le cas le plus fréquent. `chantier_mentionne` **doit** rester nul
`t23_etage_nest_pas_un_chantier`	Un étage, une aile ou un local technique ne sont pas un chantier
`t24_chantier_avec_article`	« la DGH » : le modèle rend ce qu'il entend, la normalisation vient après
`t25_prises_electriques_pas_rj45`	Le mot « prise » ne suffit pas à remplir un champ de prise réseau (rapport réel du 19 août 2026)

Deux indicateurs sont relevés par cas : l'**exactitude** (part des champs correctement remplis) et surtout le **nombre de valeurs inventées** — un champ rempli là où la vérité terrain attend `null`. Ces deux indicateurs ne sont pas symétriques : une valeur manquante se corrige au tableau de bord, une valeur inventée entre dans le journal du directeur sans que rien ne la signale.

Le **Tableau 4.8** donne le résultat des huit modèles évalués.

Tableau 4.8 : Extraction texte → rapport structuré, 8 modèles, 10 cas (27 juillet 2026).

Modèle	Fournisseur	Cas répondus	Exactitude moyenne	Cas parfaits	Valeurs inventées	Latence moyenne
`openai/gpt-oss-120b`	Groq	**10 / 10**	**1,000**	**10 / 10**	**0**	1,72 s
`gemini-flash-lite-latest`	Google	10 / 10	0,980	9 / 10	1	**1,04 s**

Modèle	Fournisseur	Cas répondus	Exactitude moyenne	Cas parfaits	Valeurs inventées	Latence moyenne
`gemini-3.5-flash-lite`	Google	10 / 10	0,980	9 / 10	1	1,00 s
`gemini-3.1-flash-lite`	Google	10 / 10	0,980	9 / 10	1	2,71 s
`gemini-3.6-flash`	Google	**4 / 10**	1,000	4 / 4	0	4,98 s
`llama-3.3-70b-versatile`	Groq	10 / 10	0,922	7 / 10	1	0,69 s
`llama-3.1-8b-instant`	Groq	7 / 10	0,876	5 / 7	**5**	0,50 s
`qwen/qwen3.6-27b`	Groq	**0 / 10**	—	—	—	—

Quatre lectures s'imposent, et aucune ne se lit dans la colonne « exactitude » seule.

- **La disponibilité prime sur l'exactitude.** `gemini-3.6-flash` obtient 1,000 — sur les quatre cas qu'il a pu traiter avant que l'offre gratuite ne renvoie `429 RESOURCE_EXHAUSTED` sur les six suivants. Un modèle qui répond parfaitement puis s'arrête est un **repli**, pas un primaire. `qwen3.6-27b` échoue plus franchement encore : 10 refus sur 10, le fournisseur rejetant la contrainte de sortie JSON (`Failed to validate JSON`).
- **La rapidité ne rachète rien.** `llama-3.1-8b-instant` est le plus rapide du tableau à 0,50 s, et le pire de tous sur le seul critère qui compte : **cinq valeurs inventées**, dont un `client` qu'aucun cas ne mentionne. Il échoue en outre trois fois à la validation Pydantic, en renvoyant du texte là où le schéma attend un nombre.
- **Le cas décisif est `t08`.** Le message ne contient aucun nombre ; tout doit revenir nul. C'est là que les modèles faibles remplissent quatre champs de quantité à partir de rien — le mode de défaillance exact que la politique « aucune donnée inventée » du §3.4.2 cherche à interdire.
- **Le seul défaut des modèles Gemini est identique et unique** : `t02`, où la quantité est énoncée en mots arabes, `cables_m` étant alors mal normalisé. Une faiblesse ciblée, reproductible, et sans invention.

Seconde campagne, 15 août 2026. Le jeu de cas ayant été réécrit ce jour-là pour le schéma enrichi, le classement de juillet portait sur un contrat qui n'existait plus : il a fallu remesurer. Le **Tableau 4.9** donne le résultat sur les vingt cas d'alors.

Tableau 4.9 : Extraction texte → rapport structuré, 8 modèles, 20 cas (15 août 2026).

Modèle	Cas répondus	Exactitude moyenne	Cas parfaits	Valeurs inventées	Latence moyenne
`gemini-3.5-flash-lite`	**20 / 20**	**0,990**	19 / 20	**0**	2,20 s
`gemini-flash-lite-latest`	**20 / 20**	**0,990**	19 / 20	**0**	**2,19 s**
`gemini-3.1-flash-lite`	20 / 20	0,955	17 / 20	0	2,04 s
`openai/gpt-oss-120b`	17 / 20	0,943	13 / 20	**2**	24,74 s
`gemini-3.6-flash`	16 / 20	0,988	15 / 20	0	8,39 s
`llama-3.3-70b-versatile`	7 / 20	1,000	7 / 7	0	15,10 s
`llama-3.1-8b-instant`	7 / 20	0,845	3 / 7	**4**	24,29 s
`qwen/qwen3.6-27b`	**0 / 20**	—	—	—	—

Ce tableau porte une réserve, et elle est conservée exprès. La campagne a été menée **avant** le correctif des chaînes « null » décrit au §4.4.4 : un pré-traitement enveloppait la chaîne `"null"` dans une liste avant que le validateur générique ne puisse la voir. Les deux modèles Llama y échouent donc sur treize cas sur vingt et `qwen` sur vingt sur vingt — non parce qu'ils sont mauvais, mais parce que **notre schéma refusait leur sortie**. Le fichier de résultats a été gardé sous ce nom pour cette raison : sans lui, « Groq est mauvais » et « nous rejetions la réponse de Groq » seraient indistinguables. C'est aussi la raison pour laquelle une reprise à vingt-cinq modèles, tentée le même soir, a été **abandonnée au deuxième modèle** : treize appels sur vingt-trois revenaient en 503 « high demand » et les suivants en 429, le banc d'essai entrant en concurrence avec le pipeline de production pour le même quota gratuit. Un classement bâti là-dessus aurait mesuré la charge d'un fournisseur un vendredi soir, pas la qualité des modèles.

La chaîne déployée est `groq:openai/gpt-oss-120b`, puis `gemini:gemini-3.5-flash-lite`, puis `gemini:gemini-flash-lite-latest` — et l'ordre ne se déduit d'aucun des deux tableaux. Il vient d'une mesure du 19 août 2026 sur un rapport réel énonçant « 4 prises doubles étanches, 13 prises simples, 9 interrupteurs, 2 tableaux électriques » : `gpt-oss-120b` a rendu les six articles

avec leurs comptes, et **les deux modèles Gemini ont rendu un objet vide** sur exactement la même invite. Le banc d'essai qui plaçait Gemini en tête n'est pas faux — il mesurait le schéma tel qu'il était, avant l'existence du champ `articles_libres`.

Une limite de méthode doit être énoncée ici, comme elle l'a été au §4.3.2 pour la transcription. Les dix cas de test ont été écrits par l'auteur du système évalué. Ils encodent donc les mêmes hypothèses que les invites qu'ils servent à juger : ce qui compte comme une quantité, ce qui doit rester nul, la forme qu'un message de technicien est censé prendre. Un modèle peut ainsi réussir parce qu'il partage la vision du concepteur plutôt que parce qu'il lit correctement un message réel. Trois éléments limitent la portée de ce biais sans le supprimer — les cas sont dérivés de messages réellement reçus pendant la phase de développement du cœur IA, la vérité terrain est vérifiable champ par champ par un tiers, et le critère décisif retenu (`t08` : n'inventer aucune valeur en l'absence de donnée) est **négatif**, donc insensible aux préférences de rédaction du concepteur. La levée complète du biais suppose un jeu de cas rédigé par un tiers à partir de messages non vus lors de la conception ; elle figure parmi les perspectives de la conclusion générale.

4.3.10 Troisième banc d'essai : de l'image au tableau

Le troisième chemin d'entrée — la **photo d'un relevé manuscrit** — a été mesuré avec les mêmes dix cas, rendus sous forme d'images, le 27 juillet 2026. Le modèle de vision lit la photo et remplit le schéma en un seul appel : il n'y a **pas de transcription intermédiaire**, donc pas de maillon où une erreur pourrait être rattrapée.

Le champ des candidats est ici structurellement plus étroit : le catalogue gratuit de Groq ne comporte **aucun modèle de vision**, ce qui réduit la chaîne image à un seul fournisseur. Le **Tableau 4.10** rassemble les résultats.

Tableau 4.10 : Extraction image → rapport structuré, 4 modèles, 10 cas (27 juillet 2026).

Modèle	Cas répondus	Exactitude moyenne	Cas parfaits	Valeurs inventées	Latence moyenne
`gemini-3.5-flash-lite`	10 / 10	**0,983**	9 / 10	**0**	1,56 s
`gemini-flash-lite-latest`	10 / 10	**0,983**	9 / 10	**0**	**1,25 s**
`gemini-3.1-flash-lite`	10 / 10	0,963	8 / 10	1	4,31 s
`gemini-3.6-flash`	**0 / 10**	—	—	—	—

Trois observations méritent d'être retenues.

- **L'écart texte / image est faible, et c'est le résultat inattendu.** On pouvait craindre que la lecture d'une écriture manuscrite dégrade franchement l'extraction ; l'exactitude passe

de 0,980 à 0,983, soit strictement rien. Le champ fautif reste le même — `cables_m` — et le défaut vient donc de la normalisation des quantités, non de la reconnaissance de l'écriture.

- **`gemini-3.6-flash` est absent du tableau pour la même raison qu'au §4.3.9** : quota gratuit épuisé, 10 échecs `429` sur 10. Il reste malgré tout **en tête de la chaîne image**, contrairement à la chaîne texte : n'ayant pas d'équivalent chez un second fournisseur, mieux vaut tenter le modèle le plus capable et se replier immédiatement que de renoncer d'emblée à sa qualité.
- **Le chemin image est traité comme intrinsèquement moins sûr**, indépendamment de ces chiffres : toute ligne issue d'une photo est écrite avec `flag_review = true` et son résumé est préfixé `[photo]`, afin qu'elle ne puisse jamais être confondue avec une parole citée du technicien (§3.5.1).

Une seconde campagne sur des données réelles, le 15 août 2026. Les dix cas rendus en images restent des images *fabriquées*. Trois photos de plans réellement annotés — cercles au feutre, points bleus, tracés de cheminement — ont donc été constituées en jeu de test, avec pour vérité terrain les zones marquées. Le critère a été fixé **avant** la campagne, comme pour la transcription : le rappel (le modèle nomme-t-il les zones marquées) et l'hallucination (revendique-t-il des zones que personne n'a marquées). Le **Tableau 4.11** en donne le résultat.

Tableau 4.11 : Extraction depuis des plans annotés, 6 modèles, 3 photos réelles (15 août 2026).

Modèle	Cas répondus	Rappel moyen	Hallucinations	Latence moyenne
`gemini-3.5-flash-lite`	3 / 3	0,92	**0**	**1,28 s**
`gemini-flash-lite-latest`	3 / 3	0,92	**0**	1,42 s
`gemini-3.6-flash`	3 / 3	0,92	1	13,37 s
`gemini-3.1-flash-lite`	3 / 3	**1,00**	**4**	1,53 s
`llama-4-scout-17b` (Groq)	**0 / 3**	—	—	404, non servi
`llama-4-maverick-17b` (Groq)	**0 / 3**	—	—	404, non servi

Trois choses s'y lisent. D'abord, **`gemini-3.1-flash-lite` est la démonstration que le rappel seul ne doit pas choisir un modèle** : il est le seul à 100 %, et il y parvient en nommant des pièces que personne n'a marquées — c'est exactement ainsi qu'un étage inventé atteint le champ `etages`. Il est donc **exclu** de la chaîne image, et non simplement classé dernier : sur le chemin image, il n'y a aucune transcription derrière la réponse pour la contredire. Ensuite, les deux modèles

multimodaux de Groq répondent 404 : le catalogue gratuit a été énuméré le même jour — quinze modèles, aucun de vision — ce qui confirme que la chaîne image ne peut pas alterner les fournisseurs. Enfin, une réserve doit accompagner ces chiffres : le corrigé est un brouillon d'auteur, et trois modèles sur quatre manquent la même zone sur la troisième photo ; tant que le propriétaire du plan ne l'a pas confirmé, ce classement est indicatif.

4.4 Difficultés techniques rencontrées et résolues

Au-delà du banc d'essai, la confrontation au réel a révélé plusieurs défauts indétectables hors ligne.

4.4.1 La boucle de répétition de Gemini

Symptôme mesuré : un clip de 84 Ko a produit **222 797 caractères en 129,2 secondes**, la fin répétant sans terme une même locution. Groq renvoyait 381 caractères en 1,0 seconde sur le même audio. Gemini traduisait de surcroît en français malgré une consigne explicite de ne pas traduire.

Résolution, en trois volets : un plafond de génération fixé à environ quatre fois le pire cas légitime observé ; une température nulle, la transcription n'étant pas une tâche créative et l'échantillonnage étant ce qui autorise le décodeur à quitter l'audio ; et surtout un **détecteur de dégénérescence qui lève une erreur** au lieu de tronquer. Ce dernier point est un choix de conception : une boucle tronquée reste plusieurs milliers de caractères de bruit que la chaîne d'extraction résumerait en une fiction confiante, et que `has_data()` ne rattraperait pas, un texte répété n'étant pas un texte vide.

4.4.2 Validation de format par Groq

L'API Whisper de Groq valide l'extension du nom de fichier **avant** de décoder les octets ; or les notes vocales arrivent en `audio/ogg`, parfois présentées comme un flux binaire générique. **Résolution :** normalisation du type MIME vers une table d'extensions acceptées, avec alias, et repli sur le format des notes vocales.

4.4.3 Rotation des modèles chez le fournisseur

Un modèle épinglé a été retiré par Google en cours de projet (404), et l'alias du modèle phare répondait 503 aux heures de pointe sur l'offre gratuite. **Résolution :** sondage systématique des modèles candidats, puis bascule vers un alias vérifié opérationnel. Cet épisode valide *a posteriori* le patron d'adaptateurs et la configuration externalisée — et impose une leçon durable : **figurer dans le catalogue d'un fournisseur ne signifie pas être appelable.**

4.4.4 Chaînes « null » des modèles

En mode JSON, certains modèles émettent la *chaîne* `"null"` au lieu de la valeur nulle, provoquant un rejet de validation sur les champs numériques. **Résolution :** un validateur de pré-traitement normalise `null`, `none` et la chaîne vide vers la valeur nulle avant le typage strict.

4.4.5 Une identité scindée en deux

Sur données réelles, un même humain est apparu **deux fois** dans l'organigramme : sa ligne WhatsApp le déclarait `directeur`, sa ligne Telegram `tech`. La cause était une correspondance

établie sur le couple (nom, rôle). **Résolution :** la correspondance se fait désormais **sur le nom seul**. C'est cette correction qui permet à un profil de suivre le collègue plutôt que le numéro de téléphone.

4.4.6 Une migration qui se planifie avant de s'exécuter

La migration de fusion des comptes lit une colonne qu'elle supprime elle-même. PostgreSQL planifiant l'intégralité d'une instruction avant de l'exécuter, elle échouait. **Résolution :** encapsulation dans un bloc conditionnel dynamique, la rendant réexécutable à chaque démarrage.

4.5 Stratégie de test et intégration continue

La suite compte **860 tests** répartis en 49 fichiers (10 916 lignes). **810 d'entre eux s'exécutent sans aucune clé d'API et sans base de données** ; c'est une règle du projet, obtenue par deux moyens : les bibliothèques externes sont importées paresseusement *à l'intérieur* des fonctions, et les appels externes sont simulés aux frontières.

Les **50 restants** portent la marque `integration` et sont **ignorés** — jamais échoués — tant que la variable `TEST_DATABASE_URL` n'est pas définie. Ils éprouvent ce qu'aucune simulation ne peut éprouver : le comportement réel de PostgreSQL sur les migrations, les contraintes d'unicité et les transactions. La distinction est faite par un marqueur plutôt que par un dossier séparé, pour une raison précise : un test d'intégration rangé à part est un test que l'on oublie d'exécuter, alors qu'un test marqué apparaît dans chaque exécution, en clair, avec la mention *skipped* et le motif — l'absence de base est ainsi **visible** au lieu d'être silencieuse.

Elle a un corollaire honnête, qu'il faut énoncer : **aucun test ne parcourt le chemin réseau réel**. Les défauts décrits en §4.4 sont précisément ceux qu'une suite hors ligne ne peut pas voir, et c'est pourquoi la vérification en conditions réelles reste une étape distincte et nécessaire.

L'intégration continue (GitHub Actions) exécute la suite avec mesure de couverture, l'analyse statique interne et externe, et un contrôle de parité des trois fichiers de traduction — dix-huit contrôles au total, détaillés au §4.9.3. La suite complète tourne en **8 min 59 s** — 810 succès, 50 ignorés, aucun échec — sur la machine de développement. La durée est dominée non par le calcul mais par les délais d'attente simulés des chaînes de repli : éprouver qu'un moteur indisponible cède la main au suivant suppose d'atteindre le délai que le code lui accorde.

4.6 Résultats

4.6.1 Chaîne de bout en bout opérationnelle

Dès les premiers jours de la phase 3 — alors que le planning la prévoyait en phase 4 — la chaîne complète a été validée en conditions réelles : note vocale → détection → téléchargement → transcription → extraction → résumé français → réponse de confirmation. Le risque d'intégration majeur du projet a ainsi été neutralisé très tôt, ce qui est précisément l'objet d'une neutralisation précoce des risques.

Lors de ces essais, **le repli d'extraction s'est déclenché en conditions réelles** (indisponibilité du fournisseur principal en soirée) : la chaîne a produit son rapport via le moteur secondaire sans intervention, validant la conception.

4.6.2 Périmètre livré

Le **Tableau 4.12** dresse l'état du périmètre livré, composant par composant.

Tableau 4.12 : État du périmètre livré.

Composant	État
Réception Telegram	En production
Réception WhatsApp Cloud API	En production
Transcription (3 moteurs, chaîne de repli)	En production
Extraction structurée (3 chaînes de repli : texte, image, profil)	En production
Entrée par photo (modèle de vision)	En production
Journal de chantier par site (une ligne par message, stock réconcilié)	En production
Miroir plat pour le tableau de bord et table d'agrégation	En production
Observabilité PostgreSQL (26 tables)	En production
File de reprise des écritures Sheets	En production
File de reprise des messages, ordonnée par technicien	En production
Archive des médias et des échanges, purgée par rétention	En production
Tableau de bord trilingue (FR/EN/AR)	En production
Organigramme, comptes, chantiers, bordereau	En production
Ordonnanceur (9 tâches) et page Automatisation	En production
Rapports quotidien et hebdomadaire par e-mail (avec document .docx joint)	En production
Colonnes d'article créées à la volée (`articles_libres`, registre, confirmation)	En production
Registre des réceptions de matériel (`nature`, `livraisons`)	En production
Alertes de stock en jours de couverture, dépassement de bordereau et excédent	En production
Mise en sourdine d'une alerte, expirant d'elle-même à la première aggravation	En production
Veille matière à la minute : cloche du tableau de bord, une fois par état	En production

Composant	État
Matériel déclaré rattaché au site déclarant, validation depuis son volet	En production
Page Consommation : quotas des offres gratuites mesurés	En production
Cloche de notifications par compte	En production
Liste de tâches partagée, rappels récurrents	En production
Rapports quotidien et hebdomadaire éditables, versionnés	En production
Réponse au technicien depuis la boîte de réception, sur un canal choisi	En production
Correction de la transcription depuis la boîte de réception, avec trace d'audit	En production
Signature des webhooks WhatsApp vérifiée	En production
Commandes `/rapport`, `/modifier`, `/supprimer`	Retirées (voir §3.6.1)
Génération d'illustrations pour les rapports	Retirée (aucun fournisseur gratuit joignable, §4.3.10)
Moteur local sur GPU, conteneurisé	En production

4.6.3 Métriques du projet

Le **Tableau 4.13** en donne les métriques.

Tableau 4.13 : Métriques du projet à la date de rédaction.

Indicateur	Valeur
Durée du projet	6 mois (2 mars – 31 août 2026)
Lignes de Python (service)	15 380
Lignes de tests	10 916 (49 fichiers, 860 tests)
Lignes de TypeScript et CSS (tableau de bord)	16 139
Clés d'interface traduites	579 × 3 langues
Routes d'API	85
Nœuds n8n	64 (dont 54 actifs)
Pages du tableau de bord	12
Instructions de modèles versionnées	5
Champs du contrat `ExtractedReport`	54
Colonnes de quantité du journal	35
Moteurs STT évalués	13

Indicateur	Valeur
Cas de test d'extraction	25 (texte) + 3 (plans annotés)
Tâches planifiées	9
Tables PostgreSQL	26
Services Docker	5
Contrôles d'intégration continue	18

4.7 Le tableau de bord en fonctionnement

Le tableau de bord est l'interface par laquelle le directeur et les chefs de chantier consultent les rapports, les corrigent, administrent l'organigramme et surveillent l'ordonnanceur. Sa conception est traitée au §3.7 ; son rendu réel — vingt-neuf captures produites en une seule passe par un script Playwright versionné dans le dépôt (`Dashboard/scripts/shots.mjs`), en thème sombre et en anglais sauf mention contraire — est reporté en **Annexe C**, où il peut être parcouru écran par écran sans interrompre le fil de ce chapitre.

4.8 Analyse des coûts et des quotas

La contrainte de budget nul n'est tenable que si elle est chiffrée. Un rapport consomme, au plus, **un appel de transcription** et **un appel d'extraction**, plus les écritures dans le classeur.

Le **Tableau 4.14** rapporte cette consommation aux plafonds des offres gratuites.

Tableau 4.14 : Consommation des offres gratuites.

Service	Offre gratuite	Consommation par rapport	Marge à 10 rapports/jour
Groq (transcription et extraction)	Quotas par minute et par jour	1 appel	Très large
Google AI Studio (Gemini)	Quotas par minute et par jour	1 à 2 appels	Très large
Moteur local (GPU)	Aucun quota	Illimité	Illimité
Telegram Bot API	Gratuit	2 à 3 appels	Très faible
WhatsApp Cloud API	Conversations de service gratuites	2 appels	Large
Google Sheets API	Quota par minute et par utilisateur	3 à 5 appels	Large
Gmail SMTP	Envois quotidiens	1 à 2 par jour	Large
PostgreSQL, n8n, Docker	Auto-hébergés	—	Sans objet

Trois décisions d'architecture entretiennent directement cette marge. La **chaîne d'extraction alterne les fournisseurs**, Gemini et Groq mesurant leurs quotas séparément — l'alternance double

la capacité effective. Le **moteur local n'a aucun quota**, ce qui en fait le filet de sécurité en cas d'épuisement simultané. Et la **file de reprise** absorbe un dépassement de quota du côté Google sans perdre une ligne : la reprise a lieu cinq minutes plus tard, quand la fenêtre s'est rouverte.

Le facteur limitant n'est donc pas le nombre de rapports mais leur **simultanéité**. À la volumétrie visée — 5 à 10 rapports par jour, envoyés en fin de journée —, aucun quota gratuit n'est approché.

4.8.1 Du tableau estimé à la mesure : la page Consommation

Le Tableau 4.14 est un raisonnement, pas un relevé. Or **un quota qui s'épuise ne produit aucune erreur visible** : la chaîne d'extraction bascule sur le fournisseur suivant, ou WhatsApp se tait — deux comportements qui, vus de l'extérieur, ressemblent exactement à une journée normale. Un projet dont la viabilité tient entièrement à des offres gratuites ne pouvait pas rester sans instrument de mesure sur ce point précis.

La page **Consommation** (`app/usage.py`, `GET /usage`) le fournit. Sa règle de conception tient en une phrase : **chaque chiffre affiche d'où il vient**, parce que les trois fournisseurs ne sont pas également bavards — ce qui a été vérifié en conditions réelles et non supposé à la lecture des documentations.

- **Groq** publie le quota réel dans les en-têtes de chaque réponse (`x-ratelimit-limit/remaining-requests`, `-tokens`, et les fenêtres de réinitialisation). Les lire ne coûte rien, mais oblige à passer par la réponse brute du SDK : le corps déjà analysé jette les en-têtes.
- **Gemini** renvoie sa consommation réelle en jetons (`usage_metadata`) et un dictionnaire d'en-têtes **vide**. La consommation est donc mesurée et le plafond reste une constante documentée, signalée comme telle dans l'interface.
- **WhatsApp** expose son plafond en direct — palier d'envoi, qualité du numéro, débit — mais pas notre consommation. Le plafond vient donc de Meta et le décompte de `pipeline_logs` ; la réponse garde les deux séparés au lieu de les additionner en un pourcentage qui n'aurait aucun sens.

Deux décisions découlent de cette hétérogénéité. Un en-tête absent est enregistré **`NULL` et non `0`** — zéro se lirait « il ne reste rien », seule conclusion que la donnée ne permet pas. Et l'écriture dans `api_usage` **ne lève jamais d'exception** : une instrumentation ne vaut pas un rapport perdu.

La page est reproduite en **Annexe C, Figure C.6**, avec les autres écrans.

4.9 Statistiques et qualité du projet

4.9.1 Répartition du code

Le **Tableau 4.15** donne la répartition du code du système livré. Le décompte porte sur les fichiers versionnés, hors fichiers générés (verrous de dépendances, artefacts de construction) et hors dossier de rédaction — la source de ce rapport et du diaporama en est donc exclue. Total : **52 557 lignes**.

Tableau 4.15 : Répartition du code par langage.

Langage	Fichiers	Lignes	Part
Python	89	29 458	56,0 %
TypeScript	63	15 709	29,9 %
Markdown	43	5 682	10,8 %
YAML	4	644	1,2 %
JavaScript	4	568	1,1 %
CSS	1	482	0,9 %
HTML	1	14	0,0 %

Deux lectures méritent d'être faites. La part de **Markdown à 10 %** n'est pas de la documentation d'accompagnement : elle contient les instructions versionnées des modèles (§3.5.5), qui sont du contenu exécutable au même titre que du code. Et l'équilibre Python/TypeScript reflète fidèlement l'architecture - un service métier substantiel et une interface d'administration complète, plutôt qu'une simple façade

4.9.2 Indicateurs de qualité mesurés

Le **Tableau 4.16** rassemble les indicateurs de qualité mesurés.

Tableau 4.16 : Indicateurs de qualité mesurés.

Indicateur	Valeur
Tests automatisés	860, répartis en 49 fichiers (810 hors ligne, 50 marqués `integration`)
Lignes de test	10 916
Ratio test / code applicatif	0,71 ligne de test par ligne de service (10 916 / 15 380)
Dépendances d'exécution requises pour les tests	**Aucune** pour 810 d'entre eux — ni clé d'API, ni base ; les 50 autres sont ignorés sans PostgreSQL
Durée de la suite complète	8 min 59 s (mesuré : 810 succès, 50 ignorés, 0 échec)
Couverture d'instructions du service	78 % (`pytest --cov=app`, 5 311 instructions)
Contrôles d'intégration continue	**18** par *push* (détail au §4.9.3)
Analyse statique externe	SonarCloud, sur chaque *push* (détail au §4.9.4)
Mises à jour de dépendances automatisées	Dependabot, 5 écosystèmes (détail au §4.9.3)
Période de développement	2 mars – 31 août 2026, soit 26 semaines
Activité versionnée	376 commits

Une précision s'impose sur la couverture, car **deux chiffres différents apparaissent dans ce chapitre** : 78 % ici et 80,7 % au Tableau 4.18. Ils ne se contredisent pas, ils ne comptent pas la même chose. `pytest --cov` rapporte un taux d'**instructions** (5 311 instructions, 1 189 non atteintes) ; SonarCloud rapporte un taux de **lignes exécutables**, une base plus large qui inclut des lignes qu'un instrumenteur Python ne compte pas comme instructions. Le second est le chiffre publié et opposable, puisque c'est celui que le portail qualité évalue ; le premier est celui que reproduit une exécution locale. Les deux sont donnés parce que **le lecteur qui lance la suite lui-même verra 78 %**, et qu'un rapport qui n'annonce que 80,7 % lui laisserait croire à une régression.

4.9.3 La chaîne d'intégration continue

Chaque *push* sur la branche principale déclenche **dix-huit contrôles automatiques**, définis dans un unique fichier `.github/workflows/ci.yml`. Ils se répartissent en cinq familles, et cette répartition n'est pas décorative : elle correspond à quatre natures de défaut distinctes, qu'aucun outil unique ne détecte toutes.

Le **Tableau 4.17** en donne le détail, famille par famille.

Tableau 4.17 : Les dix-huit contrôles d'intégration continue, par famille.

Famille	Contrôles	Ce qu'ils attrapent
Correction	`API — lint, types & tests` (ruff, 860 tests), `Dashboard — types, lint & build` (tsc, oxlint), `n8n workflow`, `Docker compose config`, `Locale key parity`	Régressions de logique, dérive de types, workflow invalide, clé de traduction manquante dans l'une des trois langues
Sécurité du code	`CodeQL (python)`, `CodeQL (javascript-typescript)`, `Semgrep (OWASP Top 10)`, `Bandit SAST`	Injections, chemins de données non assainis, primitives cryptographiques faibles
Dépendances et secrets	`Trivy filesystem scan`, `Trivy config scan`, `pip-audit`, `npm audit`, `Secret scan` (gitleaks + `scripts/check_secrets.py`)	Vulnérabilités connues des bibliothèques tierces, mauvaise configuration d'infrastructure, jeton commité
Conformité des fichiers	`Hadolint (api/Dockerfile)`, `Hadolint (Dashboard/Dockerfile)`, `actionlint`	Images Docker non conformes aux bonnes pratiques, workflow GitHub Actions syntaxiquement fragile

Famille	Contrôles	Ce qu'ils attrapent
Analyse externe	`SonarCloud`	Référentiel de qualité indépendant, appliqué à l'ensemble des sources ; consomme le rapport de couverture produit par le contrôle `API` (§4.9.4)

Deux points méritent d'être soulignés. Le contrôle de secrets s'exécute sur **l'historique complet** (`fetch-depth: 0`) et non sur le seul dernier état : un jeton retiré dans le dernier commit reste public dans l'historique, et ne le vérifier qu'au sommet donnerait une fausse assurance. Et la **parité des clés de traduction** est un contrôle propre à ce projet : l'interface étant trilingue, une clé ajoutée en français mais oubliée en arabe ne casse aucun test et ne se voit qu'à l'usage, en production, par un utilisateur arabophone.

Dependabot complète le dispositif : configuré sur **cinq écosystèmes** — `npm` pour le tableau de bord, `pip` pour le service, les deux images Docker et les actions GitHub elles-mêmes — il ouvre **quotidiennement** des demandes de mise à jour, groupées par niveau de version pour éviter une avalanche de demandes unitaires. **Trente-huit** demandes ont été ouvertes sur la durée du projet, dont **neuf restent ouvertes** à la date de rédaction ; les vingt-neuf autres ont été **fermées sans fusion**, remplacées par une demande plus récente du même groupe — c'est le comportement normal d'un regroupement par niveau de version, chaque nouvelle exécution quotidienne supplantant la précédente tant qu'elle n'a pas été traitée. Aucune n'a donc encore été fusionnée : le dispositif a été mis en place pendant la phase d'industrialisation et sa valeur, à ce stade, est de **rendre visible** l'écart entre les versions épinglées et l'état de l'amont, non de l'avoir déjà résorbé. Chacune franchit les dix-huit contrôles **avant de pouvoir être fusionnée** : la mise à jour automatique n'a d'intérêt que si elle est automatiquement vérifiée, sans quoi elle ne fait que déplacer le risque.

La **Figure 4.3** représente la chaîne complète, des deux sources de changement jusqu'à la fusion.

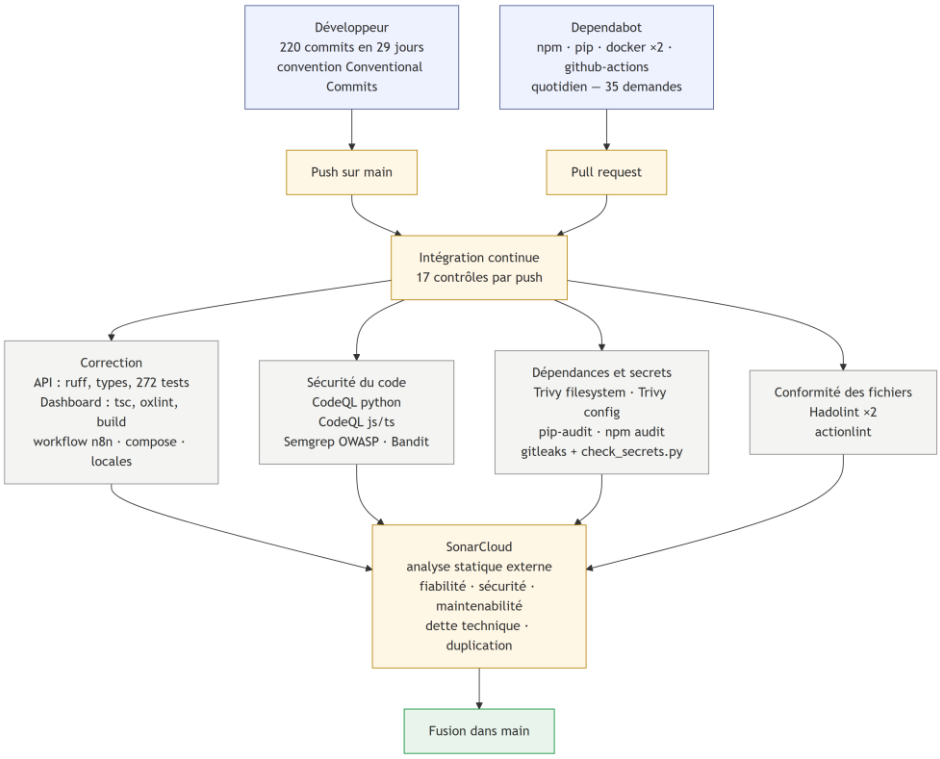


Figure 4.3 : *La chaîne d'intégration continue : les deux sources de changement, les dix-huit contrôles et l'analyse statique externe avant fusion.*

4.9.4 Analyse statique externe

Au-delà des contrôles exécutés dans le dépôt, le projet est analysé par **SonarCloud**, qui fournit une lecture **indépendante** de la qualité du code. L'intérêt n'est pas d'ajouter un outil de plus : les quatre familles du §4.9.3 sont toutes configurées par le développeur du projet, donc toutes susceptibles du même angle mort. SonarCloud applique un référentiel externe, non négocié, à l'intégralité des sources Python et TypeScript (`sonar-project.properties` : `api/app`, `Dashboard/src`, tests inclus).

Il mesure cinq dimensions que les contrôles internes ne quantifient pas : la **fiabilité** (défauts susceptibles de provoquer un comportement erroné), la **sécurité** (vulnérabilités et points sensibles à revoir), la **maintenabilité** (dette technique, exprimée en temps de correction estimé), le **taux de duplication** et la **couverture de tests**. Le **Tableau 4.18** rassemble les valeurs relevées sur la dernière analyse.

Tableau 4.18 : Indicateurs SonarCloud du projet.

Indicateur	Valeur	Note
Lignes de code analysées	21 673 (87 fichiers de source)	—
Fiabilité (*bugs*)	**0**	**A**

Indicateur	Valeur	Note
Sécurité (vulnérabilités)	**0**	**A**
Points sensibles de sécurité (*hotspots*)	0, dont 100 % revus	**A**
Maintenabilité (*code smells*)	**0**	**A**
Dette technique estimée	**0 minute**	—
Taux de dette technique	0,0 %	—
Duplication de lignes	**0,0 %**	—
Densité de commentaires	26,2 %	—
Couverture de tests (rapport produit par `pytest --cov`)	**80,7 %**	—
Couverture du *code nouveau*	**80,2 %** (seuil du portail : 80 %)	—
Portail qualité (*quality gate*)	**6 conditions sur 6 satisfaites — *Passed* —**	—

Les notes A n'ont pas été obtenues du premier coup. L'analyse initiale rendait un verdict rouge : fiabilité **E**, sécurité **C**, portail qualité en échec. Quatre constats en étaient la cause, et leur traitement illustre les trois seules issues honnêtes devant un analyseur externe.

- **Trois étaient des défauts réels**, tous dans l'image du moteur local : elle s'exécutait en `root`, installait ses dépendances depuis un fichier non épinglé et acceptait les archives sources, autorisant donc l'exécution d'un script d'installation au moment de la construction. L'image de l'API appliquait déjà exactement les protections manquantes ; c'est ce patron qui a été repris. Les dépendances sont désormais épinglées aux versions avec lesquelles le banc d'essai a réellement été exécuté — non aux dernières publiées, qui n'ont jamais transcrit un seul clip.
- **Une était un faux positif** : la règle signalait comme inexistant un argument `size` qui existe dans Pillow depuis la version 10.1.0, alors que le projet en épingle la 12.3.0. Vérifié à l'exécution, puis marqué comme tel dans l'outil, justification jointe.
- **Une dernière est un refus assumé** : l'outil demande un verrouillage par empreintes des dépendances du moteur local. Il n'est pas applicable ici, car `torch` provient de l'image de base CUDA et non du fichier de dépendances ; un verrou produit hors de cette image y ferait entrer la variante processeur de `torch` et casserait silencieusement l'inférence sur carte graphique. Le refus est enregistré dans l'outil avec ce motif.

Cet argument a été maintenu, puis les tests manquants ont été écrits : la suite est passée de 475 à **860 tests** et la couverture de 63,1 % à **80,7 %**. Les deux positions ne se contredisent pas, et il faut le dire clairement pour que la seconde ne se lise pas comme un désaveu de la première. Écrire ces tests n'a pas rendu la suite capable d'attraper une rupture de contrat externe — elle en est toujours

structurellement incapable, et le §4.5 reste vrai. Ce que l'écart de dix-sept points a réellement couvert, ce sont les branches internes : reconciliation de stock, normalisation des dates, fusion des lignes du journal, validation du schéma. Ces chemins-là sont de l'arithmétique et de la logique métier, ils sont testables hors ligne, et rien ne justifiait qu'ils ne le soient pas.

Ce chiffre décrit le service Python, et lui seul. Le tableau de bord est exclu du calcul de couverture — et de celui-là seulement, son code restant analysé pour la fiabilité, la sécurité et la maintenabilité. La raison est arithmétique : il n'a aucun exécuteur de tests, si bien que ses quelque deux mille lignes compteraient intégralement comme non couvertes et ramèneraient le résultat à 44,6 %. Ce ne serait pas une mesure basse, ce serait la moyenne d'une mesure et d'une **non**-mesure — un chiffre qui parle de l'absence d'exécuteur de tests et non du code. L'exclusion porte la condition de sa propre suppression : le jour où un exécuteur écrit le rapport que la configuration désigne déjà.

Le portail qualité satisfait aujourd'hui ses six conditions, dont la sixième qui était en échec : fiabilité, sécurité et maintenabilité du code nouveau notées **A**, duplication du code nouveau à **0,0 %**, points sensibles revus à **100 %**, et couverture du code nouveau à **80,2 %** pour un seuil à 80,0 %. Cette dernière marge — deux dixièmes de point — mérite d'être nommée plutôt que célébrée : elle signifie que la condition est satisfaite, pas qu'elle est confortablement satisfaite, et qu'un module ajouté sans tests la fera repasser au rouge. C'est le comportement attendu d'un seuil, non un défaut du dispositif.

Un défaut de méthode a été révélé par cette mise en place, et il mérite d'être rapporté parce qu'il invalidait une affirmation de ce rapport. La chaîne d'intégration continue était **rouge à chaque exécution** depuis plusieurs jours sans que cela soit vu. La cause : un test vérifiant qu'un dépassement de délai réseau se traduit bien en erreur de canal ne fournissait pas le jeton attendu par la fonction testée. Sur la machine de développement, le fichier d'environnement local en fournissait un et le test passait ; en intégration continue, où il n'y en a aucun, la fonction s'arrêtait sur son garde de configuration, bien avant le comportement que le test prétendait éprouver. **C'est la règle « les tests passent sans aucune clé d'API » violée par le test lui-même**, et masquée par l'environnement local — exactement le mode de défaillance silencieux décrit au §4.3.7 à propos du banc d'essai. Le test corrigé injecte son jeton, comme le faisait déjà son jumeau WhatsApp dans le même fichier. Les dix-huit contrôles passent depuis.

La **Figure 4.4** reproduit le tableau de bord de l'outil, tel qu'il est publié.

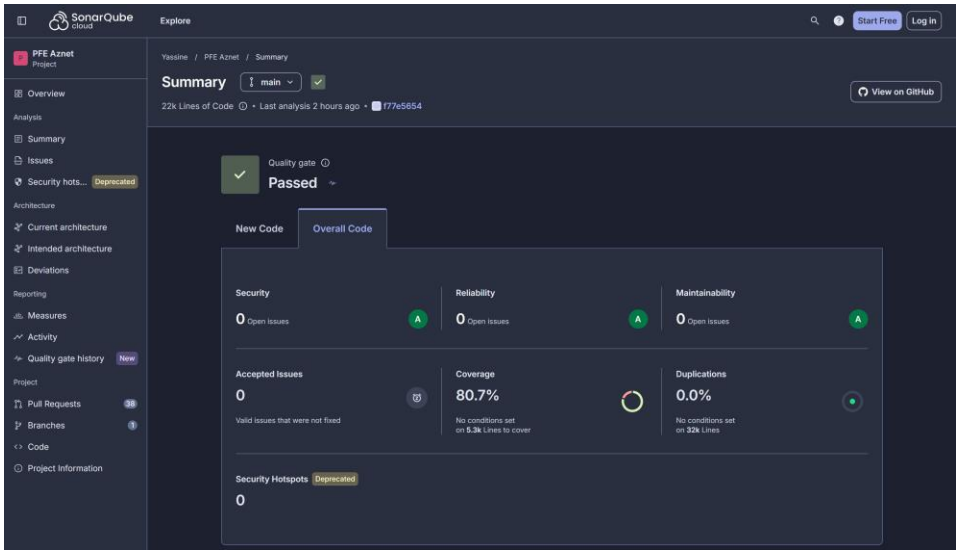


Figure 4.4 : *Le tableau de bord SonarCloud du projet, capturé le 21 août 2026 sur la branche principale.*

4.9.5 Activité du dépôt : rythme et volume

La section précédente décrit *ce que* les contrôles vérifient ; celle-ci décrit *ce qui* leur a été soumis. Les chiffres proviennent de l'historique Git lui-même, non d'un décompte tenu à la main : le dépôt totalise **376 commits** entre le **15 juillet et le 21 août 2026**, soit 38 jours calendaires dont **34 actifs**, une moyenne de **9,9 commits par jour** et un maximum de **34** sur une seule journée. Le volume cumulé est de **+117 339 lignes ajoutées** pour **45 454 retirées**.

Une précision s'impose avant toute lecture de ces chiffres, sans quoi ils induiraient en erreur : **le dépôt ne couvre pas les six mois du projet**. Il a été initialisé au début de la phase 3, et les phases 0 à 2 — cadrage, état de l'art, conception — lui sont antérieures ; leurs livrables sont des documents et des décisions, non du code. Les chiffres ci-dessus mesurent donc la **réalisation**, pas le projet.

Trois lectures méritent d'être faites, et la troisième est une limite.

Le rythme n'est pas régulier, et il n'avait pas de raison de l'être. Trois pointes se détachent : le 31 juillet (28 commits), le 15 août (34) et les 20-21 août (29 puis 28). Elles ne correspondent pas à des journées plus longues mais à des **fins de tâche**, où le travail se solde en autant de commits qu'il reste de points à traiter. Les deux dernières le montrent bien : le 15 août est la refonte du schéma métier sur les bordereaux réels et le reclassement des chaînes d'extraction ; les 20-21 août sont le durcissement final — cinq défauts trouvés par une revue croisée, la signature du webhook WhatsApp, puis la mise à zéro de la liste SonarCloud.

Le Tableau 4.19 donne la répartition par type de commit, et fait apparaître un défaut de discipline qu'il serait malhonnête de taire.

Tableau 4.19 : Répartition des 376 commits par type (convention *Conventional Commits*).

Type	Commits	Part	Ce qu'il désigne
`feat`	112	29,8 %	Une capacité nouvelle
`fix`	79	21,0 %	Une correction de comportement
`docs`	54	14,4 %	Documentation, y compris les instructions versionnées des modèles
`refactor`	10	2,7 %	Réorganisation sans changement de comportement
`chore`	9	2,4 %	Outillage, dépendances
`ci`	7	1,9 %	Chaîne d'intégration continue
`perf`	5	1,3 %	Performance
`test`	4	1,1 %	Tests seuls
`style`	3	0,8 %	Mise en forme
`revert`, `build`	3	0,8 %	Annulation, construction
Hors convention	**90**	**23,9 %**	Préfixes non normalisés (`to-do:`, `sonar fix:`…)

Le rapport `feat` / `fix` de **1,4 pour 1** est cohérent avec un projet dont l'essentiel du risque était en intégration : chaque capacité nouvelle a coûté en moyenne 0,7 correction, et les corrections ne se concentrent pas au début mais se répartissent sur toute la durée — c'est la signature d'un système qui rencontre ses défauts au contact de fournisseurs externes plutôt qu'à la compilation.

Les **90 commits hors convention** — près d'un quart — sont un manquement réel. La convention *Conventional Commits* est annoncée dans les règles du projet, et un historique où un message sur quatre s'en écarte ne peut pas alimenter la génération automatique d'un journal des versions, qui est la seule raison pratique d'adopter cette convention. Ils sont concentrés sur les commits de suivi (`to-do:`) et de correction d'analyse statique, c'est-à-dire précisément les moments où l'attention était ailleurs. La correction n'est pas rétroactive — réécrire l'historique d'un dépôt pour le rendre présentable serait le contraire d'une mesure — mais un contrôle d'intégration continue sur le format du message aurait empêché le premier écart, et son absence est un oubli de la phase d'industrialisation.

4.9.6 Chronologie réelle du développement

Le planning prévisionnel du §1.6.2 annonçait un découpage ; le **Tableau 4.20** confronte cette annonce au déroulement constaté, phase par phase, et signale les écarts.

Tableau 4.20 : Déroulement constaté, comparé au planning prévisionnel.

Phase	Prévu	Constaté	Écart et cause
0 - Cadrage	2 – 27 mars	Conforme	—
1 - État de l'art	30 mars – 30 avril	Conforme	Le jeu de test réel n'a pu être constitué qu'en fin de phase : il dépendait d'enregistrements que les techniciens devaient produire eux-mêmes
2 - Conception	4 mai – 5 juin	Conforme, révisée en phase 5	Le contrat `ExtractedReport` a été reconstruit après consultation du fichier de suivi réel du directeur — un document que la phase 2 ne connaissait pas encore —
3 - Cœur IA	8 juin – 17 juillet	Conforme, avec avance sur le bout-en-bout	La chaîne complète a été validée en conditions réelles dès les premiers jours, alors que le planning la situait en phase 4
4 - Chaîne complète et Web	20 juillet – 1er août	Conforme, périmètre élargi	Le tableau de bord a été entièrement reconstruit : la première version, bâtie sur un cadre de composants tiers, ne supportait ni le trilinguisme ni le sens d'écriture droite-à-gauche

Phase	Prévu	Constaté	Écart et cause
5 - Banc d'essai et durcissement	3 – 14 août	Conforme, effort sous-estimé	Le banc d'essai local a exigé trois corrections du harnais lui-même (§4.3.7) avant de produire un résultat exploitable
6 - rapport	17 – 28 août, tampon jusqu'au 31	Terminé	—

Deux enseignements se dégagent. D'abord, **les écarts ne portent jamais sur la durée mais sur le contenu** : aucune phase n'a débordé, mais trois ont vu leur périmètre révisé au contact du terrain — le fichier de suivi réel, la contrainte trilingue, la fragilité du harnais de mesure. Ensuite, la **neutralisation précoce du risque d'intégration** a payé : valider le maillon le plus incertain avec une phase d'avance a laissé le temps de découvrir les défauts de contrat externe (§4.4) au lieu de les rencontrer en phase 6.

4.10 Conclusion du chapitre

La réalisation a concrétisé la conception et l'a confrontée au réel. Le banc d'essai, mené sur treize moteurs, a produit un résultat qu'aucune revue de littérature n'aurait donné : le moteur le plus exact pour ce cas d'usage n'est pas le service commercial de référence mais un affinage open-source du darija, et ce constat n'a pu émerger que parce que des contrôles non affinés avaient été inclus et parce que trois bogues de harnais — dont un silencieux et flatteur — ont été débusqués. Six défauts d'intégration invisibles hors ligne ont par ailleurs été diagnostiqués et corrigés. La chaîne complète fonctionne de bout en bout depuis la phase 3, repli compris.

Bilan et compétences acquises

Compétences techniques. Ce projet a imposé d'aller au bout de plusieurs domaines plutôt que d'en effleurer beaucoup : la conception d'une API en Python avec un contrat de données strict ; l'intégration de grands modèles de langage avec sortie contrainte et chaînes de repli ; l'évaluation empirique de treize modèles de reconnaissance vocale, avec la mise au point du harnais que cela suppose ; la conception d'un schéma relationnel et la migration de données réelles ; le développement d'une interface React trilingue avec prise en charge de l'écriture de droite à gauche ; et la mise en production conteneurisée d'une pile de cinq services, dont un service GPU.

Compétences méthodologiques. Trois habitudes ont plus compté que n'importe quelle bibliothèque.

Neutraliser le risque le plus élevé en premier. La chaîne complète a été validée de bout en bout dès l'ouverture de la phase 3, alors que le planning la prévoyait en phase 4. C'est ce qui a permis de découvrir tôt les défauts d'intégration qu'aucun test hors ligne ne pouvait révéler.

Se méfier de ses propres mesures. Le banc d'essai comportait un bogue silencieux qui aurait couronné un modèle sur une transcription qu'il n'avait jamais produite. Il n'a été détecté que parce que deux indicateurs secondaires étaient incohérents. La leçon retenue est qu'un résultat flatteur mérite plus de vérification qu'un résultat décevant.

Écrire pourquoi, pas seulement quoi. Les décisions non évidentes de ce système sont documentées à l'endroit du code qu'elles concernent. Plusieurs fois au cours du projet, une correction a été évitée parce qu'un commentaire expliquait qu'un comportement surprenant était voulu.

Ce que je referais autrement. Trois choses. D'abord, **constituer la vérité terrain dès le premier jour** : son absence est la principale limite du banc d'essai, et enregistrer quelques transcriptions de référence dès la constitution du jeu de test aurait coûté une heure. Ensuite, **écrire les tests d'intégration réels plus tôt** : la suite hors ligne est excellente contre les régressions de logique, mais tous les défauts sérieux rencontrés venaient de contrats externes qu'elle ne touche pas. Enfin, **traiter la documentation comme du code dès le départ** : plusieurs affirmations du fichier d'architecture étaient devenues fausses sans que rien ne le signale, et elles n'ont été corrigées que lorsqu'un travail ultérieur les a confrontées à la réalité.

Compétences humaines. Le dialogue avec l'encadrant industriel a été déterminant sur un point précis : le schéma de données a été reconstruit à partir de son fichier de suivi réel, et non de ce qu'un développeur imaginait utile. Les trois compteurs de prises distincts, incompréhensibles hors contexte, sont la trace directe de cette conversation. C'est probablement l'enseignement le plus transférable de ce projet : la qualité d'un modèle de données dépend moins de la technique que de la précision avec laquelle on a écouté celui qui s'en servira.

Conclusion générale et perspectives

Rappel des objectifs. Ce projet de fin d'études visait à automatiser la production des rapports journaliers des techniciens de câblage d'AZNET : permettre à un technicien de transmettre son compte rendu par une simple note vocale en darija, et obtenir automatiquement un rapport structuré en français professionnel, centralisé dans le fichier de suivi et notifié aux responsables — le tout sans budget logiciel.

Contributions. Le travail réalisé apporte :

1. **Une architecture hybride** combinant orchestration low-code et service métier sur mesure, dont la frontière a été fixée par une contrainte fonctionnelle identifiée en cours de projet — une logique de lecture-modification-écriture n'est pas exprimable dans un nœud conditionnel — et non par préférence.
2. **Un banc d'essai comparatif de treize moteurs de transcription du darija**, langue peu dotée, assorti d'une critique argumentée du WER pour ce cas d'usage et d'une métrique applicative de substitution ; il a renversé l'hypothèse technologique initiale du cahier des charges.
3. **Un service d'extraction** fondé sur les grands modèles de langage avec sortie structurée contrainte, des chaînes de repli explicites alternant les fournisseurs, et une politique stricte

d'absence de données inventées, poussée jusqu'à faire échouer une localisation non reconnue plutôt que de l'ignorer.

4. **Un modèle métier reconstruit sur le fichier réel du client**, tenant un journal par chantier à raison d'une ligne par message, avec réconciliation de stock et création de colonne à la demande — trois règles sans lesquelles le seul chiffre que lit le directeur, la ligne de totaux, serait faux ou incomplet.
5. **Une conception indépendante du canal**, démontrée en pratique par la mise en service successive de deux messageries aux contraintes opposées, sans modification du cœur du système.

Résultats. La chaîne complète est opérationnelle depuis la phase 3, avec un mécanisme de repli validé en conditions réelles. Le système compte 85 routes, 64 nœuds d'orchestration, 26 tables et 860 tests automatisés exécutables sans aucune clé d'API.

Limites. Trois limites doivent être énoncées sans détour. D'abord, **le WER et le CER n'ont pas été calculés** : le choix d'une métrique applicative est argumenté au §4.3.2, mais il prive la comparaison d'un chiffre unique et universellement reconnu, et le classement final repose sur un jugement humain — rigoureux, mais non reproductible par un tiers sans le même locuteur. Ensuite, **la suite de tests ne parcourt jamais le réseau réel** ; elle sécurise les régressions de logique, non les ruptures de contrat des fournisseurs, qui ont été la source de la majorité des défauts rencontrés.

Perspectives. Plusieurs évolutions prolongent naturellement ce travail : constituer la vérité terrain qui manque, en s'appuyant sur les rapports réels accumulés, afin de publier un WER et un CER et de rendre le classement reproductible ; affiner un modèle de transcription sur ce corpus darija propre au câblage, l'apport de l'affinage étant désormais démontré pour un modèle assez grand ; remplacer Google Sheets par une base de données lorsque la volumétrie le justifiera, l'infrastructure PostgreSQL étant déjà en place ; prolonger l'analytique de chantier au-delà de ce qui a été livré — l'avancement par site, le registre des réceptions et les alertes de stock en jours de couverture existent désormais, mais la prévision de commande, elle, reste à construire ; et généraliser la plateforme à plusieurs équipes ou entreprises.

Au-delà du système livré, ce projet illustre une conviction d'ingénierie : dans les applications des grands modèles de langage, la valeur ne réside pas dans le choix des modèles, mais dans l'architecture qui les entoure — contrats de données stricts, chaînes de repli explicites, refus des échecs silencieux, et bancs d'essai assez sceptiques pour se surprendre eux-mêmes. Le banc d'essai de ce projet a renversé son hypothèse de départ deux fois, et démasqué un bogue qui lui faisait couronner un vainqueur sur une transcription jamais générée. C'est cette discipline, plus que le modèle finalement retenu, qui constitue le résultat le plus transférable de ce travail.

Références bibliographiques

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey et I. Sutskever, « Robust Speech Recognition via Large-Scale Weak Supervision », *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR vol. 202, pp. 28492–28518, 2023.

[2] Groq, « GroqCloud Documentation — Speech to Text & Chat Completions ». Disponible sur : https://console.groq.com/docs

[3] Google, « Gemini API Documentation — Structured Output & Audio Understanding ». Disponible sur : https://ai.google.dev/gemini-api/docs

[4] Telegram, « Telegram Bot API ». Disponible sur : https://core.telegram.org/bots/api

[5] Meta, « WhatsApp Business Platform — Cloud API Documentation ». Disponible sur : https://developers.facebook.com/docs/whatsapp

[6] n8n, « n8n Documentation — Workflow Automation ». Disponible sur : https://docs.n8n.io

[7] S. Ramírez, « FastAPI Documentation ». Disponible sur : https://fastapi.tiangolo.com

[8] Pydantic, « Pydantic Documentation — Data Validation ». Disponible sur : https://docs.pydantic.dev

[9] K. Schwaber et J. Sutherland, *The Scrum Guide*, 2020. Disponible sur : https://scrumguides.org

[10] E. Gamma, R. Helm, R. Johnson et J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[11] A. Baevski, Y. Zhou, A. Mohamed et M. Auli, « wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations », *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 12449–12460, 2020.

[12] A. Graves, S. Fernández, F. Gomez et J. Schmidhuber, « Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks », *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 369–376, 2006.

[13] A. Bounhar, « Morocco-Darija-STT-large ». Hugging Face, licence Apache 2.0. Disponible sur : https://huggingface.co/BounharAbdelaziz/Morocco-Darija-STT-large

[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang et W. Chen, « LoRA: Low-Rank Adaptation of Large Language Models », *International Conference on Learning Representations (ICLR)*, 2022.

[15] M. Labied et A. Belangour, « Moroccan Dialect "Darija" Automatic Speech Recognition: A Survey », in 2021 IEEE 2nd International Conference on Pattern Recognition and..., 2021.

[16] I. Benelallam, A. Allak et A. M. Naira, « Dvoice: An Open Source Dataset for Automatic Speech Recognition on Moroccan Dialectal Arabic », septembre 2021.

[17] A. Boumehdi et A. Yousfi, « MDVC Corpus: Empowering Moroccan Darija Speech Recognition », Indonesian Journal of Electrical Engineering and Computer Science, 2024.

Annexes

Annexe A : Documentation d'installation

A.1 Prérequis

Docker et Docker Compose ; Python 3.11 et Node.js pour le développement hors conteneur ; une carte NVIDIA avec pilote à jour **uniquement** si le moteur de transcription local est utilisé ; un compte de service Google disposant d'un accès en écriture au seul classeur de suivi.

A.2 Configuration

Toute la configuration est portée par un fichier `.env` non versionné, dont un modèle documenté (`.env.example`) accompagne le dépôt. Les clés d'API sont optionnelles au démarrage et vérifiées à l'appel : le service démarre sans secret — ce qui est nécessaire aux tests automatisés — mais échoue avec un message explicite si un appel en requiert un. Le mot de passe de la base n'a délibérément aucune valeur par défaut, afin qu'un déploiement mal configuré échoue bruyamment plutôt que de démarrer avec un mot de passe connu.

A.3 Démarrage de la pile

```
docker compose up -d
```

Cinq services démarrent : `postgres`, `n8n`, `api`, `stt-local` et `web`. Un seul port est publié, celui du proxy, qui sert le tableau de bord, l'API sous `/api/` et les webhooks sous `/webhook/`. Les trois services centraux sont en redémarrage automatique, de sorte que la chaîne se rétablit seule après un redémarrage de la machine.

A.4 Développement du service

```
cd api
.venv/Scripts/python.exe -m pytest              # 860 tests (810 hors ligne,
50 marqués integration)
.venv/Scripts/python.exe -m uvicorn app.main:app --reload
```

A.5 Deux pièges de déploiement

Ces deux points ont réellement coûté du temps et méritent d'être connus, car dans les deux cas **une modification peut sembler appliquée sans être exécutée** :

1. Le service est une **image construite**, code source inclus. Un simple redémarrage du conteneur réexécute l'ancien code ; il faut reconstruire l'image.
2. n8n conserve **sa propre copie** du workflow dans son volume. Modifier le fichier JSON versionné ne change rien tant qu'il n'a pas été réimporté ou poussé par l'API de n8n.

A.6 Étape manuelle après l'import du workflow

Le gestionnaire d'erreurs n'est actif que si le workflow est désigné comme son propre *Error Workflow* dans ses paramètres. Sans cette manipulation, le déclencheur d'erreur ne se déclenche jamais.

Annexe B : Protocole et données du banc d'essai

B.1 Protocole

Corpus. Onze notes vocales réelles issues des échanges des techniciens, en darija spontanée avec code-switching français, de durées et de conditions d'enregistrement variées.

Campagnes. Trois passes successives : trois moteurs accessibles par API ; neuf moteurs exécutés localement ; puis dix moteurs, une fois obtenus les accès aux deux dépôts sous licence.

Matériel. NVIDIA RTX 3070 (8 Go), demi-précision, PyTorch avec CUDA.

Mesures relevées. Transcription intégrale, latence, mesure de confiance propre au moteur, longueur du texte, et nombre de chiffres émis.

Critère de décision. Ni WER ni CER — voir §4.3.2 pour la justification — mais la préservation des quantités et des emprunts techniques français, vérifiée clip par clip, puis **confirmée à l'oreille par un locuteur natif du darija**.

B.2 Catalogue des moteurs évalués

Moteur	Famille	Voie
`whisper-large-v3` via Groq	Whisper	API
Gemini (audio natif)	Multimodal	API
Affinage darija via Hugging Face	Whisper	API (jamais fonctionnel, voir §4.3.6)
`BounharAbdelaziz/Morocco-Darija-STT-large`	Whisper affiné	Local
`openai/whisper-large-v3`	Whisper (contrôle)	Local
`openai/whisper-large-v3-turbo`	Whisper (contrôle)	Local
`ychafiqui/whisper-medium-darija`	Whisper affiné	Local
`ychafiqui/whisper-small-darija`	Whisper affiné	Local
`KandirResearch/Whisper-Small-Darija`	Whisper affiné	Local (échec de chargement)
`TaloCreations/whisper-darija-finetuned`	Whisper affiné	Local
`C-Ilyas/whisper-base-darija`	Whisper affiné	Local
`boumehdi/wav2vec2-large-xlsr-moroccan-darija`	wav2vec2	Local

Moteur	Famille	Voie
`amineouaki/wav2vec2-darija-combined`	wav2vec2	Local
`amineouaki/wav2vec2-large-xlsr-53-darija-combined`	wav2vec2	Local
`anaszil/whisper-large-v3-turbo-darija`	Adaptateur LoRA	Local (non fusionné)
`Qwen/Qwen3-ASR-1.7B`	Autre	Local (sans exécuteur)

Tableau G.1 : Catalogue des moteurs téléchargés et évalués.

B.3 Reproductibilité

Le catalogue des points de contrôle et le harnais d'exécution sont versionnés dans le dépôt ; les poids eux-mêmes ne le sont pas — 5,8 Go, dont un fragment de 4,7 Go, ce qui dépasse la limite par fichier de Git LFS. Le script de téléchargement les reconstitue à l'identique, deux d'entre eux exigeant l'acceptation préalable d'une licence sur la plateforme.

B.4 Limite assumée

Le fichier de transcriptions de référence est resté vide : le WER et le CER n'ont donc **pas** été calculés. Le classement final repose sur une lecture experte et une validation par un locuteur natif, ce qui est rigoureux mais non reproductible par un tiers sans ce même locuteur. Constituer cette vérité terrain à partir des rapports réels accumulés est la première des perspectives énoncées en conclusion.

Annexe C : Le tableau de bord, écran par écran

Les captures ci-dessous proviennent de l'application réelle, connectée à la pile complète, en thème sombre et en anglais — l'interface étant trilingue, la langue d'affichage est un choix de l'utilisateur. Elles ont été produites par un script Playwright versionné dans le dépôt (`Dashboard/scripts/shots.mjs`), qui ouvre une session, attend la disparition des squelettes de chargement avant chaque capture — et l'attend **trois fois de suite**, les panneaux se montant par vagues, si bien qu'une seule lecture passe dans l'intervalle entre un panneau qui finit et le suivant qui commence — puis cadre chaque écran en 16:9. La vue téléphone fait exception au cadrage : elle est prise à 414 points de large. L'ensemble provient d'une seule exécution : un document reconstruit ne peut donc pas mélanger deux générations de l'interface.

C.1 Écrans principaux

La **Figure C.1** présente la vue d'ensemble, première page affichée après authentification.

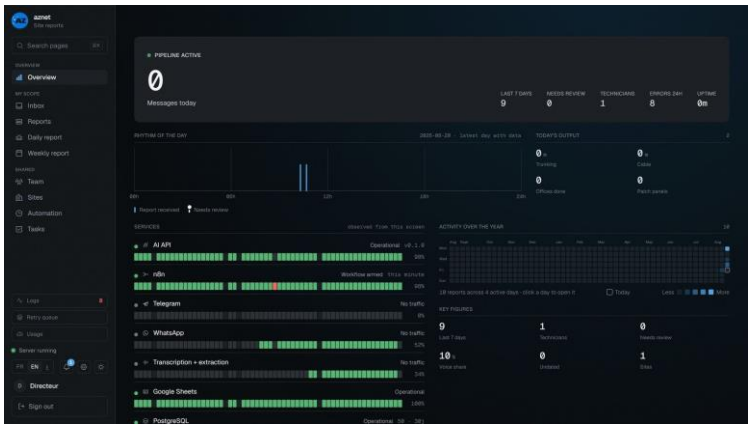


Figure C.1 : *Vue d'ensemble. Les repères du jour, le rythme horaire des messages, l'état des sept services et l'activité de l'année.*

La **Figure C.2** donne la boîte de réception, où chaque message reste rattaché au média qui l'a produit.

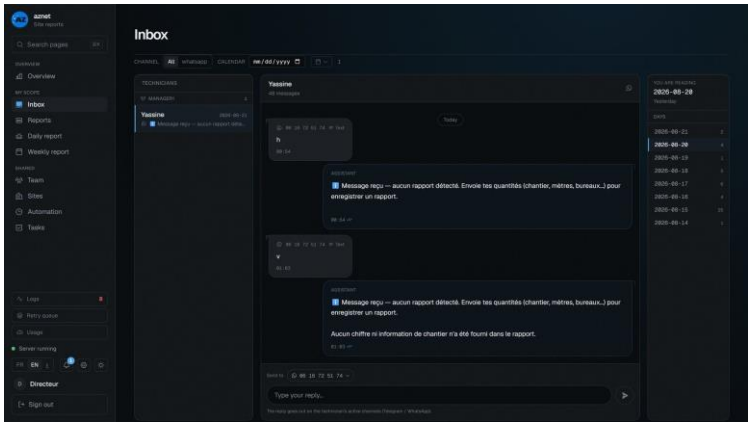


Figure C.2 : *Réception. Le flux des messages entrants, canal par canal, avec le média d'origine et le texte transcrit — l'écran qui permet de remonter d'une ligne douteuse à l'audio qui l'a produite.*

La **Figure C.3** montre l'écran des rapports, sur lequel une correction est écrite dans le classeur et non seulement affichée.

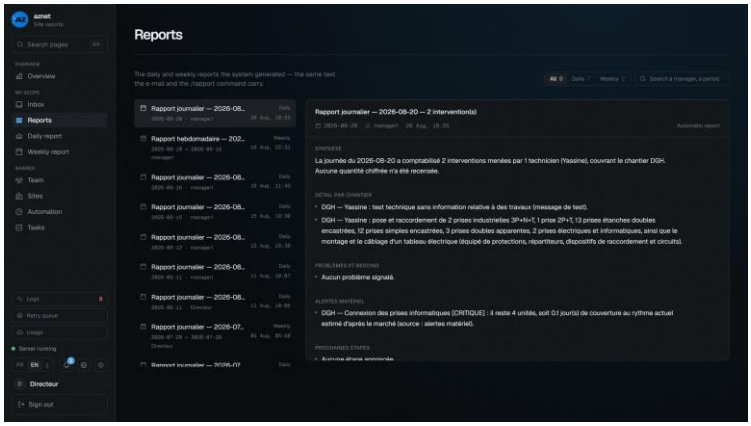


Figure C.3 : *Rapports. Le miroir plat du journal, filtrable et corrigeable ; une correction faite ici est écrite dans le classeur, pas seulement affichée.*

La **Figure C.4** donne la page Automatisation et l'état des neuf tâches planifiées.

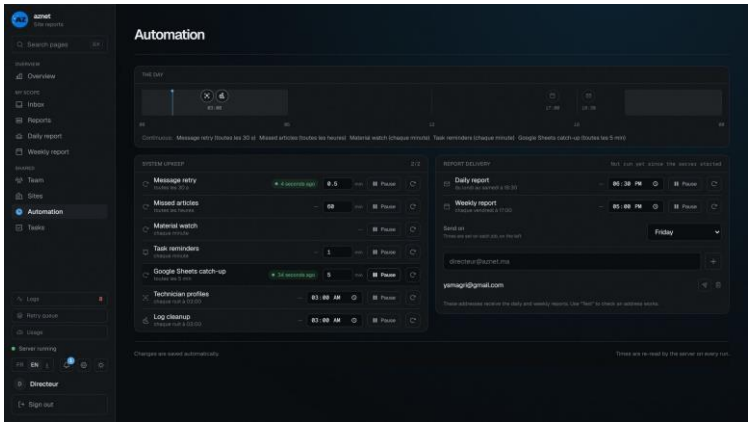


Figure C.4 : *Automatisation. Les neuf tâches planifiées, leur dernier résultat et leur horaire.*

La **Figure C.5** présente les journaux, où les étapes publiées par n8n et les erreurs du service se rejoignent en une seule chronologie.

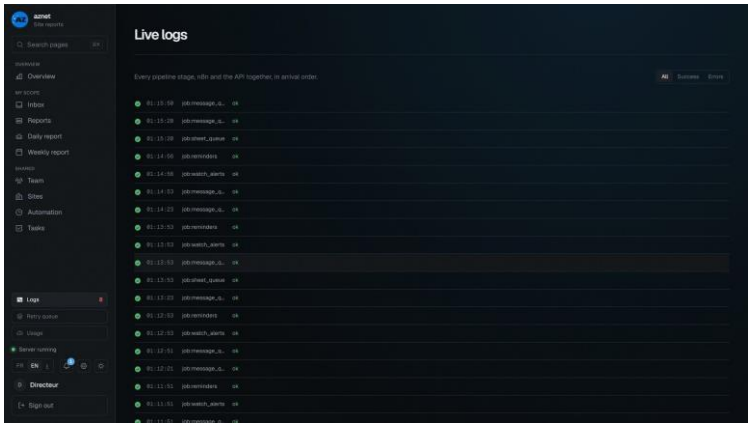


Figure C.5 : *Journaux. La chronologie du pipeline reconstituée par identifiant de corrélation : les étapes publiées par n8n et les erreurs journalisées par le service apparaissent dans une seule suite.*

La **Figure C.6** montre la page Consommation, où chaque offre gratuite dit ce qu'il lui reste et **d'où vient le chiffre**.

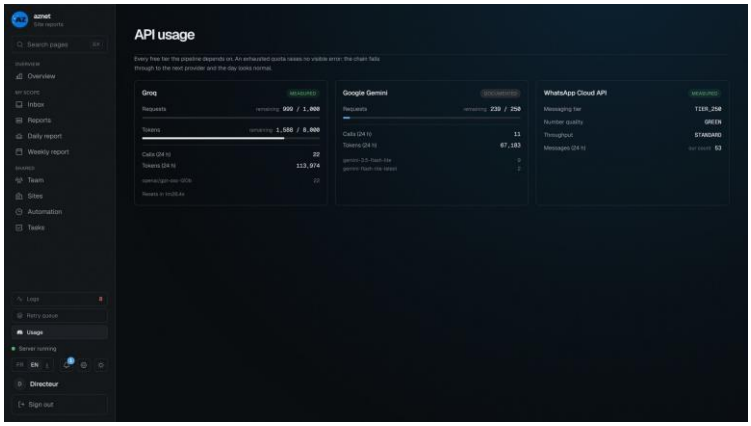


Figure C.6 : *Consommation des API. Groq publie son quota dans les en-têtes de chaque réponse et la carte est marquée measured ; Gemini ne publie rien, sa consommation est mesurée mais son plafond est une constante documented ; WhatsApp expose son palier d'envoi et sa qualité de numéro, le décompte des messages restant le nôtre.*

La **Figure C.7** donne la page Chantiers, qui rassemble l'administration d'un site.

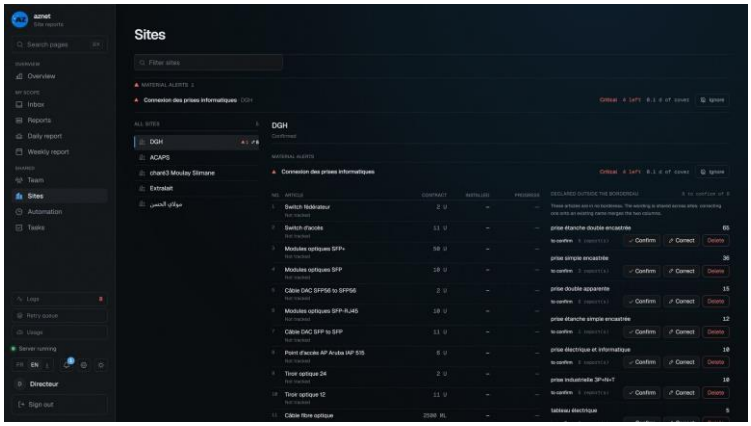


Figure C.7 : *Chantiers, la page entière.*

La **Figure C.8** donne l'organigramme et ses trois bandes.

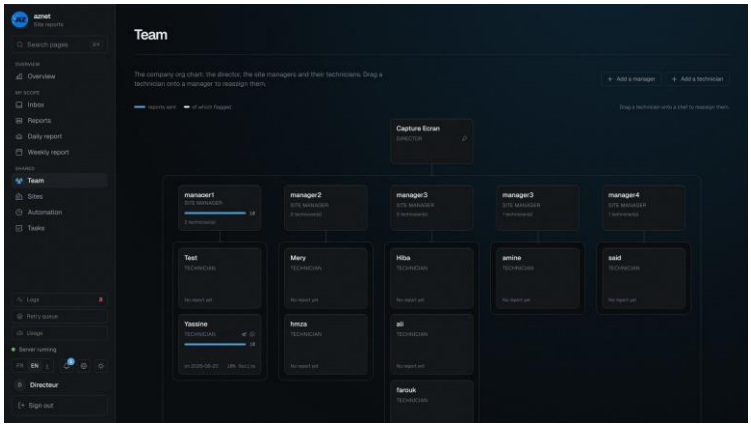


Figure C.8 : *Organigramme. Trois bandes fixes — directeur, chefs, techniciens — et non une récursion, ce qui est la raison pour laquelle aucun garde-fou anti-cycle n'est nécessaire.*

C.2 Usage mobile

L'interface est utilisable depuis un téléphone, ce qui compte pour un chef de chantier qui consulte l'avancement sans revenir au bureau.

La **Figure C.9** montre la vue d'ensemble sur téléphone.

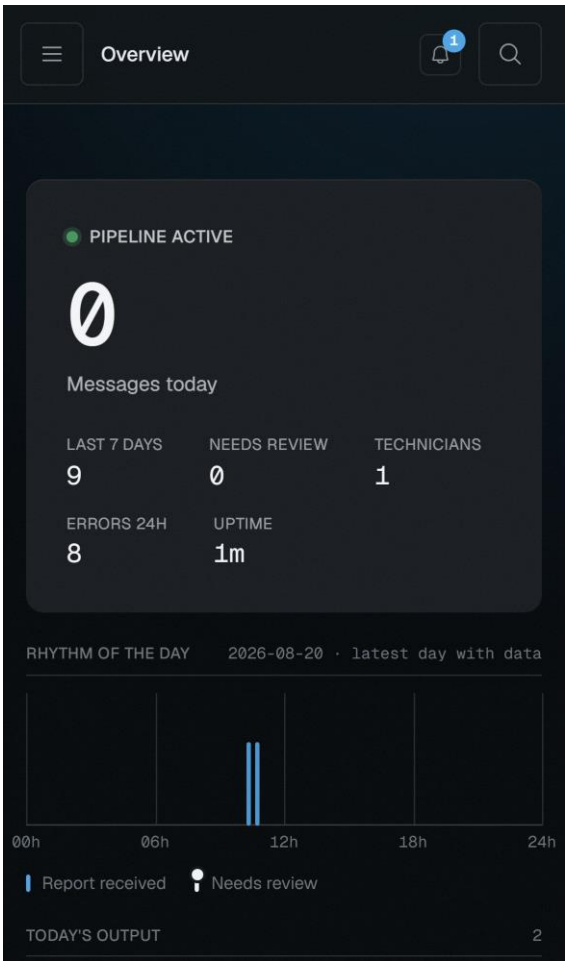


Figure C.9 : *Vue d'ensemble sur téléphone : la barre latérale se replie et les indicateurs passent sur une colonne.*